Design

- 1: Design
- 2: Protocol

1 - Design

Motivation

We designed Kafka to be able to act as a unified platform for handling all the real-time data feeds a large company might have. To do this we had to think through a fairly broad set of use cases.

It would have to have high-throughput to support high volume event streams such as real-time log aggregation.

It would need to deal gracefully with large data backlogs to be able to support periodic data loads from offline systems.

It also meant the system would have to handle low-latency delivery to handle more traditional messaging use-cases.

We wanted to support partitioned, distributed, real-time processing of these feeds to create new, derived feeds. This motivated our partitioning and consumer model.

Finally in cases where the stream is fed into other data systems for serving, we knew the system would have to be able to guarantee fault-tolerance in the presence of machine failures.

Supporting these uses led us to a design with a number of unique elements, more akin to a database log than a traditional messaging system. We will outline some elements of the design in the following sections.

Persistence

Don’t fear the filesystem!

Kafka relies heavily on the filesystem for storing and caching messages. There is a general perception that “disks are slow” which makes people skeptical that a persistent structure can offer competitive performance. In fact disks are both much slower and much faster than people expect depending on how they are used; and a properly designed disk structure can often be as fast as the network.

The key fact about disk performance is that the throughput of hard drives has been diverging from the latency of a disk seek for the last decade. As a result the performance of linear writes on a JBOD configuration with six 7200rpm SATA RAID-5 array is about 600MB/sec but the performance of random writes is only about 100kB/sec–a difference of over 6000X. These linear reads and writes are the most predictable of all usage patterns, and are heavily optimized by the operating system. A modern operating system provides read-ahead and write-behind techniques that prefetch data in large block multiples and group smaller logical writes into large physical writes. A further discussion of this issue can be found in this ACM Queue article; they actually find that sequential disk access can in some cases be faster than random memory access!

To compensate for this performance divergence, modern operating systems have become increasingly aggressive in their use of main memory for disk caching. A modern OS will happily divert *all* free memory to disk caching with little performance penalty when the memory is reclaimed. All disk reads and writes will go through this unified cache. This feature cannot easily be turned off without using direct I/O, so even if a process maintains an in-process cache of the data, this data will likely be duplicated in OS pagecache, effectively storing everything twice.

Furthermore, we are building on top of the JVM, and anyone who has spent any time with Java memory usage knows two things:

1. The memory overhead of objects is very high, often doubling the size of the data stored (or worse).
2. Java garbage collection becomes increasingly fiddly and slow as the in-heap data increases.

As a result of these factors using the filesystem and relying on pagecache is superior to maintaining an in-memory cache or other structure–we at least double the available cache by having automatic access to all free memory, and likely double again by storing a compact byte structure rather than individual objects. Doing so will result in a cache of up to 28-30GB on a 32GB machine without GC penalties. Furthermore, this cache will stay warm even if the service is restarted, whereas the in-process cache will need to be rebuilt in memory (which for a 10GB cache may take 10 minutes) or else it will need to start with a completely cold cache (which likely means terrible initial performance). This also greatly simplifies the code as all logic for maintaining coherency between the cache and filesystem is now in the OS, which tends to do so more efficiently and more correctly than one-off in-process attempts. If your disk usage favors linear reads then read-ahead is effectively pre-populating this cache with useful data on each disk read.

This suggests a design which is very simple: rather than maintain as much as possible in-memory and flush it all out to the filesystem in a panic when we run out of space, we invert that. All data is immediately written to a persistent log on the filesystem without necessarily flushing to disk. In effect this just means that it is transferred into the kernel's pagecache.

This style of pagecache-centric design is described in an article on the design of Varnish here (along with a healthy dose of arrogance).

Constant Time Suffices

The persistent data structure used in messaging systems are often a per-consumer queue with an associated BTree or other general-purpose random access data structures to maintain metadata about messages. BTrees are the most versatile data structure available, and make it possible to support a wide variety of transactional and non-transactional semantics in the messaging system. They do come with a fairly high cost, though: Btree operations are O(log N). Normally O(log N) is considered essentially equivalent to constant time, but this is not true for disk operations. Disk seeks come at 10 ms a pop, and each disk can do only one seek at a time so parallelism is limited. Hence even a handful of disk seeks leads to very high overhead. Since storage systems mix very fast cached operations with very slow physical disk operations, the observed performance of tree structures is often superlinear as data increases with fixed cache–i.e. doubling your data makes things much worse than twice as slow.

Intuitively a persistent queue could be built on simple reads and appends to files as is commonly the case with logging solutions. This structure has the advantage that all

operations are O(1) and reads do not block writes or each other. This has obvious performance advantages since the performance is completely decoupled from the data size–one server can now take full advantage of a number of cheap, low-rotational speed 1+TB SATA drives. Though they have poor seek performance, these drives have acceptable performance for large reads and writes and come at 1/3 the price and 3x the capacity.

Having access to virtually unlimited disk space without any performance penalty means that we can provide some features not usually found in a messaging system. For example, in Kafka, instead of attempting to delete messages as soon as they are consumed, we can retain messages for a relatively long period (say a week). This leads to a great deal of flexibility for consumers, as we will describe.

Efficiency

We have put significant effort into efficiency. One of our primary use cases is handling web activity data, which is very high volume: each page view may generate dozens of writes. Furthermore, we assume each message published is read by at least one consumer (often many), hence we strive to make consumption as cheap as possible.

We have also found, from experience building and running a number of similar systems, that efficiency is a key to effective multi-tenant operations. If the downstream infrastructure service can easily become a bottleneck due to a small bump in usage by the application, such small changes will often create problems. By being very fast we help ensure that the application will tip over under load before the infrastructure. This is particularly important when trying to run a centralized service that supports dozens or hundreds of applications on a centralized cluster as changes in usage patterns are a near-daily occurrence.

We discussed disk efficiency in the previous section. Once poor disk access patterns have been eliminated, there are two common causes of inefficiency in this type of system: too many small I/O operations, and excessive byte copying.

The small I/O problem happens both between the client and the server and in the server's own persistent operations.

To avoid this, our protocol is built around a "message set" abstraction that naturally groups messages together. This allows network requests to group messages together and amortize the overhead of the network roundtrip rather than sending a single message at a time. The server in turn appends chunks of messages to its log in one go, and the consumer fetches

large linear chunks at a time.

This simple optimization produces orders of magnitude speed up. Batching leads to larger network packets, larger sequential disk operations, contiguous memory blocks, and so on, all of which allows Kafka to turn a bursty stream of random message writes into linear writes that flow to the consumers.

The other inefficiency is in byte copying. At low message rates this is not an issue, but under load the impact is significant. To avoid this we employ a standardized binary message format that is shared by the producer, the broker, and the consumer (so data chunks can be transferred without modification between them).

The message log maintained by the broker is itself just a directory of files, each populated by a sequence of message sets that have been written to disk in the same format used by the producer and consumer. Maintaining this common format allows optimization of the most important operation: network transfer of persistent log chunks. Modern Unix operating systems offer a highly optimized code path for transferring data out of pagecache to a socket; in Linux this is done with the sendfile system call.

To understand the impact of sendfile, it is important to understand the common data path for transfer of data from file to socket:

1. The operating system reads data from the disk into pagecache in kernel space
2. The application reads the data from kernel space into a user-space buffer
3. The application writes the data back into kernel space into a socket buffer
4. The operating system copies the data from the socket buffer to the NIC buffer where it is sent over the network

This is clearly inefficient, there are four copies and two system calls. Using sendfile, this re-copying is avoided by allowing the OS to send the data from pagecache to the network directly. So in this optimized path, only the final copy to the NIC buffer is needed.

We expect a common use case to be multiple consumers on a topic. Using the zero-copy optimization above, data is copied into pagecache exactly once and reused on each consumption instead of being stored in memory and copied out to user-space every time it is read. This allows messages to be consumed at a rate that approaches the limit of the network connection.

This combination of pagecache and sendfile means that on a Kafka cluster where the consumers are mostly caught up you will see no read activity on the disks whatsoever as they

will be serving data entirely from cache.

TLS/SSL libraries operate at the user space (in-kernel `SSL_sendfile` is currently not supported by Kafka). Due to this restriction, `sendfile` is not used when SSL is enabled. For enabling SSL configuration, refer to `security.protocol` and `security.inter.broker.protocol`

For more background on the sendfile and zero-copy support in Java, see this article.

End-to-end Batch Compression

In some cases the bottleneck is actually not CPU or disk but network bandwidth. This is particularly true for a data pipeline that needs to send messages between data centers over a wide-area network. Of course, the user can always compress its messages one at a time without any support needed from Kafka, but this can lead to very poor compression ratios as much of the redundancy is due to repetition between messages of the same type (e.g. field names in JSON or user agents in web logs or common string values). Efficient compression requires compressing multiple messages together rather than compressing each message individually.

Kafka supports this with an efficient batching format. A batch of messages can be grouped together, compressed, and sent to the server in this form. The broker decompresses the batch in order to validate it. For example, it validates that the number of records in the batch is same as what batch header states. This batch of messages is then written to disk in compressed form. The batch will remain compressed in the log and it will also be transmitted to the consumer in compressed form. The consumer decompresses any compressed data that it receives.

Kafka supports GZIP, Snappy, LZ4 and ZStandard compression protocols. More details on compression can be found here.

The Producer

Load balancing

The producer sends data directly to the broker that is the leader for the partition without any intervening routing tier. To help the producer do this all Kafka nodes can answer a request for metadata about which servers are alive and where the leaders for the partitions of a topic

are at any given time to allow the producer to appropriately direct its requests.

The client controls which partition it publishes messages to. This can be done at random, implementing a kind of random load balancing, or it can be done by some semantic partitioning function. We expose the interface for semantic partitioning by allowing the user to specify a key to partition by and using this to hash to a partition (there is also an option to override the partition function if need be). For example if the key chosen was a user id then all data for a given user would be sent to the same partition. This in turn will allow consumers to make locality assumptions about their consumption. This style of partitioning is explicitly designed to allow locality-sensitive processing in consumers.

Asynchronous send

Batching is one of the big drivers of efficiency, and to enable batching the Kafka producer will attempt to accumulate data in memory and to send out larger batches in a single request. The batching can be configured to accumulate no more than a fixed number of messages and to wait no longer than some fixed latency bound (say 64kB or 10 ms). This allows the accumulation of more bytes to send, and few larger I/O operations on the servers. This buffering is configurable and gives a mechanism to trade off a small amount of additional latency for better throughput.

Details on configuration and the api for the producer can be found elsewhere in the documentation.

The Consumer

The Kafka consumer works by issuing “fetch” requests to the brokers leading the partitions it wants to consume. The consumer specifies its offset in the log with each request and receives back a chunk of log beginning from that position. The consumer thus has significant control over this position and can rewind it to re-consume data if need be.

Push vs. pull

An initial question we considered is whether consumers should pull data from brokers or brokers should push data to the consumer. In this respect Kafka follows a more traditional design, shared by most messaging systems, where data is pushed to the broker from the producer and pulled from the broker by the consumer. Some logging-centric systems, such

as Scribe and Apache Flume, follow a very different push-based path where data is pushed downstream. There are pros and cons to both approaches. However, a push-based system has difficulty dealing with diverse consumers as the broker controls the rate at which data is transferred. The goal is generally for the consumer to be able to consume at the maximum possible rate; unfortunately, in a push system this means the consumer tends to be overwhelmed when its rate of consumption falls below the rate of production (a denial of service attack, in essence). A pull-based system has the nicer property that the consumer simply falls behind and catches up when it can. This can be mitigated with some kind of backoff protocol by which the consumer can indicate it is overwhelmed, but getting the rate of transfer to fully utilize (but never over-utilize) the consumer is trickier than it seems. Previous attempts at building systems in this fashion led us to go with a more traditional pull model.

Another advantage of a pull-based system is that it lends itself to aggressive batching of data sent to the consumer. A push-based system must choose to either send a request immediately or accumulate more data and then send it later without knowledge of whether the downstream consumer will be able to immediately process it. If tuned for low latency, this will result in sending a single message at a time only for the transfer to end up being buffered anyway, which is wasteful. A pull-based design fixes this as the consumer always pulls all available messages after its current position in the log (or up to some configurable max size). So one gets optimal batching without introducing unnecessary latency.

The deficiency of a naive pull-based system is that if the broker has no data the consumer may end up polling in a tight loop, effectively busy-waiting for data to arrive. To avoid this we have parameters in our pull request that allow the consumer request to block in a “long poll” waiting until data arrives (and optionally waiting until a given number of bytes is available to ensure large transfer sizes).

You could imagine other possible designs which would be only pull, end-to-end. The producer would locally write to a local log, and brokers would pull from that with consumers pulling from them. A similar type of “store-and-forward” producer is often proposed. This is intriguing but we felt not very suitable for our target use cases which have thousands of producers. Our experience running persistent data systems at scale led us to feel that involving thousands of disks in the system across many applications would not actually make things more reliable and would be a nightmare to operate. And in practice we have found that we can run a pipeline with strong SLAs at large scale without a need for producer persistence.

Consumer Position

Keeping track of *what* has been consumed is, surprisingly, one of the key performance points of a messaging system.

Most messaging systems keep metadata about what messages have been consumed on the broker. That is, as a message is handed out to a consumer, the broker either records that fact locally immediately or it may wait for acknowledgement from the consumer. This is a fairly intuitive choice, and indeed for a single machine server it is not clear where else this state could go. Since the data structures used for storage in many messaging systems scale poorly, this is also a pragmatic choice–since the broker knows what is consumed it can immediately delete it, keeping the data size small.

What is perhaps not obvious is that getting the broker and consumer to come into agreement about what has been consumed is not a trivial problem. If the broker records a message as **consumed** immediately every time it is handed out over the network, then if the consumer fails to process the message (say because it crashes or the request times out or whatever) that message will be lost. To solve this problem, many messaging systems add an acknowledgement feature which means that messages are only marked as **sent** not **consumed** when they are sent; the broker waits for a specific acknowledgement from the consumer to record the message as **consumed**. This strategy fixes the problem of losing messages, but creates new problems. First of all, if the consumer processes the message but fails before it can send an acknowledgement then the message will be consumed twice. The second problem is around performance, now the broker must keep multiple states about every single message (first to lock it so it is not given out a second time, and then to mark it as permanently consumed so that it can be removed). Tricky problems must be dealt with, like what to do with messages that are sent but never acknowledged.

Kafka handles this differently. Our topic is divided into a set of totally ordered partitions, each of which is consumed by exactly one consumer within each subscribing consumer group at any given time. This means that the position of a consumer in each partition is just a single integer, the offset of the next message to consume. This makes the state about what has been consumed very small, just one number for each partition. This state can be periodically checkpointed. This makes the equivalent of message acknowledgements very cheap.

There is a side benefit of this decision. A consumer can deliberately *rewind* back to an old offset and re-consume data. This violates the common contract of a queue, but turns out to be an essential feature for many consumers. For example, if the consumer code has a bug

and is discovered after some messages are consumed, the consumer can re-consume those messages once the bug is fixed.

Offline Data Load

Scalable persistence allows for the possibility of consumers that only periodically consume such as batch data loads that periodically bulk-load data into an offline system such as Hadoop or a relational data warehouse.

In the case of Hadoop we parallelize the data load by splitting the load over individual map tasks, one for each node/topic/partition combination, allowing full parallelism in the loading. Hadoop provides the task management, and tasks which fail can restart without danger of duplicate data–they simply restart from their original position.

Static Membership

Static membership aims to improve the availability of stream applications, consumer groups and other applications built on top of the group rebalance protocol. The rebalance protocol relies on the group coordinator to allocate entity ids to group members. These generated ids are ephemeral and will change when members restart and rejoin. For consumer based apps, this “dynamic membership” can cause a large percentage of tasks re-assigned to different instances during administrative operations such as code deploys, configuration updates and periodic restarts. For large state applications, shuffled tasks need a long time to recover their local states before processing and cause applications to be partially or entirely unavailable. Motivated by this observation, Kafka’s group management protocol allows group members to provide persistent entity ids. Group membership remains unchanged based on those ids, thus no rebalance will be triggered.

If you want to use static membership,

- Upgrade both broker cluster and client apps to 2.3 or beyond, and also make sure the upgraded brokers are using `inter.broker.protocol.version` of 2.3 or beyond as well.
- Set the config `ConsumerConfig#GROUP_INSTANCE_ID_CONFIG` to a unique value for each consumer instance under one group.
- For Kafka Streams applications, it is sufficient to set a unique `ConsumerConfig#GROUP_INSTANCE_ID_CONFIG` per KafkaStreams instance, independent of the number of used threads for an instance.

If your broker is on an older version than 2.3, but you choose to set

`ConsumerConfig#GROUP_INSTANCE_ID_CONFIG` on the client side, the application will detect the broker version and then throws an UnsupportedException. If you accidentally configure duplicate ids for different instances, a fencing mechanism on broker side will inform your duplicate client to shutdown immediately by triggering a `org.apache.kafka.common.errors.FencedInstanceIdException`. For more details, see KIP-345

Message Delivery Semantics

Now that we understand a little about how producers and consumers work, let's discuss the semantic guarantees Kafka provides between producer and consumer. Clearly there are multiple possible message delivery guarantees that could be provided:

- *At most once* –Messages may be lost but are never redelivered.
- *At least once* –Messages are never lost but may be redelivered.
- *Exactly once* –Each message is processed once and only once.

It's worth noting that this breaks down into two problems: the durability guarantees for publishing a message and the guarantees when consuming a message.

Many systems claim to provide "exactly-once" delivery semantics, but it is important to read the fine print, because sometimes these claims are misleading (i.e. they don't translate to the case where consumers or producers can fail, cases where there are multiple consumer processes, or cases where data written to disk can be lost).

Kafka's semantics are straightforward. When publishing a message we have a notion of the message being "committed" to the log. A message is considered committed only when all replicas in the in-sync replicas (ISR) for that partition have applied it to their log. Once a published message is committed, it will not be lost as long as one broker that replicates the partition to which this message was written remains "alive". The definition of committed message and alive partition as well as a description of which types of failures we attempt to handle will be described in more detail in the next section. For now let's assume a perfect, lossless broker and try to understand the guarantees to the producer and consumer. If a producer attempts to publish a message and experiences a network error, it cannot be sure if this error happened before or after the message was committed. This is similar to the semantics of inserting into a database table with an autogenerated key.

Prior to 0.11.0.0, if a producer failed to receive a response indicating that a message was committed, it had little choice but to resend the message. This provides at-least-once delivery

semantics since the message may be written to the log again during resending if the original request had in fact succeeded. Since 0.11.0.0, the Kafka producer also supports an idempotent delivery option which guarantees that resending will not result in duplicate entries in the log. To achieve this, the broker assigns each producer an ID and deduplicates messages using a sequence number that is sent by the producer along with every message. Also beginning with 0.11.0.0, the producer supports the ability to send messages atomically to multiple topic partitions using transactions, so that either all messages are successfully written or none of them are.

Not all use cases require such strong guarantees. For use cases which are latency-sensitive, we allow the producer to specify the durability level it desires. If the producer specifies that it wants to wait on the message being committed, this can take on the order of 10 ms. However the producer can also specify that it wants to perform the send completely asynchronously or that it wants to wait only until the leader (but not necessarily the followers) have the message.

Now let's describe the semantics from the point of view of the consumer. All replicas have the exact same log with the same offsets. The consumer controls its position in this log. If the consumer never crashed it could just store this position in memory, but if the consumer fails and we want this topic partition to be taken over by another process, the new process will need to choose an appropriate position from which to start processing. Let's say the consumer reads some messages – it has several options for processing the messages and updating its position.

1. It can read the messages, then save its position in the log, and finally process the messages. In this case there is a possibility that the consumer process crashes after saving its position but before saving the output of its message processing. In this case the process that took over processing would start at the saved position even though a few messages prior to that position had not been processed. This corresponds to "at-most-once" semantics as in the case of a consumer failure messages may not be processed.
2. It can read the messages, process the messages, and finally save its position. In this case there is a possibility that the consumer process crashes after processing messages but before saving its position. In this case when the new process takes over the first few messages it receives will already have been processed. This corresponds to the "at-least-once" semantics in the case of consumer failure. In many cases messages have a primary key and so the updates are idempotent (receiving the same message twice just overwrites a record with another copy of itself).

So what about exactly-once semantics? When consuming from a Kafka topic and producing to another topic (as in a Kafka Streams application), we can leverage the new transactional producer capabilities in 0.11.0.0 that were mentioned above. The consumer's position is stored as a message in an internal topic, so we can write the offset to Kafka in the same transaction as the output topics receiving the processed data. If the transaction is aborted, the consumer's stored position will revert to its old value (although the consumer has to refetch the committed offset because it does not automatically rewind) and the produced data on the output topics will not be visible to other consumers, depending on their "isolation level". In the default "read_uncommitted" isolation level, all messages are visible to consumers even if they were part of an aborted transaction, but in "read_committed" isolation level, the consumer will only return messages from transactions which were committed (and any messages which were not part of a transaction).

When writing to an external system, the limitation is in the need to coordinate the consumer's position with what is actually stored as output. The classic way of achieving this would be to introduce a two-phase commit between the storage of the consumer position and the storage of the consumers output. This can be handled more simply and generally by letting the consumer store its offset in the same place as its output. This is better because many of the output systems a consumer might want to write to will not support a two-phase commit. As an example of this, consider a Kafka Connect connector which populates data in HDFS along with the offsets of the data it reads so that it is guaranteed that either data and offsets are both updated or neither is. We follow similar patterns for many other data systems which require these stronger semantics and for which the messages do not have a primary key to allow for deduplication.

As a result, Kafka supports exactly-once delivery in Kafka Streams, and the transactional producer and the consumer using read-committed isolation level can be used generally to provide exactly-once delivery when reading, processing and writing data on Kafka topics. Exactly-once delivery for other destination systems generally requires cooperation with such systems, but Kafka provides the primitives which makes implementing this feasible (see also Kafka Connect). Otherwise, Kafka guarantees at-least-once delivery by default, and allows the user to implement at-most-once delivery by disabling retries on the producer and committing offsets in the consumer prior to processing a batch of messages.

Using Transactions

As mentioned above, the simplest way to get exactly-once semantics from Kafka is to use

Kafka Streams. However, it is also possible to achieve the same transactional guarantees using the Kafka producer and consumer directly by using them in the same way as Kafka Streams does.

Kafka transactions are a bit different from transactions in other messaging systems. In Kafka, the consumer and producer are separate, and it is only the producer which is transactional. It is however able to make transactional updates to the consumer's position (confusingly called the "committed offset"), and it is this which gives the overall exactly-once behavior.

There are three key aspects to exactly-once processing using the producer and consumer, which match how Kafka Streams works.

1. The consumer uses partition assignment to ensure that it is the only consumer in the consumer group currently processing each partition.
2. The producer uses transactions so that all the records it produces, and any offsets it updates on behalf of the consumer, are performed atomically.
3. In order to handle transactions properly in combination with rebalancing, it is advisable to use one producer instance for each consumer instance. More complicated and efficient schemes are possible, but at the cost of greater complexity.

In addition, it is generally considered a good practice to use the read-committed isolation level if trying to achieve exactly-once processing. Strictly speaking, the consumer doesn't have to use read-committed isolation level, but if it does not, it will see records from aborted transactions and also open transactions which have not yet completed.

The consumer configuration must include `isolation.level=read_committed` and `enable.auto.commit=false`. The producer configuration must set `transactional.id` to the name of the transactional ID to be used, which configures the producer for transactional delivery and also makes sure that a restarted application causes any in-flight transaction from the previous instance to abort. Only the producer has the `transactional.id` configuration.

Here's an example of a transactional message copier which uses these principles. It uses a `KafkaConsumer` to consume records from one topic and a `KafkaProducer` to produce records to another topic. It uses transactions to ensure that there is no duplication or loss of records as they are copied, provided that the `--use-group-metadata` option is set.

It is important to handle exceptions and aborted transactions correctly. Any records written by the transactional producer will be marked as being part of the transactions, and then when the transaction commits or aborts, transaction marker records are written to indicate

the outcome of the transaction. This is how the read-committed consumer does not see records from aborted transactions. However, in the event of a transaction abort, the application's state and in particular the current position of the consumer must be reset explicitly so that it can reprocess the records processed by the aborted transaction.

The error handling for transactional producer has been standardized which ensures consistent behavior and clearer error handling patterns. The exception categories are now more precisely defined:

1. **RetriableException** : Temporary exceptions that are retried automatically by the client. These are handled internally and don't bubble up to the application.
2. **RefreshRetriableException** : Exceptions requiring metadata refresh before retry. These are handled internally by the client after refreshing metadata and don't bubble up to the application.
3. **AbortableException** : Exceptions that require transaction abort and reprocessing. These bubble up to the application, which must handle them by aborting the transaction and resetting the consumer position.
4. **ApplicationRecoverableException** : Exceptions that bubble up to the application and require application handling. The application must implement its own recovery strategy, which must include restarting the producer.
5. **InvalidConfigurationException** : Configuration-related exceptions that bubble up to the application and require application handling. The producer doesn't need to restart, but the application may choose to restart it.
6. **KafkaException** : General Kafka exceptions that don't fit into the above categories. These bubble up to the application for handling.

Example template code for handling transaction exceptions link : Transaction Client Demo

A simple policy for handling exceptions and aborted transactions is to discard and recreate the Kafka producer and consumer objects and start afresh. As part of recreating the consumer, the consumer group will rebalance and fetch the last committed offset, which has the effect of rewinding back to the state before the transaction aborted. Alternatively, a more sophisticated application (such as the transactional message copier) can choose not to use `KafkaConsumer.committed` to retrieve the committed offset from Kafka, and then `KafkaConsumer.seek` to rewind the current position.

The Share Consumer

Most consumption of data from Kafka is performed by consumers in consumer groups. The way that consumer groups assign partitions to members of the group gives a powerful combination of ordering and scalability. However, some traditional messaging workloads are not a good fit for consumer groups.

Share groups are another type of group, existing alongside traditional consumer groups. They offer an alternative to consumer groups, particularly when applications require finer-grained sharing of partitions and records. Share groups enable consumers to cooperatively consume and process records from topics. The consumers in a share group are called share consumers, and they use a familiar but different programming interface.

The fundamental differences between a share group and a consumer group are:

- The consumers in a share group cooperatively consume records, and partitions may be assigned to multiple consumers.
- The number of consumers in a share group can exceed the number of partitions in a topic.
- Records are acknowledged individually, though the system is optimized for batch processing to improve efficiency.
- Delivery attempts to consumers in a share group are counted, which enables automated handling of unprocessable records.

All share consumers in the same share group subscribed to the same topic will cooperatively consume the records of that topic. If a topic is accessed by consumers in multiple share groups, each share group consumes from that topic independently of the others.

Each share consumer can dynamically set its list of subscribed topics. In practice, all consumers in a share group typically subscribe to the same topic or topics.

When a share consumer fetches records, it receives available records from any of the topic-partitions matching its subscriptions. Records are acquired for delivery to this share consumer with a time-limited acquisition lock. While a record is acquired, it is unavailable to other consumers in the same share group for the duration of the lock.

By default, the lock duration is 30 seconds, but you can control it using the group configuration parameter `share.record.lock.duration.ms`. The lock is released automatically once its duration elapses, making the record available to another delivery attempt. A share consumer holding the lock can handle the record in the following ways:

- Acknowledge successful processing of the record.

- Release the record, making it available for another delivery attempt.
- Reject the record, indicating it's unprocessable and preventing further delivery attempts for that record.
- Renew the record, extending the delivery attempt because the record is still being processed.
- Do nothing, in which case the lock is automatically released when its duration elapses.

The Kafka cluster limits the number of records for each topic-partition acquired by share consumers in each share group. Once this limit is reached, fetching operations will temporarily yield no further records until the number of acquired records decreases (as locks naturally time out). This limit is controlled by the broker configuration property `group.share.partition.max.record.locks`. By limiting the duration of the acquisition lock and automatically releasing the locks, the broker ensures delivery progresses even in the presence of consumer failures.

Replication

Kafka replicates the log for each topic's partitions across a configurable number of servers (you can set this replication factor on a topic-by-topic basis). This allows automatic failover to these replicas when a server in the cluster fails so messages remain available in the presence of failures.

Other messaging systems provide some replication-related features, but, in our (totally biased) opinion, this appears to be a tacked-on thing, not heavily used, and with large downsides: replicas are inactive, throughput is heavily impacted, it requires fiddly manual configuration, etc. Kafka is meant to be used with replication by default–in fact we implement un-replicated topics as replicated topics where the replication factor is one.

The unit of replication is the topic partition. Under non-failure conditions, each partition in Kafka has a single leader and zero or more followers. The total number of replicas including the leader constitute the replication factor. All writes go to the leader of the partition, and reads can go to the leader or the followers of the partition. Typically, there are many more partitions than brokers and the leaders are evenly distributed among brokers. The logs on the followers are identical to the leader's log–all have the same offsets and messages in the same order (though, of course, at any given time the leader may have a few as-yet unreplicated messages at the end of its log).

Followers consume messages from the leader just as a normal Kafka consumer would and

apply them to their own log. Having the followers pull from the leader has the nice property of allowing the follower to naturally batch together log entries they are applying to their log.

As with most distributed systems, automatically handling failures requires a precise definition of what it means for a node to be “alive.” In Kafka, a special node known as the “controller” is responsible for managing the registration of brokers in the cluster. Broker liveness has two conditions:

1. Brokers must maintain an active session with the controller in order to receive regular metadata updates.
2. Brokers acting as followers must replicate the writes from the leader and not fall “too far” behind.

What is meant by an “active session” depends on the cluster configuration. For KRaft clusters, an active session is maintained by sending periodic heartbeats to the controller. If the controller fails to receive a heartbeat before the timeout configured by `broker.session.timeout.ms` expires, then the node is considered offline.

We refer to nodes satisfying these two conditions as being “in sync” to avoid the vagueness of “alive” or “failed”. The leader keeps track of the set of “in sync” replicas, which is known as the ISR. If either of these conditions fail to be satisfied, then the broker will be removed from the ISR. For example, if a follower dies, then the controller will notice the failure through the loss of its session, and will remove the broker from the ISR. On the other hand, if the follower lags too far behind the leader but still has an active session, then the leader can also remove it from the ISR. The determination of lagging replicas is controlled through the `replica.lag.time.max.ms` configuration. Replicas that cannot catch up to the end of the log on the leader within the max time set by this configuration are removed from the ISR.

In distributed systems terminology we only attempt to handle a “fail/recover” model of failures where nodes suddenly cease working and then later recover (perhaps without knowing that they have died). Kafka does not handle so-called “Byzantine” failures in which nodes produce arbitrary or malicious responses (perhaps due to bugs or foul play).

Only committed messages are ever given out to the consumer. This means that the consumer need not worry about potentially seeing a message that could be lost if the leader fails. Producers, on the other hand, have the option of either waiting for the message to be committed or not, depending on their preference for tradeoff between latency and durability. This preference is controlled by the `acks` setting that the producer uses. Note that topics have a setting for the minimum number of in-sync replicas (`min.insync.replicas`) that is

checked when the producer requests acknowledgment that a message has been written to the full set of in-sync replicas. If a less stringent acknowledgment is requested by the producer, then the message is committed asynchronously across the set of in-sync replicas if `acks=0`, or synchronously only on the leader if `acks=1`. Regardless of the `acks` setting, the messages will not be visible to the consumers until all the following conditions are met:

1. The messages are replicated to all the in-sync replicas.
2. The number of the in-sync replicas is no less than the `min.insync.replicas` setting.

The guarantee that Kafka offers is that a committed message will not be lost, as long as there is at least one in sync replica alive, at all times.

Kafka will remain available in the presence of node failures after a short fail-over period, but may not remain available in the presence of network partitions.

Replicated Logs: Quorums, ISRs, and State Machines (Oh my!)

At its heart a Kafka partition is a replicated log. The replicated log is one of the most basic primitives in distributed data systems, and there are many approaches for implementing one. A replicated log can be used by other systems as a primitive for implementing other distributed systems in the state-machine style.

A replicated log models the process of coming into consensus on the order of a series of values (generally numbering the log entries 0, 1, 2, ...). There are many ways to implement this, but the simplest and fastest is with a leader who chooses the ordering of values provided to it. As long as the leader remains alive, all followers need to only copy the values and ordering the leader chooses.

Of course if leaders didn't fail we wouldn't need followers! When the leader does die we need to choose a new leader from among the followers. But followers themselves may fall behind or crash so we must ensure we choose an up-to-date follower. The fundamental guarantee a log replication algorithm must provide is that if we tell the client a message is committed, and the leader fails, the new leader we elect must also have that message. This yields a tradeoff: if the leader waits for more followers to acknowledge a message before declaring it committed then there will be more potentially electable leaders.

If you choose the number of acknowledgements required and the number of logs that must be compared to elect a leader such that there is guaranteed to be an overlap, then this is called a Quorum.

A common approach to this tradeoff is to use a majority vote for both the commit decision and the leader election. This is not what Kafka does, but let's explore it anyway to understand the tradeoffs. Let's say we have 2 *f* +1 replicas. If *f* +1 replicas must receive a message prior to a commit being declared by the leader, and if we elect a new leader by electing the follower with the most complete log from at least *f* +1 replicas, then, with no more than *f* failures, the leader is guaranteed to have all committed messages. This is because among any *f* +1 replicas, there must be at least one replica that contains all committed messages. That replica's log will be the most complete and therefore will be selected as the new leader. There are many remaining details that each algorithm must handle (such as precisely defined what makes a log more complete, ensuring log consistency during leader failure or changing the set of servers in the replica set) but we will ignore these for now.

This majority vote approach has a very nice property: the latency is dependent on only the fastest servers. That is, if the replication factor is three, the latency is determined by the faster follower not the slower one.

There are a rich variety of algorithms in this family including ZooKeeper's Zab, Raft, and Viewstamped Replication. The most similar academic publication we are aware of to Kafka's actual implementation is PacificA from Microsoft.

The downside of majority vote is that it doesn't take many failures to leave you with no electable leaders. To tolerate one failure requires three copies of the data, and to tolerate two failures requires five copies of the data. In our experience having only enough redundancy to tolerate a single failure is not enough for a practical system, but doing every write five times, with 5x the disk space requirements and 1/5th the throughput, is not very practical for large volume data problems. This is likely why quorum algorithms more commonly appear for shared cluster configuration such as ZooKeeper but are less common for primary data storage. For example in HDFS the namenode's high-availability feature is built on a majority-vote-based journal, but this more expensive approach is not used for the data itself.

Kafka takes a slightly different approach to choosing its quorum set. Instead of majority vote, Kafka dynamically maintains a set of in-sync replicas (ISR) that are caught-up to the leader. Only members of this set are eligible for election as leader. A write to a Kafka partition is not considered committed until *all* in-sync replicas have received the write. This ISR set is persisted in the cluster metadata whenever it changes. Because of this, any replica in the ISR is eligible to be elected leader. This is an important factor for Kafka's usage model where there are many partitions and ensuring leadership balance is important. With this ISR model and *f+1* replicas, a Kafka topic can tolerate *f* failures without losing committed messages.

For most use cases we hope to handle, we think this tradeoff is a reasonable one. In practice, to tolerate *f* failures, both the majority vote and the ISR approach will wait for the same number of replicas to acknowledge before committing a message (e.g. to survive one failure a majority quorum needs three replicas and one acknowledgement and the ISR approach requires two replicas and one acknowledgement). The ability to commit without the slowest servers is an advantage of the majority vote approach. However, we think it is ameliorated by allowing the client to choose whether they block on the message commit or not, and the additional throughput and disk space due to the lower required replication factor is worth it.

Another important design distinction is that Kafka does not require that crashed nodes recover with all their data intact. It is not uncommon for replication algorithms in this space to depend on the existence of "stable storage" that cannot be lost in any failure-recovery scenario without potential consistency violations. There are two primary problems with this assumption. First, disk errors are the most common problem we observe in real operation of persistent data systems and they often do not leave data intact. Secondly, even if this were not a problem, we do not want to require the use of fsync on every write for our consistency guarantees as this can reduce performance by two to three orders of magnitude. Our protocol for allowing a replica to rejoin the ISR ensures that before rejoining, it must fully re-sync again even if it lost unflushed data in its crash.

Unclean leader election: What if they all die?

Note that Kafka's guarantee with respect to data loss is predicated on at least one replica remaining in sync. If all the nodes replicating a partition die, this guarantee no longer holds.

However a practical system needs to do something reasonable when all the replicas die. If you are unlucky enough to have this occur, it is important to consider what will happen. There are two behaviors that could be implemented:

1. Wait for a replica in the ISR to come back to life and choose this replica as the leader (hopefully it still has all its data).
2. Choose the first replica (not necessarily in the ISR) that comes back to life as the leader.

This is a simple tradeoff between availability and consistency. If we wait for replicas in the ISR, then we will remain unavailable as long as those replicas are down. If such replicas were destroyed or their data was lost, then we are permanently down. If, on the other hand, a non-in-sync replica comes back to life and we allow it to become leader, then its log becomes the source of truth even though it is not guaranteed to have every committed message. By default from version 0.11.0.0, Kafka chooses the first strategy and favor waiting for a

consistent replica. This behavior can be changed using configuration property `unclean.leader.election.enable`, to support use cases where uptime is preferable to consistency.

This dilemma is not specific to Kafka. It exists in any quorum-based scheme. For example in a majority voting scheme, if a majority of servers suffer a permanent failure, then you must either choose to lose 100% of your data or violate consistency by taking what remains on an existing server as your new source of truth.

Availability and Durability Guarantees

When writing to Kafka, producers can choose whether they wait for the message to be acknowledged by 0,1 or all (-1) replicas. Note that "acknowledgement by all replicas" does not guarantee that the full set of assigned replicas have received the message. By default, when acks=all, acknowledgement happens as soon as all the current in-sync replicas have received the message. For example, if a topic is configured with only two replicas and one fails (i.e., only one in sync replica remains), then writes that specify acks=all will succeed. However, these writes could be lost if the remaining replica also fails. Although this ensures maximum availability of the partition, this behavior may be undesirable to some users who prefer durability over availability. Therefore, we provide two topic configurations that can be used to prefer message durability over availability:

1. Disable unclean leader election - if all replicas become unavailable, then the partition will remain unavailable until the most recent leader becomes available again. This effectively prefers unavailability over the risk of message loss. See the previous section on Unclean Leader Election for clarification.
2. Specify a minimum ISR size - the partition will only accept writes if the size of the ISR is above a certain minimum, in order to prevent the loss of messages that were written to just a single replica, which subsequently becomes unavailable. This setting only takes effect if the producer uses acks=all and guarantees that the message will be acknowledged by at least this many in-sync replicas. This setting offers a trade-off between consistency and availability. A higher setting for minimum ISR size guarantees better consistency since the message is guaranteed to be written to more replicas which reduces the probability that it will be lost. However, it reduces availability since the partition will be unavailable for writes if the number of in-sync replicas drops below the minimum threshold.

Replica Management

The above discussion on replicated logs really covers only a single log, i.e. one topic partition. However a Kafka cluster will manage hundreds or thousands of these partitions. We attempt to balance partitions within a cluster in a round-robin fashion to avoid clustering all partitions for high-volume topics on a small number of nodes. Likewise we try to balance leadership so that each node is the leader for a proportional share of its partitions.

It is also important to optimize the leadership election process as that is the critical window of unavailability. A naive implementation of leader election would end up running an election per partition for all partitions a node hosted when that node failed. As discussed above in the section on replication, Kafka clusters have a special role known as the “controller” which is responsible for managing the registration of brokers. If the controller detects the failure of a broker, it is responsible for electing one of the remaining members of the ISR to serve as the new leader. The result is that we are able to batch together many of the required leadership change notifications which makes the election process far cheaper and faster for a large number of partitions. If the controller itself fails, then another controller will be elected.

Log Compaction

Log compaction ensures that Kafka will always retain at least the last known value for each message key within the log of data for a single topic partition. It addresses use cases and scenarios such as restoring state after application crashes or system failure, or reloading caches after application restarts during operational maintenance. Let’s dive into these use cases in more detail and then describe how compaction works.

So far we have described only the simpler approach to data retention where old log data is discarded after a fixed period of time or when the log reaches some predetermined size. This works well for temporal event data such as logging where each record stands alone. However an important class of data streams are the log of changes to keyed, mutable data (for example, the changes to a database table).

Let’s discuss a concrete example of such a stream. Say we have a topic containing user email addresses; every time a user updates their email address we send a message to this topic using their user id as the primary key. Now say we send the following messages over some time period for a user with id 123, each message corresponding to a change in email address (messages for other ids are omitted):

```
123 => bill@microsoft.com
```

```
        .
        .
        .
123 => bill@gatesfoundation.org
        .
        .
        .
123 => bill@gmail.com
```

Log compaction gives us a more granular retention mechanism so that we are guaranteed to retain at least the last update for each primary key (e.g. `bill@gmail.com`). By doing this we guarantee that the log contains a full snapshot of the final value for every key not just keys that changed recently. This means downstream consumers can restore their own state off this topic without us having to retain a complete log of all changes.

Let's start by looking at a few use cases where this is useful, then we'll see how it can be used.

1. *Database change subscription*. It is often necessary to have a data set in multiple data systems, and often one of these systems is a database of some kind (either a RDBMS or perhaps a new-fangled key-value store). For example you might have a database, a cache, a search cluster, and a Hadoop cluster. Each change to the database will need to be reflected in the cache, the search cluster, and eventually in Hadoop. In the case that one is only handling the real-time updates you only need recent log. But if you want to be able to reload the cache or restore a failed search node you may need a complete data set.
2. *Event sourcing*. This is a style of application design which co-locates query processing with application design and uses a log of changes as the primary store for the application.
3. *Journaling for high-availability*. A process that does local computation can be made fault-tolerant by logging out changes that it makes to its local state so another process can reload these changes and carry on if it should fail. A concrete example of this is handling counts, aggregations, and other "group by"-like processing in a stream query system. Samza, a real-time stream-processing framework, uses this feature for exactly this purpose. In each of these cases one needs primarily to handle the real-time feed of changes, but occasionally, when a machine crashes or data needs to be re-loaded or re-processed, one needs to do a full load. Log compaction allows feeding both of these use cases off the same backing topic. This style of usage of a log is described in more detail in this blog post.

The general idea is quite simple. If we had infinite log retention, and we logged each change in the above cases, then we would have captured the state of the system at each time from

when it first began. Using this complete log, we could restore to any point in time by replaying the first N records in the log. This hypothetical complete log is not very practical for systems that update a single record many times as the log will grow without bound even for a stable dataset. The simple log retention mechanism which throws away old updates will bound space but the log is no longer a way to restore the current state–now restoring from the beginning of the log no longer recreates the current state as old updates may not be captured at all.

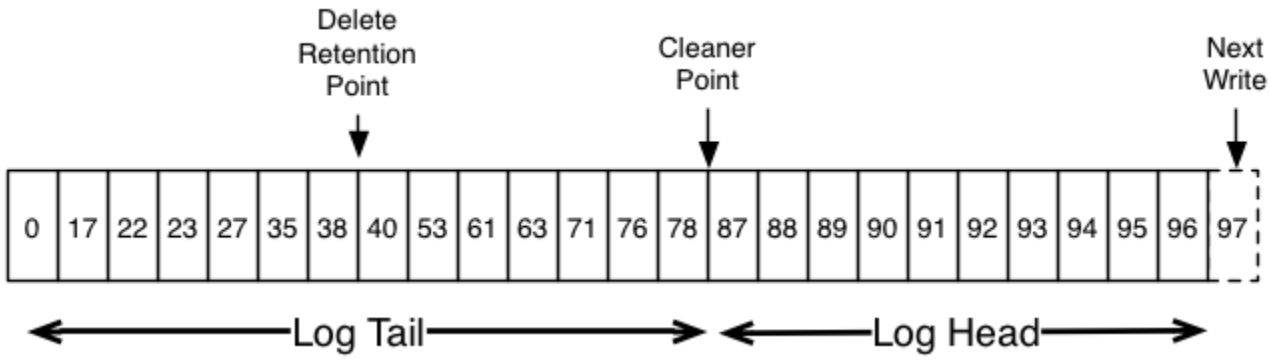
Log compaction is a mechanism to give finer-grained per-record retention, rather than the coarser-grained time-based retention. The idea is to selectively remove records where we have a more recent update with the same primary key. This way the log is guaranteed to have at least the last state for each key.

This retention policy can be set per-topic, so a single cluster can have some topics where retention is enforced by size or time and other topics where retention is enforced by compaction.

This functionality is inspired by one of LinkedIn's oldest and most successful pieces of infrastructure–a database changelog caching service called Databus. Unlike most log-structured storage systems Kafka is built for subscription and organizes data for fast linear reads and writes. Unlike Databus, Kafka acts as a source-of-truth store so it is useful even in situations where the upstream data source would not otherwise be replayable.

Log Compaction Basics

Here is a high-level picture that shows the logical structure of a Kafka log with the offset for each message.

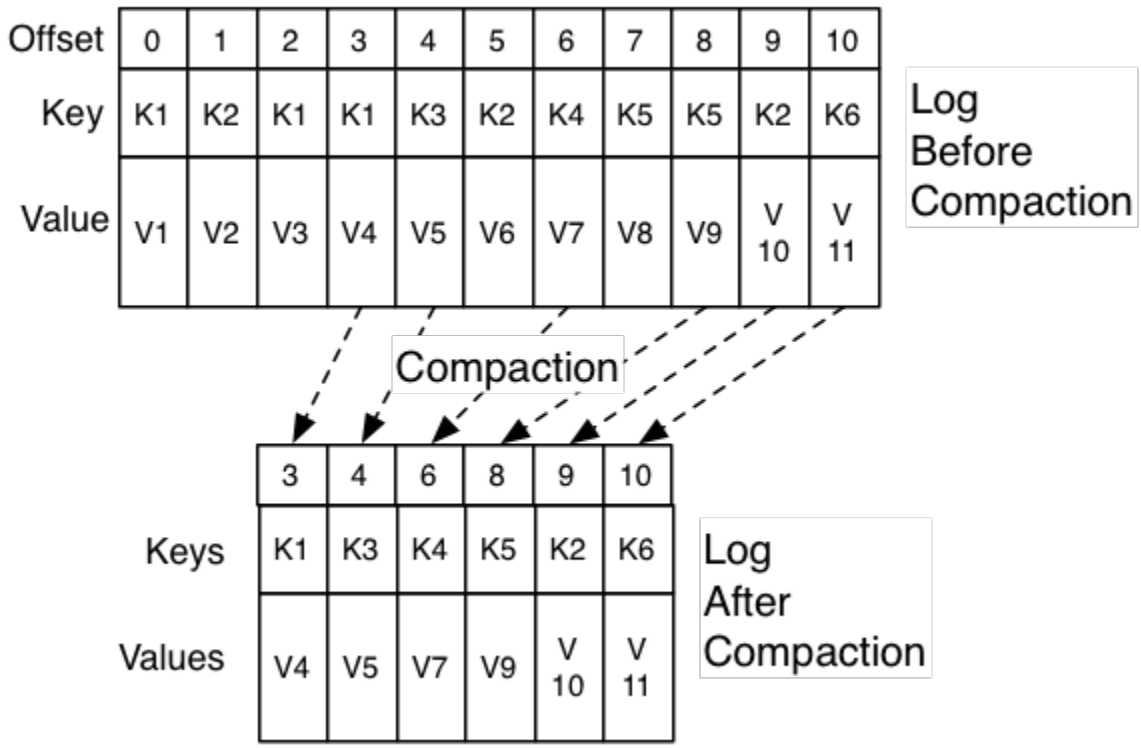


The head of the log is identical to a traditional Kafka log. It has dense, sequential offsets and retains all messages. Log compaction adds an option for handling the tail of the log. The

picture above shows a log with a compacted tail. Note that the messages in the tail of the log retain the original offset assigned when they were first written–that never changes. Note also that all offsets remain valid positions in the log, even if the message with that offset has been compacted away; in this case this position is indistinguishable from the next highest offset that does appear in the log. For example, in the picture above the offsets 36, 37, and 38 are all equivalent positions and a read beginning at any of these offsets would return a message set beginning with 38.

Compaction also allows for deletes. A message with a key and a null payload will be treated as a delete from the log. Such a record is sometimes referred to as a *tombstone*. This delete marker will cause any prior message with that key to be removed (as would any new message with that key), but delete markers are special in that they will themselves be cleaned out of the log after a period of time to free up space. The point in time at which deletes are no longer retained is marked as the "delete retention point" in the above diagram.

The compaction is done in the background by periodically recopying log segments. Cleaning does not block reads and can be throttled to use no more than a configurable amount of I/O throughput to avoid impacting producers and consumers. The actual process of compacting a log segment looks something like this:



What guarantees does log compaction provide?

Log compaction guarantees the following:

1. Any consumer that stays caught-up to within the head of the log will see every message that is written; these messages will have sequential offsets. The topic's `min.compaction.lag.ms` can be used to guarantee the minimum length of time must pass after a message is written before it could be compacted, that is, it provides a lower bound on how long each message will remain in the (uncompacted) head. The topic's `max.compaction.lag.ms` can be used to guarantee the maximum delay between the time a message is written and the time the message becomes eligible for compaction.
2. Ordering of messages is always maintained. Compaction will never re-order messages, just remove some.
3. The offset for a message never changes. It is the permanent identifier for a position in the log.
4. Any consumer progressing from the start of the log will see at least the final state of all records in the order they were written. Additionally, all delete markers for deleted records will be seen, provided the consumer reaches the head of the log in a time period less than the topic's `delete.retention.ms` setting (the default is 24 hours). In other words: since the removal of delete markers happens concurrently with reads, it is possible for a consumer to miss delete markers if it lags by more than `delete.retention.ms`.

Log Compaction Details

Log compaction is handled by the log cleaner, a pool of background threads that recopy log segment files, removing records whose key appears in the head of the log. Each compactor thread works as follows:

1. It chooses the log that has the highest ratio of log head to log tail
2. It creates a succinct summary of the last offset for each key in the head of the log
3. It recopies the log from beginning to end removing keys which have a later occurrence in the log. New, clean segments are swapped into the log immediately so the additional disk space required is just one additional log segment (not a full copy of the log).
4. The summary of the log head is essentially just a space-compact hash table. It uses exactly 24 bytes per entry. As a result with 8GB of cleaner buffer one cleaner iteration can clean around 366GB of log head (assuming 1kB messages).

Configuring The Log Cleaner

The log cleaner is enabled by default. This will start the pool of cleaner threads. To enable log cleaning on a particular topic, add the log-specific property

```
log.cleanup.policy=compact
```

The `log.cleanup.policy` property is a broker configuration setting defined in the broker's `server.properties` file; it affects all of the topics in the cluster that do not have a configuration override in place as documented here. The log cleaner can be configured to retain a minimum amount of the uncompacted "head" of the log. This is enabled by setting the compaction time lag.

```
log.cleaner.min.compaction.lag.ms
```

This can be used to prevent messages newer than a minimum message age from being subject to compaction. If not set, all log segments are eligible for compaction except for the last segment, i.e. the one currently being written to. The active segment will not be compacted even if all of its messages are older than the minimum compaction time lag. The log cleaner can be configured to ensure a maximum delay after which the uncompacted "head" of the log becomes eligible for log compaction.

```
log.cleaner.max.compaction.lag.ms
```

This can be used to prevent log with low produce rate from remaining ineligible for compaction for an unbounded duration. If not set, logs that do not exceed min.cleanable.dirty.ratio are not compacted. Note that this compaction deadline is not a hard guarantee since it is still subjected to the availability of log cleaner threads and the actual compaction time. You will want to monitor the uncleanable-partitions-count, max-clean-time-secs and max-compaction-delay-secs metrics.

Further cleaner configurations are described here.

Quotas

Kafka cluster has the ability to enforce quotas on requests to control the broker resources used by clients. Two types of client quotas can be enforced by Kafka brokers for each group of clients sharing a quota:

1. Network bandwidth quotas define byte-rate thresholds (since 0.9)
2. Request rate quotas define CPU utilization thresholds as a percentage of network and I/O threads (since 0.11)

Why are quotas necessary?

It is possible for producers and consumers to produce/consume very high volumes of data or generate requests at a very high rate and thus monopolize broker resources, cause network saturation and generally DOS other clients and the brokers themselves. Having quotas protects against these issues and is all the more important in large multi-tenant clusters where a small set of badly behaved clients can degrade user experience for the well behaved ones. In fact, when running Kafka as a service this even makes it possible to enforce API limits according to an agreed upon contract.

Client groups

The identity of Kafka clients is the user principal which represents an authenticated user in a secure cluster. In a cluster that supports unauthenticated clients, user principal is a grouping of unauthenticated users chosen by the broker using a configurable `PrincipalBuilder`. Client-id is a logical grouping of clients with a meaningful name chosen by the client application. The tuple (user, client-id) defines a secure logical group of clients that share both user principal and client-id.

Quotas can be applied to (user, client-id), user or client-id groups. For a given connection, the most specific quota matching the connection is applied. All connections of a quota group share the quota configured for the group. For example, if (user="test-user", client-id="test-client") has a produce quota of 10MB/sec, this is shared across all producer instances of user "test-user" with the client-id "test-client".

Quota Configuration

Quota configuration may be defined for (user, client-id), user and client-id groups. It is possible to override the default quota at any of the quota levels that needs a higher (or even lower) quota. The mechanism is similar to the per-topic log config overrides. User and (user, client-id) quota overrides are written to the metadata log. These overrides are read by all brokers and are effective immediately. This lets us change quotas without having to do a rolling restart of the entire cluster. See here for details. Default quotas for each group may also be updated dynamically using the same mechanism.

The order of precedence for quota configuration is:

1. matching user and client-id quotas
2. matching user and default client-id quotas
3. matching user quota
4. default user and matching client-id quotas
5. default user and default client-id quotas
6. default user quota
7. matching client-id quota
8. default client-id quota

Network Bandwidth Quotas

Network bandwidth quotas are defined as the byte rate threshold for each group of clients sharing a quota. By default, each unique client group receives a fixed quota in bytes/sec as configured by the cluster. This quota is defined on a per-broker basis. Each group of clients can publish/fetch a maximum of X bytes/sec per broker before clients are throttled.

Request Rate Quotas

Request rate quotas are defined as the percentage of time a client can utilize on request handler I/O threads and network threads of each broker within a quota window. A quota of `n%` represents `n%` of one thread, so the quota is out of a total capacity of `((num.io.threads + num.network.threads) * 100)%`. Each group of clients may use a total percentage of upto `n%` across all I/O and network threads in a quota window before being throttled. Since the number of threads allocated for I/O and network threads are typically based on the number of cores available on the broker host, request rate quotas represent the total percentage of CPU that may be used by each group of clients sharing the quota.

Enforcement

By default, each unique client group receives a fixed quota as configured by the cluster. This quota is defined on a per-broker basis. Each client can utilize this quota per broker before it gets throttled. We decided that defining these quotas per broker is much better than having a fixed cluster wide bandwidth per client because that would require a mechanism to share client quota usage among all the brokers. This can be harder to get right than the quota implementation itself!

How does a broker react when it detects a quota violation? In our solution, the broker first computes the amount of delay needed to bring the violating client under its quota and returns a response with the delay immediately. In case of a fetch request, the response will not contain any data. Then, the broker mutes the channel to the client, not to process requests from the client anymore, until the delay is over. Upon receiving a response with a non-zero delay duration, the Kafka client will also refrain from sending further requests to the broker during the delay. Therefore, requests from a throttled client are effectively blocked from both sides. Even with older client implementations that do not respect the delay response from the broker, the back pressure applied by the broker via muting its socket channel can still handle the throttling of badly behaving clients. Those clients who sent further requests to the throttled channel will receive responses only after the delay is over.

Byte-rate and thread utilization are measured over multiple small windows (e.g. 30 windows of 1 second each) in order to detect and correct quota violations quickly. Typically, having large measurement windows (for e.g. 10 windows of 30 seconds each) leads to large bursts of traffic followed by long delays which is not great in terms of user experience.

2 - Protocol

Kafka protocol guide

This document covers the wire protocol implemented in Kafka. It is meant to give a readable guide to the protocol that covers the available requests, their binary format, and the proper way to make use of them to implement a client. This document assumes you understand the basic design and terminology described here

Preliminaries

Network

Kafka uses a binary protocol over TCP. The protocol defines all APIs as request response message pairs. All messages are size delimited and are made up of the following primitive types.

The client initiates a socket connection and then writes a sequence of request messages and reads back the corresponding response message. No handshake is required on connection or disconnection. TCP is happier if you maintain persistent connections used for many requests to amortize the cost of the TCP handshake, but beyond this penalty connecting is pretty cheap.

The client will likely need to maintain a connection to multiple brokers, as data is partitioned and the clients will need to talk to the server that has their data. However it should not generally be necessary to maintain multiple connections to a single broker from a single client instance (i.e. connection pooling).

The server guarantees that on a single TCP connection, requests will be processed in the order they are sent and responses will return in that order as well. The broker's request processing allows only a single in-flight request per connection in order to guarantee this ordering. Note that clients can (and ideally should) use non-blocking IO to implement request pipelining and achieve higher throughput. i.e., clients can send requests even while awaiting responses for preceding requests since the outstanding requests will be buffered in the underlying OS socket buffer. All requests are initiated by the client, and result in a corresponding response message from the server except where noted.

The server has a configurable maximum limit on request size and any request that exceeds

this limit will result in the socket being disconnected.

Partitioning and bootstrapping

Kafka is a partitioned system so not all servers have the complete data set. Instead recall that topics are split into a pre-defined number of partitions, P, and each partition is replicated with some replication factor, N. Topic partitions themselves are just ordered “commit logs” numbered 0, 1, ..., P-1.

All systems of this nature have the question of how a particular piece of data is assigned to a particular partition. Kafka clients directly control this assignment, the brokers themselves enforce no particular semantics of which messages should be published to a particular partition. Rather, to publish messages the client directly addresses messages to a particular partition, and when fetching messages, fetches from a particular partition. If two clients want to use the same partitioning scheme they must use the same method to compute the mapping of key to partition.

These requests to publish or fetch data must be sent to the broker that is currently acting as the leader for a given partition. This condition is enforced by the broker, so a request for a particular partition to the wrong broker will result in the NotLeaderForPartition error code (described below).

How can the client find out which topics exist, what partitions they have, and which brokers currently host those partitions so that it can direct its requests to the right hosts? This information is dynamic, so you can’t just configure each client with some static mapping file. Instead all Kafka brokers can answer a metadata request that describes the current state of the cluster: what topics there are, which partitions those topics have, which broker is the leader for those partitions, and the host and port information for these brokers.

In other words, the client needs to somehow find one broker and that broker will tell the client about all the other brokers that exist and what partitions they host. This first broker may itself go down so the best practice for a client implementation is to take a list of two or three URLs to bootstrap from. The user can then choose to use a load balancer or just statically configure two or three of their Kafka hosts in the clients.

The client does not need to keep polling to see if the cluster has changed; it can fetch metadata once when it is instantiated cache that metadata until it receives an error indicating that the metadata is out of date. This error can come in two forms: (1) a socket error indicating the client cannot communicate with a particular broker, (2) an error code in the

response to a request indicating that this broker no longer hosts the partition for which data was requested.

1. Cycle through a list of "bootstrap" Kafka URLs until we find one we can connect to. Fetch cluster metadata.
2. Process fetch or produce requests, directing them to the appropriate broker based on the topic/partitions they send to or fetch from.
3. If we get an appropriate error, refresh the metadata and try again.

Partitioning Strategies

As mentioned above the assignment of messages to partitions is something the producing client controls. That said, how should this functionality be exposed to the end-user?

Partitioning really serves two purposes in Kafka:

1. It balances data and request load over brokers
2. It serves as a way to divvy up processing among consumer processes while allowing local state and preserving order within the partition. We call this semantic partitioning.

For a given use case you may care about only one of these or both.

To accomplish simple load balancing a simple approach would be for the client to just round robin requests over all brokers. Another alternative, in an environment where there are many more producers than brokers, would be to have each client choose a single partition at random and publish to that. This latter strategy will result in far fewer TCP connections.

Semantic partitioning means using some key in the message to assign messages to partitions. For example if you were processing a click message stream you might want to partition the stream by the user id so that all data for a particular user would go to a single consumer. To accomplish this the client can take a key associated with the message and use some hash of this key to choose the partition to which to deliver the message.

Batching

Our APIs encourage batching small things together for efficiency. We have found this is a very significant performance win. Both our API to send messages and our API to fetch messages always work with a sequence of messages not a single message to encourage this. A clever client can make use of this and support an "asynchronous" mode in which it batches together

messages sent individually and sends them in larger clumps. We go even further with this and allow the batching across multiple topics and partitions, so a produce request may contain data to append to many partitions and a fetch request may pull data from many partitions all at once.

The client implementer can choose to ignore this and send everything one at a time if they like.

Compatibility

Kafka has a “bidirectional” client compatibility policy. In other words, new clients can talk to old servers, and old clients can talk to new servers. This allows users to upgrade either clients or servers without experiencing any downtime.

Since the Kafka protocol has changed over time, clients and servers need to agree on the schema of the message that they are sending over the wire. This is done through API versioning.

Before each request is sent, the client sends the API key and the API version. These two 16-bit numbers, when taken together, uniquely identify the schema of the message to follow.

The intention is that clients will support a range of API versions. When communicating with a particular broker, a given client should use the highest API version supported by both and indicate this version in their requests.

The server will reject requests with a version it does not support, and will always respond to the client with exactly the protocol format it expects based on the version it included in its request. The intended upgrade path is that new features would first be rolled out on the server (with the older clients not making use of them) and then as newer clients are deployed these new features would gradually be taken advantage of. Note there is an exceptional case while retrieving supported API versions where the server can respond with a different version.

Note that KIP-482 tagged fields can be added to a request without incrementing the version number. This offers an additional way of evolving the message schema without breaking compatibility. Tagged fields do not take up any space when the field is not set. Therefore, if a field is rarely used, it is more efficient to make it a tagged field than to put it in the mandatory schema. However, tagged fields are ignored by recipients that don’t know about them, which could pose a challenge if this is not the behavior that the sender wants. In such cases, a

version bump may be more appropriate.

Retrieving Supported API versions

In order to work against multiple broker versions, clients need to know what versions of various APIs a broker supports. The broker exposes this information since 0.10.0.0 as described in KIP-35. Clients should use the supported API versions information to choose the highest API version supported by both client and broker. If no such version exists, an error should be reported to the user.

The following sequence may be used by a client to obtain supported API versions from a broker.

1. Client sends `ApiVersionsRequest` to a broker after connection has been established with the broker. If SSL is enabled, this happens after SSL connection has been established.
2. On receiving `ApiVersionsRequest`, a broker returns its full list of supported ApiKeys and versions regardless of current authentication state (e.g., before SASL authentication on an SASL listener, do note that no Kafka protocol requests may take place on an SSL listener before the SSL handshake is finished). If this is considered to leak information about the broker version a workaround is to use SSL with client authentication which is performed at an earlier stage of the connection where the `ApiVersionRequest` is not available. Also, note that broker versions older than 0.10.0.0 do not support this API and will either ignore the request or close connection in response to the request. Also note that if the client `ApiVersionsRequest` version is unsupported by the broker (client is ahead), and the broker version is 2.4.0 or greater, then the broker will respond with a version 0 ApiVersionsResponse with the error code set to `UNSUPPORTED_VERSION` and the `api_versions` field populated with the supported version of the `ApiVersionsRequest`. It is then up to the client to retry, making another `ApiVersionsRequest` using the highest version supported by the client and broker. See KIP-511: Collect and Expose Client's Name and Version in the Brokers
3. If multiple versions of an API are supported by broker and client, clients are recommended to use the latest version supported by the broker and itself.
4. Deprecation of a protocol version is done by marking an API version as deprecated in the protocol documentation.
5. Supported API versions obtained from a broker are only valid for the connection on which that information is obtained. In the event of disconnection, the client should obtain the information from the broker again, as the broker might have been upgraded/downgraded in the meantime.

SASL Authentication Sequence

The following sequence is used for SASL authentication:

1. Kafka `ApiVersionsRequest` may be sent by the client to obtain the version ranges of requests supported by the broker. This is optional.
2. Kafka `SaslHandshakeRequest` containing the SASL mechanism for authentication is sent by the client. If the requested mechanism is not enabled in the server, the server responds with the list of supported mechanisms and closes the client connection. If the mechanism is enabled in the server, the server sends a successful response and continues with SASL authentication.
3. The actual SASL authentication is now performed. If `SaslHandshakeRequest` version is v0, a series of SASL client and server tokens corresponding to the mechanism are sent as opaque packets without wrapping the messages with Kafka protocol headers. If `SaslHandshakeRequest` version is v1, the `SaslAuthenticate` request/response are used, where the actual SASL tokens are wrapped in the Kafka protocol. The error code in the final message from the broker will indicate if authentication succeeded or failed.
4. If authentication succeeds, subsequent packets are handled as Kafka API requests. Otherwise, the client connection is closed.

For interoperability with 0.9.0.x clients, the first packet received by the server is handled as a SASL/GSSAPI client token if it is not a valid Kafka request. SASL/GSSAPI authentication is performed starting with this packet, skipping the first two steps above.

The Protocol

Protocol Primitive Types

The protocol is built out of the following primitive types.

TYPE	DESCRIPTION
BOOLEAN	Represents a boolean value in a byte. Values 0 and 1 are used to represent false and true respectively. When reading a boolean value, any non-zero value is considered true.

INT8	Represents an integer between -2^7 and 2^7-1 inclusive.
INT16	Represents an integer between -2^{15} and $2^{15}-1$ inclusive. The values are encoded using two bytes in network byte order (big-endian).
INT32	Represents an integer between -2^{31} and $2^{31}-1$ inclusive. The values are encoded using four bytes in network byte order (big-endian).
INT64	Represents an integer between -2^{63} and $2^{63}-1$ inclusive. The values are encoded using eight bytes in network byte order (big-endian).
UINT16	Represents an integer between 0 and 65535 inclusive. The values are encoded using two bytes in network byte order (big-endian).
UINT32	Represents an integer between 0 and $2^{32}-1$ inclusive. The values are encoded using four bytes in network byte order (big-endian).
VARINT	Represents an integer between -2^{31} and $2^{31}-1$ inclusive. Encoding follows the variable-length zig-zag encoding from Google Protocol Buffers.
VARLONG	Represents an integer between -2^{63} and $2^{63}-1$ inclusive. Encoding follows the variable-length zig-zag encoding from Google Protocol Buffers.
UUID	Represents a type 4 immutable universally unique identifier (Uuid). The values are encoded using sixteen bytes in network byte order (big-endian).
FLOAT64	Represents a double-precision 64-bit format IEEE 754 value. The values are encoded using eight bytes in network byte order (big-endian).
STRING	Represents a sequence of characters. First the length N is given as an INT16. Then N bytes follow which are the UTF-8 encoding of the character sequence. Length must not be negative.

COMPACT_STRING	Represents a sequence of characters. First the length N + 1 is given as an UNSIGNED_VARINT . Then N bytes follow which are the UTF-8 encoding of the character sequence.
NULLABLE_STRING	Represents a sequence of characters or null. For non-null strings, first the length N is given as an INT16. Then N bytes follow which are the UTF-8 encoding of the character sequence. A null value is encoded with length of -1 and there are no following bytes.
COMPACT_NULLABLE_STRING	Represents a sequence of characters. First the length N + 1 is given as an UNSIGNED_VARINT . Then N bytes follow which are the UTF-8 encoding of the character sequence. A null string is represented with a length of 0.
BYTES	Represents a raw sequence of bytes. First the length N is given as an INT32. Then N bytes follow.
COMPACT_BYTES	Represents a raw sequence of bytes. First the length N+1 is given as an UNSIGNED_VARINT. Then N bytes follow.
NULLABLE_BYTES	Represents a raw sequence of bytes or null. For non-null values, first the length N is given as an INT32. Then N bytes follow. A null value is encoded with length of -1 and there are no following bytes.
COMPACT_NULLABLE_BYTES	Represents a raw sequence of bytes. First the length N+1 is given as an UNSIGNED_VARINT. Then N bytes follow. A null object is represented with a length of 0.
RECORDS	Represents a sequence of Kafka records as BYTES. For a detailed description of records see Message Sets.
COMPACT_RECORDS	Represents a sequence of Kafka records as COMPACT_BYTES. For a detailed description of records see Message Sets.
NULLABLE_RECORDS	Represents a sequence of Kafka records as NULLABLE_BYTES. For a detailed description of records see Message Sets.

COMPACT_NULLABLE_RECORDS	Represents a sequence of Kafka records as COMPACT_NULLABLE_BYTES. For a detailed description of records see Message Sets.
ARRAY	Represents a sequence of objects of a given type T. Type T can be either a primitive type (e.g. STRING) or a structure. First, the length N is given as an INT32. Then N instances of type T follow. In protocol documentation an array of T instances is referred to as [T].
COMPACT_ARRAY	Represents a sequence of objects of a given type T. Type T can be either a primitive type (e.g. STRING) or a structure. First, the length N + 1 is given as an UNSIGNED_VARINT. Then N instances of type T follow. In protocol documentation a compact array of T instances is referred to as (T).
NULLABLE_ARRAY	Represents a sequence of objects of a given type T. Type T can be either a primitive type (e.g. STRING) or a structure. First, the length N is given as an INT32. Then N instances of type T follow. A null array is represented with a length of -1. In protocol documentation a nullable array of T instances is referred to as ?[T].
COMPACT_NULLABLE_ARRAY	Represents a sequence of objects of a given type T. Type T can be either a primitive type (e.g. STRING) or a structure. First, the length N + 1 is given as an UNSIGNED_VARINT. Then N instances of type T follow. A null array is represented with a length of 0. In protocol documentation a compact nullable array of T instances is referred to as ?(T).
STRUCT	A struct is named by a string with a capitalized first letter and consists of one or more fields. It represents a composite object encoded as the serialization of each field in the order they are defined.In protocol documentation a struct containing multiple fields is enclosed by { and }.
NULLABLE_STRUCT	A nullable struct is named by a string with a capitalized first letter and consists of one or more fields. It

	represents a composite object or null. For non-null values, the first byte has value 1, followed by the serialization of each field in the order they are defined. A null value is encoded as a byte with value -1 and there are no following bytes.In protocol documentation a nullable struct containing multiple fields is enclosed by ?{ and }.

Notes on reading the request format grammars

The BNFs below give an exact context free grammar for the request and response binary format. The BNF is intentionally not compact in order to give human-readable name. As always in a BNF a sequence of productions indicates concatenation. When there are multiple possible productions these are separated with '|' and may be enclosed in parenthesis for grouping. The top-level definition is always given first and subsequent sub-parts are indented.

Common Request and Response Structure

All requests and responses originate from the following grammar which will be incrementally describe through the rest of this document:

```
RequestOrResponse => Size (RequestMessage | ResponseMessage)
  Size => int32
```

FIELD	DESCRIPTION
message_size	The message_size field gives the size of the subsequent request or response message in bytes. The client can read requests by first reading this 4 byte size as an integer N, and then reading and parsing the subsequent N bytes of the request.

Request and Response Headers

Different request and response versions require different versions of the corresponding headers. These header versions are specified below together with API message descriptions.

Record Batch

A description of the record batch format can be found here.

Constants

Error Codes

We use numeric codes to indicate what problem occurred on the server. These can be translated by the client into exceptions or whatever the appropriate error handling mechanism in the client language. Here is a table of the error codes currently in use:

ERROR	CODE	RETRIABLE	DESCRIPTION
UNKNOWN_SERVER_ERROR	-1	False	The server experience unexpected error whe processing the reques
NONE	0	False	
OFFSET_OUT_OF_RANGE	1	False	The requested offset within the range of of maintained by the ser
CORRUPT_MESSAGE	2	True	This message has faile checksum, exceeds th size, has a null key for compacted topic, or is otherwise corrupt.
UNKNOWN_TOPIC_OR_PARTITION	3	True	This server does not h topic-partition.
INVALID_FETCH_SIZE	4	False	The requested fetch s invalid.
LEADER_NOT_AVAILABLE	5	True	There is no leader for partition as we are in middle of a leadershi

NOT_LEADER_OR_FOLLOWER	6	True	For requests intended the leader, this error i that the broker is not current leader. For re intended for any repli error indicates that th is not a replica of the partition.
REQUEST_TIMED_OUT	7	True	The request timed ou
BROKER_NOT_AVAILABLE	8	False	The broker is not avai
REPLICA_NOT_AVAILABLE	9	True	The replica is not avai the requested topic-p Produce/Fetch reques other requests intend for the leader or follo NOT_LEADER_OR_FOL the broker is not a rep topic-partition.
MESSAGE_TOO_LARGE	10	False	The request included message larger than t message size the serv accept.
STALE_CONTROLLER_EPOCH	11	False	The controller moved another broker.
OFFSET_METADATA_TOO_LARGE	12	False	The metadata field of request was too large
NETWORK_EXCEPTION	13	True	The server disconnect a response was receiv
COORDINATOR_LOAD_IN_PROGRESS	14	True	The coordinator is loa hence can't process r
COORDINATOR_NOT_AVAILABLE	15	True	The coordinator is no

NOT_COORDINATOR	16	True	This is not the correct coordinator.
INVALID_TOPIC_EXCEPTION	17	False	The request attempte perform an operation invalid topic.
RECORD_LIST_TOO_LARGE	18	False	The request included batch larger than the configured segment s server.
NOT_ENOUGH_REPLICAS	19	True	Messages are rejecte there are fewer in-syr than required.
NOT_ENOUGH_REPLICAS_AFTER_APPEND	20	True	Messages are written but to fewer in-sync r than required.
INVALID_REQUIRED_ACKS	21	False	Produce request spec invalid value for requi
ILLEGAL_GENERATION	22	False	Specified group gener not valid.
INCONSISTENT_GROUP_PROTOCOL	23	False	The group member's protocols are incomp those of existing mem first group member tr with empty protocol t empty protocol list.
INVALID_GROUP_ID	24	False	The group id is invalid
UNKNOWN_MEMBER_ID	25	False	The coordinator is no this member.
INVALID_SESSION_TIMEOUT	26	False	The session timeout i within the range allow broker (as configured group.min.session.tim

			and group.max.session.tir
REBALANCE_IN_PROGRESS	27	False	The group is rebalanc rejoin is needed.
INVALID_COMMIT_OFFSET_SIZE	28	False	The committing offset is not valid.
TOPIC_AUTHORIZATION_FAILED	29	False	Topic authorization fa
GROUP_AUTHORIZATION_FAILED	30	False	Group authorization f
CLUSTER_AUTHORIZATION_FAILED	31	False	Cluster authorization
INVALID_TIMESTAMP	32	False	The timestamp of the is out of acceptable ra
UNSUPPORTED_SASL_MECHANISM	33	False	The broker does not s the requested SASL mechanism.
ILLEGAL_SASL_STATE	34	False	Request is not valid gi current SASL state.
UNSUPPORTED_VERSION	35	False	The version of API is r supported.
TOPIC_ALREADY_EXISTS	36	False	Topic with this name exists.
INVALID_PARTITIONS	37	False	Number of partitions 1.
INVALID_REPLICATION_FACTOR	38	False	Replication factor is b larger than the numb available brokers.
INVALID_REPLICA_ASSIGNMENT	39	False	Replica assignment is
INVALID_CONFIG	40	False	Configuration is invali
NOT_CONTROLLER	41	True	This is not the correct for this cluster.

INVALID_REQUEST	42	False	This most likely occur of a request being ma by the client library or message was sent to incompatible broker. broker logs for more
UNSUPPORTED_FOR_MESSAGE_FORMAT	43	False	The message format v the broker does not s the request.
POLICY_VIOLATION	44	False	Request parameters d satisfy the configured
OUT_OF_ORDER_SEQUENCE_NUMBER	45	False	The broker received a order sequence numb
DUPLICATE_SEQUENCE_NUMBER	46	False	The broker received a sequence number.
INVALID_PRODUCER_EPOCH	47	False	Producer attempted t with an old epoch.
INVALID_TXN_STATE	48	False	The producer attempt transactional operatic invalid state.
INVALID_PRODUCER_ID_MAPPING	49	False	The producer attempt a producer id which is currently assigned to transactional id.
INVALID_TRANSACTION_TIMEOUT	50	False	The transaction timec larger than the maxim allowed by the broker configured by transaction.max.time
CONCURRENT_TRANSACTIONS	51	True	The producer attempt update a transaction v another concurrent o on the same transacti

			ongoing.
TRANSACTION_COORDINATOR_FENCED	52	False	Indicates that the trar coordinator sending a WriteTxnMarker is no the current coordinat given producer.
TRANSACTIONAL_ID_AUTHORIZATION_FAILED	53	False	Transactional Id auth failed.
SECURITY_DISABLED	54	False	Security features are
OPERATION_NOT_ATTEMPTED	55	False	The broker did not att execute this operatio may happen for batch where some operatio batch failed, causing t to respond without tr rest.
KAFKA_STORAGE_ERROR	56	True	Disk error when trying log file on the disk.
LOG_DIR_NOT_FOUND	57	False	The user-specified log is not found in the bro config.
SASL_AUTHENTICATION_FAILED	58	False	SASL Authentication f
UNKNOWN_PRODUCER_ID	59	False	This exception is raise broker if it could not l producer metadata as with the producerId ir question. This could h for instance, the prod records were deleted their retention time h elapsed. Once the last of the producerId are the producer's metad removed from the bro

			future appends by the will return this except
REASSIGNMENT_IN_PROGRESS	60	False	A partition reassignm progress.
DELEGATION_TOKEN_AUTH_DISABLED	61	False	Delegation Token feat enabled.
DELEGATION_TOKEN_NOT_FOUND	62	False	Delegation Token is n on server.
DELEGATION_TOKEN_OWNER_MISMATCH	63	False	Specified Principal is r Owner/Renewer.
DELEGATION_TOKEN_REQUEST_NOT_ALLOWED	64	False	Delegation Token req not allowed on PLAIN way SSL channels and delegation token auth channels.
DELEGATION_TOKEN_AUTHORIZATION_FAILED	65	False	Delegation Token aut failed.
DELEGATION_TOKEN_EXPIRED	66	False	Delegation Token is e
INVALID_PRINCIPAL_TYPE	67	False	Supplied principalTyp supported.
NON_EMPTY_GROUP	68	False	The group is not emp
GROUP_ID_NOT_FOUND	69	False	The group id does not
FETCH_SESSION_ID_NOT_FOUND	70	True	The fetch session ID w found.
INVALID_FETCH_SESSION_EPOCH	71	True	The fetch session epo invalid.
LISTENER_NOT_FOUND	72	True	There is no listener on leader broker that ma listener on which met request was processe

TOPIC_DELETION_DISABLED	73	False	Topic deletion is disab
FENCED_LEADER_EPOCH	74	True	The leader epoch in th is older than the epoc broker.
UNKNOWN_LEADER_EPOCH	75	True	The leader epoch in th is newer than the epo broker.
UNSUPPORTED_COMPRESSION_TYPE	76	False	The requesting client support the compress of given partition.
STALE_BROKER_EPOCH	77	False	Broker epoch has cha
OFFSET_NOT_AVAILABLE	78	True	The leader high water not caught up from a leader election so the cannot be guaranteed monotonically increas
MEMBER_ID_REQUIRED	79	False	The group member ne have a valid member actually entering a co group.
PREFERRED_LEADER_NOT_AVAILABLE	80	True	The preferred leader available.
GROUP_MAX_SIZE_REACHED	81	False	The group has reache maximum size.
FENCED_INSTANCE_ID	82	False	The broker rejected th consumer since anoth consumer with the sa group.instance.id has registered with a diffe member.id.
ELIGIBLE_LEADERS_NOT_AVAILABLE	83	True	Eligible topic partition are not available.

ELECTION_NOT_NEEDED	84	True	Leader election not n topic partition.
NO_REASSIGNMENT_IN_PROGRESS	85	False	No partition reassignr progress.
GROUP_SUBSCRIBED_TO_TOPIC	86	False	Deleting offsets of a t forbidden while the c group is actively subs it.
INVALID_RECORD	87	False	This record has failed validation on broker a will be rejected.
UNSTABLE_OFFSET_COMMIT	88	True	There are unstable of need to be cleared.
THROTTLING_QUOTA_EXCEEDED	89	True	The throttling quota h exceeded.
PRODUCER_FENCED	90	False	There is a newer prod the same transactiona fences the current on
RESOURCE_NOT_FOUND	91	False	A request illegally refe resource that does nc
DUPLICATE_RESOURCE	92	False	A request illegally refe the same resource tw
UNACCEPTABLE_CREDENTIAL	93	False	Requested credential meet criteria for accep
INCONSISTENT_VOTER_SET	94	False	Indicates that the eith sender or recipient of only request is not on expected voters.
INVALID_UPDATE_VERSION	95	False	The given update vers invalid.

FEATURE_UPDATE_FAILED	96	False	Unable to update fina features due to an un server error.
PRINCIPAL_DESERIALIZATION_FAILURE	97	False	Request principal deserialization failed forwarding. This indic internal error on the b cluster security setup
SNAPSHOT_NOT_FOUND	98	False	Requested snapshot found.
POSITION_OUT_OF_RANGE	99	False	Requested position is greater than or equal and less than the size snapshot.
UNKNOWN_TOPIC_ID	100	True	This server does not h topic ID.
DUPLICATE_BROKER_REGISTRATION	101	False	This broker ID is alrea
BROKER_ID_NOT_REGISTERED	102	False	The given broker ID w registered.
INCONSISTENT_TOPIC_ID	103	True	The log's topic ID did the topic ID in the req
INCONSISTENT_CLUSTER_ID	104	False	The clusterId in the re does not match that f the server.
TRANSACTIONAL_ID_NOT_FOUND	105	False	The transactionalId co be found.
FETCH_SESSION_TOPIC_ID_ERROR	106	True	The fetch session enc inconsistent topic ID u
INELIGIBLE_REPLICA	107	False	The new ISR contains one ineligible replica.

NEW_LEADER_ELECTED	108	False	The AlterPartition req successfully updated partition state but the has changed.
OFFSET_MOVED_TO_TIERED_STORAGE	109	False	The requested offset to tiered storage.
FENCED_MEMBER_EPOCH	110	False	The member epoch is the group coordinato member must abando partitions and rejoin.
UNRELEASED_INSTANCE_ID	111	False	The instance ID is still another member in th consumer group. That must leave first.
UNSUPPORTED_ASSIGNOR	112	False	The assignor or its ver range is not supporte consumer group.
STALE_MEMBER_EPOCH	113	False	The member epoch is member must retry a receiving its updated epoch via the ConsumerGroupHear
MISMATCHED_ENDPOINT_TYPE	114	False	The request was sent endpoint of the wrong
UNSUPPORTED_ENDPOINT_TYPE	115	False	This endpoint type is supported yet.
UNKNOWN_CONTROLLER_ID	116	False	This controller ID is no
UNKNOWN_SUBSCRIPTION_ID	117	False	Client sent a push tele request with an invali outdated subscriptior
TELEMETRY_TOO_LARGE	118	False	Client sent a push tele request larger than th

			maximum size the br accept.
INVALID_REGISTRATION	119	False	The controller has cor the broker registratio invalid.
TRANSACTION_ABORTABLE	120	False	The server encounter error with the transac client can abort the tr to continue using this transactional ID.
INVALID_RECORD_STATE	121	False	The record state is inv acknowledgement of could not be complet
SHARE_SESSION_NOT_FOUND	122	True	The share session wa found.
INVALID_SHARE_SESSION_EPOCH	123	True	The share session epc invalid.
FENCED_STATE_EPOCH	124	False	The share coordinato the request because t group state epoch dic match.
INVALID_VOTER_KEY	125	False	The voter key doesn't the receiving replica's
DUPLICATE_VOTER	126	False	The voter is already p set of voters.
VOTER_NOT_FOUND	127	False	The voter is not part c of voters.
INVALID_REGULAR_EXPRESSION	128	False	The regular expressio valid.
REBOOTSTRAP_REQUIRED	129	False	Client metadata is sta client should rebootst obtain new metadata

STREAMS_INVALID_TOPOLOGY	130	False	The supplied topology
STREAMS_INVALID_TOPOLOGY_EPOCH	131	False	The supplied topology invalid.
STREAMS_TOPOLOGY_FENCED	132	False	The supplied topology outdated.
SHARE_SESSION_LIMIT_REACHED	133	True	The limit of share sess been reached.

Api Keys

The following are the numeric codes that the stable ApiKey in the request can take for each of the below request types.

NAME	KEY
Produce	0
Fetch	1
ListOffsets	2
Metadata	3
OffsetCommit	8
OffsetFetch	9
FindCoordinator	10
JoinGroup	11
Heartbeat	12
LeaveGroup	13
SyncGroup	14
DescribeGroups	15
ListGroups	16

SaslHandshake	17
ApiVersions	18
CreateTopics	19
DeleteTopics	20
DeleteRecords	21
InitProducerId	22
OffsetForLeaderEpoch	23
AddPartitionsToTxn	24
AddOffsetsToTxn	25
EndTxn	26
WriteTxnMarkers	27
TxnOffsetCommit	28
DescribeAcls	29
CreateAcls	30
DeleteAcls	31
DescribeConfigs	32
AlterConfigs	33
AlterReplicaLogDirs	34
DescribeLogDirs	35
SaslAuthenticate	36
CreatePartitions	37
CreateDelegationToken	38
RenewDelegationToken	39

ExpireDelegationToken	40
DescribeDelegationToken	41
DeleteGroups	42
ElectLeaders	43
IncrementalAlterConfigs	44
AlterPartitionReassignments	45
ListPartitionReassignments	46
OffsetDelete	47
DescribeClientQuotas	48
AlterClientQuotas	49
DescribeUserScramCredentials	50
AlterUserScramCredentials	51
DescribeQuorum	55
UpdateFeatures	57
DescribeCluster	60
DescribeProducers	61
UnregisterBroker	64
DescribeTransactions	65
ListTransactions	66
ConsumerGroupHeartbeat	68
ConsumerGroupDescribe	69
GetTelemetrySubscriptions	71
PushTelemetry	72

ListConfigResources	74
DescribeTopicPartitions	75
ShareGroupHeartbeat	76
ShareGroupDescribe	77
ShareFetch	78
ShareAcknowledge	79
AddRaftVoter	80
RemoveRaftVoter	81
InitializeShareGroupState	83
ReadShareGroupState	84
WriteShareGroupState	85
DeleteShareGroupState	86
ReadShareGroupStateSummary	87
StreamsGroupHeartbeat	88
StreamsGroupDescribe	89
DescribeShareGroupOffsets	90
AlterShareGroupOffsets	91
DeleteShareGroupOffsets	92

The Messages

This section gives details on each of the individual API Messages, their usage, their binary format, and the meaning of their fields.

The message consists of the header and body:

```
Message => RequestOrResponseHeader Body
```

`RequestOrResponseHeader` is the versioned request or response header. `Body` is the message-specific body.

Headers:

```
Request Header v1 => { request_api_key request_api_version correlation_id client_
  request_api_key => INT16
  request_api_version => INT16
  correlation_id => INT32
  client_id => NULLABLE_STRING
```

FIELD	DESCRIPTION
request_api_key	The API key of this request.
request_api_version	The API version of this request.
correlation_id	The correlation ID of this request.
client_id	The client ID string.

```
Request Header v2 => { request_api_key request_api_version correlation_id client_
  request_api_key => INT16
  request_api_version => INT16
  correlation_id => INT32
  client_id => NULLABLE_STRING
```

FIELD	DESCRIPTION
request_api_key	The API key of this request.
request_api_version	The API version of this request.
correlation_id	The correlation ID of this request.
client_id	The client ID string.

```
Response Header v0 => { correlation_id }
  correlation_id => INT32
```

FIELD	DESCRIPTION
correlation_id	The correlation ID of this response.

```
Response Header v1 => { correlation_id }
  correlation_id => INT32
```

FIELD	DESCRIPTION
correlation_id	The correlation ID of this response.

Produce API (Key: 0):

Requests:

```
Produce Request (Version: 3) => { transactional_id acks timeout_ms [topic_data] }
  transactional_id => NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name [partition_data] }
    name => STRING
    partition_data => { index records }
      index => INT32
      records => NULLABLE_RECORDS
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.

topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 4) => { transactional_id acks timeout_ms [topic_data] }
  transactional_id => NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name [partition_data] }
    name => STRING
    partition_data => { index records }
      index => INT32
      records => NULLABLE_RECORDS
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 5) => { transactional_id acks timeout_ms [topic_data] }
  transactional_id => NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name [partition_data] }
    name => STRING
    partition_data => { index records }
      index => INT32
      records => NULLABLE_RECORDS
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 6) => { transactional_id acks timeout_ms [topic_data] }
  transactional_id => NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name [partition_data] }
    name => STRING
    partition_data => { index records }
      index => INT32
      records => NULLABLE_RECORDS
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 7) => { transactional_id acks timeout_ms [topic_data] }
  transactional_id => NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name [partition_data] }
    name => STRING
    partition_data => { index records }
      index => INT32
      records => NULLABLE_RECORDS
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.

timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 8) => { transactional_id acks timeout_ms [topic_data] }
  transactional_id => NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name [partition_data] }
    name => STRING
    partition_data => { index records }
      index => INT32
      records => NULLABLE_RECORDS
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 9) => { transactional_id acks timeout_ms (topic_data) }
  transactional_id => COMPACT_NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name (partition_data) }
    name => COMPACT_STRING
    partition_data => { index records }
      index => INT32
      records => COMPACT_NULLABLE_RECORDS
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 10) => { transactional_id acks timeout_ms (topic_data)
  transactional_id => COMPACT_NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name (partition_data) }
    name => COMPACT_STRING
    partition_data => { index records }
      index => INT32
      records => COMPACT_NULLABLE_RECORDS
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 11) => { transactional_id acks timeout_ms (topic_data)
  transactional_id => COMPACT_NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name (partition_data) }
    name => COMPACT_STRING
    partition_data => { index records }
      index => INT32
      records => COMPACT_NULLABLE_RECORDS
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.

timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 12) => { transactional_id acks timeout_ms (topic_data)
  transactional_id => COMPACT_NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { name (partition_data) }
    name => COMPACT_STRING
    partition_data => { index records }
      index => INT32
      records => COMPACT_NULLABLE_RECORDS
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
name	The topic name.
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

```
Produce Request (Version: 13) => { transactional_id acks timeout_ms (topic_data)
  transactional_id => COMPACT_NULLABLE_STRING
  acks => INT16
  timeout_ms => INT32
  topic_data => { topic_id (partition_data) }
    topic_id => UUID
    partition_data => { index records }
      index => INT32
      records => COMPACT_NULLABLE_RECORDS
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional ID, or null if the producer is not transactional.
acks	The number of acknowledgments the producer requires the leader to have received before considering a request complete. Allowed values: 0 for no acknowledgments, 1 for only the leader and -1 for the full ISR.
timeout_ms	The timeout to await a response in milliseconds.
topic_data	Each topic to produce to.
topic_id	The unique topic ID
partition_data	Each partition to produce to.
index	The partition index.
records	The record data to be produced.

Responses:

```
Produce Response (Version: 3) => { [responses] throttle_time_ms }
  responses => { name [partition_responses] }
    name => STRING
    partition_responses => { index error_code base_offset log_append_time_ms }
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
Produce Response (Version: 4) => { [responses] throttle_time_ms }
  responses => { name [partition_responses] }
    name => STRING
    partition_responses => { index error_code base_offset log_append_time_ms }
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.

partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
Produce Response (Version: 5) => { [responses] throttle_time_ms }
  responses => { name [partition_responses] }
    name => STRING
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
      log_start_offset => INT64
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.

base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
Produce Response (Version: 6) => { [responses] throttle_time_ms }
  responses => { name [partition_responses] }
    name => STRING
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
      log_start_offset => INT64
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If

	LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
Produce Response (Version: 7) => { [responses] throttle_time_ms }
  responses => { name [partition_responses] }
    name => STRING
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
      log_start_offset => INT64
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
Produce Response (Version: 8) => { [responses] throttle_time_ms }
  responses => { name [partition_responses] }
    name => STRING
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
      log_start_offset => INT64
      record_errors => { batch_index batch_index_error_message }
        batch_index => INT32
        batch_index_error_message => NULLABLE_STRING
      error_message => NULLABLE_STRING
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.

record_errors	The batch indices of records that caused the batch to be dropped.
batch_index	The batch index of the record that caused the batch to be dropped.
batch_index_error_message	The error message of the record that caused the batch to be dropped.
error_message	The global error message summarizing the common root cause of the records that caused the batch to be dropped.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
Produce Response (Version: 9) => { (responses) throttle_time_ms }
  responses => { name (partition_responses) }
    name => COMPACT_STRING
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
      log_start_offset => INT64
      record_errors => { batch_index batch_index_error_message }
        batch_index => INT32
        batch_index_error_message => COMPACT_NULLABLE_STRING
      error_message => COMPACT_NULLABLE_STRING
  throttle_time_ms => INT32
```

Response header version: 1

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.

error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.
record_errors	The batch indices of records that caused the batch to be dropped.
batch_index	The batch index of the record that caused the batch to be dropped.
batch_index_error_message	The error message of the record that caused the batch to be dropped.
error_message	The global error message summarizing the common root cause of the records that caused the batch to be dropped.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
Produce Response (Version: 10) => { (responses) throttle_time_ms node_endpoints<t
  responses => { name (partition_responses) }
    name => COMPACT_STRING
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
      log_start_offset => INT64
      record_errors => { batch_index batch_index_error_message }
        batch_index => INT32
        batch_index_error_message => COMPACT_NULLABLE_STRING
      error_message => COMPACT_NULLABLE_STRING
      current_leader<tag: 0> => { leader_id leader_epoch }
        leader_id => INT32
```

```
      leader_epoch => INT32
  throttle_time_ms => INT32
  node_endpoints<tag: 0> => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.
record_errors	The batch indices of records that caused the batch to be dropped.
batch_index	The batch index of the record that caused the batch to be dropped.
batch_index_error_message	The error message of the record that caused the batch to be dropped.
error_message	The global error message summarizing the common root cause of the records that caused the batch to be dropped.

<table>
<tr><td>current_leader<tag: 0></td><td>The leader broker that the producer should use for future requests.<table><tr><th>FIELD</th><th>DESCRIPTION</th></tr><tr><td>leader_id</td><td>The ID of the current leader or -1 if the leader is unknown.</td></tr><tr><td>leader_epoch</td><td>The latest known leader epoch.</td></tr></table></td></tr>
<tr><td>throttle_time_ms</td><td>The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.</td></tr>
<tr><td>node_endpoints<tag: 0></td><td>Endpoints for all current-leaders enumerated in PartitionProduceResponses, with errors NOT_LEADER_OR_FOLLOWER.<table><tr><th>FIELD</th><th>DESCRIPTION</th></tr><tr><td>node_id</td><td>The ID of the associated node.</td></tr><tr><td>host</td><td>The node's hostname.</td></tr><tr><td>port</td><td>The node's port.</td></tr><tr><td>rack</td><td>The rack of the node, or null if it has not been assigned to a rack.</td></tr></table></td></tr>
</table>

```
Produce Response (Version: 11) => { (responses) throttle_time_ms node_endpoints<t
  responses => { name (partition_responses) }
    name => COMPACT_STRING
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
      log_start_offset => INT64
      record_errors => { batch_index batch_index_error_message }
        batch_index => INT32
        batch_index_error_message => COMPACT_NULLABLE_STRING
      error_message => COMPACT_NULLABLE_STRING
```

```
      current_leader<tag: 0> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
  throttle_time_ms => INT32
  node_endpoints<tag: 0> => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.
record_errors	The batch indices of records that caused the batch to be dropped.
batch_index	The batch index of the record that caused the batch to be dropped.
batch_index_error_message	The error message of the record that caused the batch to be dropped.

error_message	The global error message summarizing the common root cause of the records that caused the batch to be dropped.
current_leader<tag: 0>	The leader broker that the producer should use for future requests. **FIELD** / **DESCRIPTION** leader_id: The ID of the current leader or -1 if the leader is unknown. leader_epoch: The latest known leader epoch.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
node_endpoints<tag: 0>	Endpoints for all current-leaders enumerated in PartitionProduceResponses, with errors NOT_LEADER_OR_FOLLOWER. **FIELD** / **DESCRIPTION** node_id: The ID of the associated node. host: The node's hostname. port: The node's port. rack: The rack of the node, or null if it has not been assigned to a rack.

```
Produce Response (Version: 12) => { (responses) throttle_time_ms node_endpoints<t
  responses => { name (partition_responses) }
    name => COMPACT_STRING
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
      log_append_time_ms => INT64
      log_start_offset => INT64
      record_errors => { batch_index batch_index_error_message }
```

```
      batch_index => INT32
      batch_index_error_message => COMPACT_NULLABLE_STRING
    error_message => COMPACT_NULLABLE_STRING
    current_leader<tag: 0> => { leader_id leader_epoch }
      leader_id => INT32
      leader_epoch => INT32
  throttle_time_ms => INT32
  node_endpoints<tag: 0> => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
responses	Each produce response.
name	The topic name.
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.
record_errors	The batch indices of records that caused the batch to be dropped.
batch_index	The batch index of the record that caused the batch to be dropped.

batch_index_error_message	The error message of the record that caused the batch to be dropped.
error_message	The global error message summarizing the common root cause of the records that caused the batch to be dropped.
current_leader<tag: 0>	The leader broker that the producer should use for future requests. **FIELD** / **DESCRIPTION** leader_id: The ID of the current leader or -1 if the leader is unknown. leader_epoch: The latest known leader epoch.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
node_endpoints<tag: 0>	Endpoints for all current-leaders enumerated in PartitionProduceResponses, with errors NOT_LEADER_OR_FOLLOWER. **FIELD** / **DESCRIPTION** node_id: The ID of the associated node. host: The node's hostname. port: The node's port. rack: The rack of the node, or null if it has not been assigned to a rack.

```
Produce Response (Version: 13) => { (responses) throttle_time_ms node_endpoints<t
  responses => { topic_id (partition_responses) }
    topic_id => UUID
    partition_responses => { index error_code base_offset log_append_time_ms log_
      index => INT32
      error_code => INT16
      base_offset => INT64
```

```
      log_append_time_ms => INT64
      log_start_offset => INT64
      record_errors => { batch_index batch_index_error_message }
        batch_index => INT32
        batch_index_error_message => COMPACT_NULLABLE_STRING
      error_message => COMPACT_NULLABLE_STRING
      current_leader<tag: 0> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
  throttle_time_ms => INT32
  node_endpoints<tag: 0> => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
responses	Each produce response.
topic_id	The unique topic ID
partition_responses	Each partition that we produced to within the topic.
index	The partition index.
error_code	The error code, or 0 if there was no error.
base_offset	The base offset.
log_append_time_ms	The timestamp returned by broker after appending the messages. If CreateTime is used for the topic, the timestamp will be -1. If LogAppendTime is used for the topic, the timestamp will be the broker local time when the messages are appended.
log_start_offset	The log start offset.
record_errors	The batch indices of records that caused the batch to be dropped.

batch_index	The batch index of the record that caused the batch to be dropped.
batch_index_error_message	The error message of the record that caused the batch to be dropped.
error_message	The global error message summarizing the common root cause of the records that caused the batch to be dropped.
current_leader<tag: 0>	The leader broker that the producer should use for future requests. FIELD: leader_id — The ID of the current leader or -1 if the leader is unknown. FIELD: leader_epoch — The latest known leader epoch.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
node_endpoints<tag: 0>	Endpoints for all current-leaders enumerated in PartitionProduceResponses, with errors NOT_LEADER_OR_FOLLOWER. FIELD: node_id — The ID of the associated node. FIELD: host — The node's hostname. FIELD: port — The node's port. FIELD: rack — The rack of the node, or null if it has not been assigned to a rack.

Fetch API (Key: 1):

Requests:

```
Fetch Request (Version: 4) => { replica_id max_wait_ms min_bytes max_bytes isolat
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition fetch_offset partition_max_bytes }
      partition => INT32
      fetch_offset => INT64
      partition_max_bytes => INT32
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	The topics to fetch.
topic	The name of the topic to fetch.
partitions	The partitions to fetch.

partition	The partition index.
fetch_offset	The message offset.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.

```
Fetch Request (Version: 5) => { replica_id max_wait_ms min_bytes max_bytes isolat
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition fetch_offset log_start_offset partition_max_bytes }
      partition => INT32
      fetch_offset => INT64
      log_start_offset => INT64
      partition_max_bytes => INT32
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to

	discard ABORTED transactional records.
topics	The topics to fetch.
topic	The name of the topic to fetch.
partitions	The partitions to fetch.
partition	The partition index.
fetch_offset	The message offset.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.

```
Fetch Request (Version: 6) => { replica_id max_wait_ms min_bytes max_bytes isolat
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition fetch_offset log_start_offset partition_max_bytes }
      partition => INT32
      fetch_offset => INT64
      log_start_offset => INT64
      partition_max_bytes => INT32
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.

max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	The topics to fetch.
topic	The name of the topic to fetch.
partitions	The partitions to fetch.
partition	The partition index.
fetch_offset	The message offset.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.

```
Fetch Request (Version: 7) => { replica_id max_wait_ms min_bytes max_bytes isolat
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition fetch_offset log_start_offset partition_max_bytes }
      partition => INT32
      fetch_offset => INT64
      log_start_offset => INT64
      partition_max_bytes => INT32
```

```
  forgotten_topics_data => { topic [partitions] }
    topic => STRING
    partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic	The name of the topic to fetch.
partitions	The partitions to fetch.
partition	The partition index.
fetch_offset	The message offset.
log_start_offset	The earliest available offset of the follower replica. The field is only

	used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic	The topic name.
partitions	The partitions indexes to forget.

```
Fetch Request (Version: 8) => { replica_id max_wait_ms min_bytes max_bytes isolat
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition fetch_offset log_start_offset partition_max_bytes }
      partition => INT32
      fetch_offset => INT64
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic [partitions] }
    topic => STRING
    partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.

isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic	The name of the topic to fetch.
partitions	The partitions to fetch.
partition	The partition index.
fetch_offset	The message offset.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic	The topic name.
partitions	The partitions indexes to forget.

```
Fetch Request (Version: 9) => { replica_id max_wait_ms min_bytes max_bytes isolat
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
```

```
  session_epoch => INT32
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition current_leader_epoch fetch_offset log_start_offset
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic [partitions] }
    topic => STRING
    partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.

topic	The name of the topic to fetch.
partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic	The topic name.
partitions	The partitions indexes to forget.

```
Fetch Request (Version: 10) => { replica_id max_wait_ms min_bytes max_bytes isola
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition current_leader_epoch fetch_offset log_start_offset
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic [partitions] }
    topic => STRING
    partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic	The name of the topic to fetch.
partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.

forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic	The topic name.
partitions	The partitions indexes to forget.

```
Fetch Request (Version: 11) => { replica_id max_wait_ms min_bytes max_bytes isola
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition current_leader_epoch fetch_offset log_start_offset
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic [partitions] }
    topic => STRING
    partitions => INT32
  rack_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible.

	With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic	The name of the topic to fetch.
partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic	The topic name.
partitions	The partitions indexes to forget.
rack_id	Rack ID of the consumer making this request.

```
Fetch Request (Version: 12) => { replica_id max_wait_ms min_bytes max_bytes isola
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
```

```
  session_id => INT32
  session_epoch => INT32
  topics => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => { partition current_leader_epoch fetch_offset last_fetched_epoc
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      last_fetched_epoch => INT32
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => INT32
  rack_id => COMPACT_STRING
  cluster_id<tag: 0> => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.

session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic	The name of the topic to fetch.
partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
last_fetched_epoch	The epoch of the last fetched record or -1 if there is none.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic	The topic name.
partitions	The partitions indexes to forget.
rack_id	Rack ID of the consumer making this request.
cluster_id<tag: 0>	The clusterId if known. This is used to validate metadata fetches prior to broker registration.

```
Fetch Request (Version: 13) => { replica_id max_wait_ms min_bytes max_bytes isola
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic_id (partitions) }
    topic_id => UUID
```

```
    partitions => { partition current_leader_epoch fetch_offset last_fetched_epoc
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      last_fetched_epoch => INT32
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
  rack_id => COMPACT_STRING
  cluster_id<tag: 0> => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.

topic_id	The unique topic ID.
partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
last_fetched_epoch	The epoch of the last fetched record or -1 if there is none.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic_id	The unique topic ID.
partitions	The partitions indexes to forget.
rack_id	Rack ID of the consumer making this request.
cluster_id<tag: 0>	The clusterId if known. This is used to validate metadata fetches prior to broker registration.

```
Fetch Request (Version: 14) => { replica_id max_wait_ms min_bytes max_bytes isola
  replica_id => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition current_leader_epoch fetch_offset last_fetched_epoc
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      last_fetched_epoch => INT32
```

```
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
  rack_id => COMPACT_STRING
  cluster_id<tag: 0> => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic_id	The unique topic ID.
partitions	The partitions to fetch.
partition	The partition index.

current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
last_fetched_epoch	The epoch of the last fetched record or -1 if there is none.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic_id	The unique topic ID.
partitions	The partitions indexes to forget.
rack_id	Rack ID of the consumer making this request.
cluster_id<tag: 0>	The clusterId if known. This is used to validate metadata fetches prior to broker registration.

```
Fetch Request (Version: 15) => { max_wait_ms min_bytes max_bytes isolation_level
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition current_leader_epoch fetch_offset last_fetched_epoc
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      last_fetched_epoch => INT32
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
  rack_id => COMPACT_STRING
  cluster_id<tag: 0> => COMPACT_NULLABLE_STRING
```

```
  replica_state<tag: 1> => { replica_id replica_epoch }
    replica_id => INT32
    replica_epoch => INT64
```

Request header version: 2

FIELD	DESCRIPTION
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic_id	The unique topic ID.
partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
last_fetched_epoch	The epoch of the last fetched record or -1 if there is none.

log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic_id	The unique topic ID.
partitions	The partitions indexes to forget.
rack_id	Rack ID of the consumer making this request.
cluster_id<tag: 0>	The clusterId if known. This is used to validate metadata fetches prior to broker registration.
replica_state<tag: 1>	The state of the replica in the follower.

FIELD	DESCRIPTION
replica_id	The replica ID of the follower, or -1 if this request is from a consumer.
replica_epoch	The epoch of this follower, or -1 if not available.

```
Fetch Request (Version: 16) => { max_wait_ms min_bytes max_bytes isolation_level
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition current_leader_epoch fetch_offset last_fetched_epoc
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      last_fetched_epoch => INT32
      log_start_offset => INT64
      partition_max_bytes => INT32
  forgotten_topics_data => { topic_id (partitions) }
```

```
    topic_id => UUID
    partitions => INT32
  rack_id => COMPACT_STRING
  cluster_id<tag: 0> => COMPACT_NULLABLE_STRING
  replica_state<tag: 1> => { replica_id replica_epoch }
    replica_id => INT32
    replica_epoch => INT64
```

Request header version: 2

FIELD	DESCRIPTION
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic_id	The unique topic ID.
partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.

fetch_offset	The message offset.
last_fetched_epoch	The epoch of the last fetched record or -1 if there is none.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic_id	The unique topic ID.
partitions	The partitions indexes to forget.
rack_id	Rack ID of the consumer making this request.
cluster_id<tag: 0>	The clusterId if known. This is used to validate metadata fetches prior to broker registration.
replica_state<tag: 1>	The state of the replica in the follower.

FIELD	DESCRIPTION
replica_id	The replica ID of the follower, or -1 if this request is from a consumer.
replica_epoch	The epoch of this follower, or -1 if not available.

```
Fetch Request (Version: 17) => { max_wait_ms min_bytes max_bytes isolation_level
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition current_leader_epoch fetch_offset last_fetched_epoc
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
```

```
      last_fetched_epoch => INT32
      log_start_offset => INT64
      partition_max_bytes => INT32
      replica_directory_id<tag: 0> => UUID
  forgotten_topics_data => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
  rack_id => COMPACT_STRING
  cluster_id<tag: 0> => COMPACT_NULLABLE_STRING
  replica_state<tag: 1> => { replica_id replica_epoch }
    replica_id => INT32
    replica_epoch => INT64
```

Request header version: 2

FIELD	DESCRIPTION
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic_id	The unique topic ID.

partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
last_fetched_epoch	The epoch of the last fetched record or -1 if there is none.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
replica_directory_id<tag: 0>	The directory id of the follower fetching.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic_id	The unique topic ID.
partitions	The partitions indexes to forget.
rack_id	Rack ID of the consumer making this request.
cluster_id<tag: 0>	The clusterId if known. This is used to validate metadata fetches prior to broker registration.
replica_state<tag: 1>	The state of the replica in the follower.

FIELD	DESCRIPTION
replica_id	The replica ID of the follower, or -1 if this request is from a consumer.
replica_epoch	The epoch of this follower, or -1 if not available.

```
Fetch Request (Version: 18) => { max_wait_ms min_bytes max_bytes isolation_level
  max_wait_ms => INT32
  min_bytes => INT32
```

```
  max_bytes => INT32
  isolation_level => INT8
  session_id => INT32
  session_epoch => INT32
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition current_leader_epoch fetch_offset last_fetched_epoc
      partition => INT32
      current_leader_epoch => INT32
      fetch_offset => INT64
      last_fetched_epoch => INT32
      log_start_offset => INT64
      partition_max_bytes => INT32
      replica_directory_id<tag: 0> => UUID
      high_watermark<tag: 1> => INT64
  forgotten_topics_data => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
  rack_id => COMPACT_STRING
  cluster_id<tag: 0> => COMPACT_NULLABLE_STRING
  replica_state<tag: 1> => { replica_id replica_epoch }
    replica_id => INT32
    replica_epoch => INT64
```

Request header version: 2

FIELD	DESCRIPTION
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED

	transactional records.
session_id	The fetch session ID.
session_epoch	The fetch session epoch, which is used for ordering requests in a session.
topics	The topics to fetch.
topic_id	The unique topic ID.
partitions	The partitions to fetch.
partition	The partition index.
current_leader_epoch	The current leader epoch of the partition.
fetch_offset	The message offset.
last_fetched_epoch	The epoch of the last fetched record or -1 if there is none.
log_start_offset	The earliest available offset of the follower replica. The field is only used when the request is sent by the follower.
partition_max_bytes	The maximum bytes to fetch from this partition. See KIP-74 for cases where this limit may not be honored.
replica_directory_id<tag: 0>	The directory id of the follower fetching.
high_watermark<tag: 1>	The high-watermark known by the replica. -1 if the high-watermark is not known and 9223372036854775807 if the feature is not supported.
forgotten_topics_data	In an incremental fetch request, the partitions to remove.
topic_id	The unique topic ID.
partitions	The partitions indexes to forget.
rack_id	Rack ID of the consumer making this request.
cluster_id<tag: 0>	The clusterId if known. This is used to validate metadata fetches prior to broker registration.

replica_state<tag: 1> The state of the replica in the follower.

FIELD	DESCRIPTION
replica_id	The replica ID of the follower, or -1 if this request is from a consumer.
replica_epoch	The epoch of this follower, or -1 if not available.

Responses:

```
Fetch Response (Version: 4) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { topic [partitions] }
    topic => STRING
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      records => NULLABLE_RECORDS
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The response topics.
topic	The topic name.
partitions	The topic partitions.
partition_index	The partition index.

error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
records	The record data.

```
Fetch Response (Version: 5) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { topic [partitions] }
    topic => STRING
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      records => NULLABLE_RECORDS
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The response topics.
topic	The topic name.

partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
records	The record data.

```
Fetch Response (Version: 6) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { topic [partitions] }
    topic => STRING
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      records => NULLABLE_RECORDS
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any

	quota.
responses	The response topics.
topic	The topic name.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
records	The record data.

```
Fetch Response (Version: 7) => { throttle_time_ms error_code session_id [response
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic [partitions] }
    topic => STRING
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      records => NULLABLE_RECORDS
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic	The topic name.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
records	The record data.

```
Fetch Response (Version: 8) => { throttle_time_ms error_code session_id [response
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic [partitions] }
    topic => STRING
```

```
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      records => NULLABLE_RECORDS
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic	The topic name.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.

first_offset	The first offset in the aborted transaction.
records	The record data.

```
Fetch Response (Version: 9) => { throttle_time_ms error_code session_id [response
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic [partitions] }
    topic => STRING
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      records => NULLABLE_RECORDS
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic	The topic name.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.

high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
records	The record data.

```
Fetch Response (Version: 10) => { throttle_time_ms error_code session_id [respons
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic [partitions] }
    topic => STRING
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      records => NULLABLE_RECORDS
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.

session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic	The topic name.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
records	The record data.

```
Fetch Response (Version: 11) => { throttle_time_ms error_code session_id [respons
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic [partitions] }
    topic => STRING
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      preferred_read_replica => INT32
```

```
      records => NULLABLE_RECORDS
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic	The topic name.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
preferred_read_replica	The preferred read replica for the consumer to use on its next fetch request.
records	The record data.

```
Fetch Response (Version: 12) => { throttle_time_ms error_code session_id (respons
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      preferred_read_replica => INT32
      records => COMPACT_NULLABLE_RECORDS
      diverging_epoch<tag: 0> => { epoch end_offset }
        epoch => INT32
        end_offset => INT64
      current_leader<tag: 1> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
      snapshot_id<tag: 2> => { end_offset epoch }
        end_offset => INT64
        epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic	The topic name.

partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
preferred_read_replica	The preferred read replica for the consumer to use on its next fetch request.
records	The record data.
diverging_epoch<tag: 0>	In case divergence is detected based on the `LastFetchedEpoch` and `FetchOffset` in the request, this field indicates the largest epoch and its end offset such that subsequent records are known to diverge.

FIELD	DESCRIPTION
epoch	The largest epoch.
end_offset	The end offset of the epoch.

current_leader<tag: 1> The current leader of the partition.

FIELD	DESCRIPTION
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.

snapshot_id<tag: 2> In the case of fetching an offset less than the LogStartOffset, this is the end offset and epoch that should be used in the FetchSnapshot request.

FIELD	DESCRIPTION
end_offset	The end offset of the epoch.
epoch	The largest epoch.

```
Fetch Response (Version: 13) => { throttle_time_ms error_code session_id (respons
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      preferred_read_replica => INT32
      records => COMPACT_NULLABLE_RECORDS
      diverging_epoch<tag: 0> => { epoch end_offset }
        epoch => INT32
        end_offset => INT64
      current_leader<tag: 1> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
```

```
      snapshot_id<tag: 2> => { end_offset epoch }
        end_offset => INT64
        epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
preferred_read_replica	The preferred read replica for the consumer to use on its next fetch request.
records	The record data.

diverging_epoch<tag: 0>	In case divergence is detected based on the `LastFetchedEpoch` and `FetchOffset` in the request, this field indicates the largest epoch and its end offset such that subsequent records are known to diverge. **FIELD** / **DESCRIPTION** epoch: The largest epoch. end_offset: The end offset of the epoch.
current_leader<tag: 1>	The current leader of the partition. **FIELD** / **DESCRIPTION** leader_id: The ID of the current leader or -1 if the leader is unknown. leader_epoch: The latest known leader epoch.
snapshot_id<tag: 2>	In the case of fetching an offset less than the LogStartOffset, this is the end offset and epoch that should be used in the FetchSnapshot request. **FIELD** / **DESCRIPTION** end_offset: The end offset of the epoch. epoch: The largest epoch.

```
Fetch Response (Version: 14) => { throttle_time_ms error_code session_id (respons
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
```

```
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      preferred_read_replica => INT32
      records => COMPACT_NULLABLE_RECORDS
      diverging_epoch<tag: 0> => { epoch end_offset }
        epoch => INT32
        end_offset => INT64
      current_leader<tag: 1> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
      snapshot_id<tag: 2> => { end_offset epoch }
        end_offset => INT64
        epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).

log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
preferred_read_replica	The preferred read replica for the consumer to use on its next fetch request.
records	The record data.
diverging_epoch<tag: 0>	In case divergence is detected based on the `LastFetchedEpoch` and `FetchOffset` in the request, this field indicates the largest epoch and its end offset such that subsequent records are known to diverge.

FIELD	DESCRIPTION
epoch	The largest epoch.
end_offset	The end offset of the epoch.

current_leader<tag: 1>	The current leader of the partition.

FIELD	DESCRIPTION
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.

snapshot_id<tag: 2>	In the case of fetching an offset less than the LogStartOffset, this is the end offset and epoch that should be used in the FetchSnapshot request.

FIELD	DESCRIPTION
end_offset	The end offset of the epoch.
epoch	The largest epoch.

```
Fetch Response (Version: 15) => { throttle_time_ms error_code session_id (respons
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      preferred_read_replica => INT32
      records => COMPACT_NULLABLE_RECORDS
      diverging_epoch<tag: 0> => { epoch end_offset }
        epoch => INT32
        end_offset => INT64
      current_leader<tag: 1> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
      snapshot_id<tag: 2> => { end_offset epoch }
        end_offset => INT64
        epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic_id	The unique topic ID.

partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
preferred_read_replica	The preferred read replica for the consumer to use on its next fetch request.
records	The record data.
diverging_epoch<tag: 0>	In case divergence is detected based on the `LastFetchedEpoch` and `FetchOffset` in the request, this field indicates the largest epoch and its end offset such that subsequent records are known to diverge.

FIELD	DESCRIPTION
epoch	The largest epoch.
end_offset	The end offset of the epoch.

current_leader<tag: 1> The current leader of the partition.

FIELD	DESCRIPTION
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.

snapshot_id<tag: 2> In the case of fetching an offset less than the LogStartOffset, this is the end offset and epoch that should be used in the FetchSnapshot request.

FIELD	DESCRIPTION
end_offset	The end offset of the epoch.
epoch	The largest epoch.

```
Fetch Response (Version: 16) => { throttle_time_ms error_code session_id (respons
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      preferred_read_replica => INT32
      records => COMPACT_NULLABLE_RECORDS
      diverging_epoch<tag: 0> => { epoch end_offset }
        epoch => INT32
        end_offset => INT64
      current_leader<tag: 1> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
```

```
      snapshot_id<tag: 2> => { end_offset epoch }
        end_offset => INT64
        epoch => INT32
  node_endpoints<tag: 0> => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.

<table>
<tr><td>preferred_read_replica</td><td>The preferred read replica for the consumer to use on its next fetch request.</td></tr>
<tr><td>records</td><td>The record data.</td></tr>
<tr><td>diverging_epoch<tag: 0></td><td>In case divergence is detected based on the `LastFetchedEpoch` and `FetchOffset` in the request, this field indicates the largest epoch and its end offset such that subsequent records are known to diverge.<table><tr><th>FIELD</th><th>DESCRIPTION</th></tr><tr><td>epoch</td><td>The largest epoch.</td></tr><tr><td>end_offset</td><td>The end offset of the epoch.</td></tr></table></td></tr>
<tr><td>current_leader<tag: 1></td><td>The current leader of the partition.<table><tr><th>FIELD</th><th>DESCRIPTION</th></tr><tr><td>leader_id</td><td>The ID of the current leader or -1 if the leader is unknown.</td></tr><tr><td>leader_epoch</td><td>The latest known leader epoch.</td></tr></table></td></tr>
<tr><td>snapshot_id<tag: 2></td><td>In the case of fetching an offset less than the LogStartOffset, this is the end offset and epoch that should be used in the FetchSnapshot request.<table><tr><th>FIELD</th><th>DESCRIPTION</th></tr><tr><td>end_offset</td><td>The end offset of the epoch.</td></tr><tr><td>epoch</td><td>The largest epoch.</td></tr></table></td></tr>
<tr><td>node_endpoints<tag: 0></td><td>Endpoints for all current-leaders enumerated in PartitionData, with errors NOT_LEADER_OR_FOLLOWER & FENCED_LEADER_EPOCH.<table><tr><th>FIELD</th><th>DESCRIPTION</th></tr><tr><td>node_id</td><td>The ID of the associated node.</td></tr></table></td></tr>
</table>

host	The node's hostname.
port	The node's port.
rack	The rack of the node, or null if it has not been assigned to a rack.

```
Fetch Response (Version: 17) => { throttle_time_ms error_code session_id (respons
  throttle_time_ms => INT32
  error_code => INT16
  session_id => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      preferred_read_replica => INT32
      records => COMPACT_NULLABLE_RECORDS
      diverging_epoch<tag: 0> => { epoch end_offset }
        epoch => INT32
        end_offset => INT64
      current_leader<tag: 1> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
      snapshot_id<tag: 2> => { end_offset epoch }
        end_offset => INT64
        epoch => INT32
  node_endpoints<tag: 0> => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.
topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
preferred_read_replica	The preferred read replica for the consumer to use on its next fetch request.
records	The record data.
diverging_epoch<tag: 0>	In case divergence is detected based on the `LastFetchedEpoch` and `FetchOffset` in the request, this field indicates the largest epoch and its end offset such that subsequent records are known to diverge.

FIELD	DESCRIPTION

	epoch: The largest epoch. end_offset: The end offset of the epoch.
current_leader<tag: 1>	The current leader of the partition. leader_id: The ID of the current leader or -1 if the leader is unknown. leader_epoch: The latest known leader epoch.
snapshot_id<tag: 2>	In the case of fetching an offset less than the LogStartOffset, this is the end offset and epoch that should be used in the FetchSnapshot request. end_offset: The end offset of the epoch. epoch: The largest epoch.
node_endpoints<tag: 0>	Endpoints for all current-leaders enumerated in PartitionData, with errors NOT_LEADER_OR_FOLLOWER & FENCED_LEADER_EPOCH. node_id: The ID of the associated node. host: The node's hostname. port: The node's port. rack: The rack of the node, or null if it has not been assigned to a rack.

```
Fetch Response (Version: 18) => { throttle_time_ms error_code session_id (respons
  throttle_time_ms => INT32
```

```
  error_code => INT16
  session_id => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code high_watermark last_stable_offset
      partition_index => INT32
      error_code => INT16
      high_watermark => INT64
      last_stable_offset => INT64
      log_start_offset => INT64
      aborted_transactions => { producer_id first_offset }
        producer_id => INT64
        first_offset => INT64
      preferred_read_replica => INT32
      records => COMPACT_NULLABLE_RECORDS
      diverging_epoch<tag: 0> => { epoch end_offset }
        epoch => INT32
        end_offset => INT64
      current_leader<tag: 1> => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
      snapshot_id<tag: 2> => { end_offset epoch }
        end_offset => INT64
        epoch => INT32
  node_endpoints<tag: 0> => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
session_id	The fetch session ID, or 0 if this is not part of a fetch session.
responses	The response topics.

topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no fetch error.
high_watermark	The current high water mark.
last_stable_offset	The last stable offset (or LSO) of the partition. This is the last offset such that the state of all transactional records prior to this offset have been decided (ABORTED or COMMITTED).
log_start_offset	The current log start offset.
aborted_transactions	The aborted transactions.
producer_id	The producer id associated with the aborted transaction.
first_offset	The first offset in the aborted transaction.
preferred_read_replica	The preferred read replica for the consumer to use on its next fetch request.
records	The record data.
diverging_epoch<tag: 0>	In case divergence is detected based on the `LastFetchedEpoch` and `FetchOffset` in the request, this field indicates the largest epoch and its end offset such that subsequent records are known to diverge.

FIELD	DESCRIPTION
epoch	The largest epoch.
end_offset	The end offset of the epoch.

current_leader<tag: 1>	The current leader of the partition.

FIELD	DESCRIPTION
leader_id	The ID of the current leader or -1 if the leader is unknown.

leader_epoch	The latest known leader epoch.

snapshot_id<tag: 2>	In the case of fetching an offset less than the LogStartOffset, this is the end offset and epoch that should be used in the FetchSnapshot request. **FIELD** / **DESCRIPTION** end_offset: The end offset of the epoch. epoch: The largest epoch.
node_endpoints<tag: 0>	Endpoints for all current-leaders enumerated in PartitionData, with errors NOT_LEADER_OR_FOLLOWER & FENCED_LEADER_EPOCH. **FIELD** / **DESCRIPTION** node_id: The ID of the associated node. host: The node's hostname. port: The node's port. rack: The rack of the node, or null if it has not been assigned to a rack.

ListOffsets API (Key: 2):

Requests:

```
ListOffsets Request (Version: 1) => { replica_id [topics] }
  replica_id => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index timestamp }
      partition_index => INT32
      timestamp => INT64
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 2) => { replica_id isolation_level [topics] }
  replica_id => INT32
  isolation_level => INT8
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index timestamp }
      partition_index => INT32
      timestamp => INT64
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.

name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 3) => { replica_id isolation_level [topics] }
  replica_id => INT32
  isolation_level => INT8
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index timestamp }
      partition_index => INT32
      timestamp => INT64
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 4) => { replica_id isolation_level [topics] }
  replica_id => INT32
  isolation_level => INT8
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index current_leader_epoch timestamp }
      partition_index => INT32
      current_leader_epoch => INT32
      timestamp => INT64
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
current_leader_epoch	The current leader epoch.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 5) => { replica_id isolation_level [topics] }
  replica_id => INT32
  isolation_level => INT8
  topics => { name [partitions] }
```

```
    name => STRING
    partitions => { partition_index current_leader_epoch timestamp }
      partition_index => INT32
      current_leader_epoch => INT32
      timestamp => INT64
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
current_leader_epoch	The current leader epoch.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 6) => { replica_id isolation_level (topics) }
  replica_id => INT32
  isolation_level => INT8
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index current_leader_epoch timestamp }
      partition_index => INT32
      current_leader_epoch => INT32
      timestamp => INT64
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
current_leader_epoch	The current leader epoch.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 7) => { replica_id isolation_level (topics) }
  replica_id => INT32
  isolation_level => INT8
  topics => { name (partitions) }
```

```
    name => COMPACT_STRING
    partitions => { partition_index current_leader_epoch timestamp }
      partition_index => INT32
      current_leader_epoch => INT32
      timestamp => INT64
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
current_leader_epoch	The current leader epoch.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 8) => { replica_id isolation_level (topics) }
  replica_id => INT32
  isolation_level => INT8
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index current_leader_epoch timestamp }
      partition_index => INT32
      current_leader_epoch => INT32
      timestamp => INT64
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
current_leader_epoch	The current leader epoch.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 9) => { replica_id isolation_level (topics) }
  replica_id => INT32
  isolation_level => INT8
  topics => { name (partitions) }
```

```
    name => COMPACT_STRING
    partitions => { partition_index current_leader_epoch timestamp }
      partition_index => INT32
      current_leader_epoch => INT32
      timestamp => INT64
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
current_leader_epoch	The current leader epoch.
timestamp	The current timestamp.

```
ListOffsets Request (Version: 10) => { replica_id isolation_level (topics) timeou
  replica_id => INT32
  isolation_level => INT8
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index current_leader_epoch timestamp }
      partition_index => INT32
      current_leader_epoch => INT32
      timestamp => INT64
```

```
  timeout_ms => INT32
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
current_leader_epoch	The current leader epoch.
timestamp	The current timestamp.
timeout_ms	The timeout to await a response in milliseconds for requests that require reading from remote storage for topics enabled with tiered storage.

```
ListOffsets Request (Version: 11) => { replica_id isolation_level (topics) timeou
  replica_id => INT32
  isolation_level => INT8
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index current_leader_epoch timestamp }
      partition_index => INT32
      current_leader_epoch => INT32
```

```
      timestamp => INT64
  timeout_ms => INT32
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the requester, or -1 if this request is being made by a normal consumer.
isolation_level	This setting controls the visibility of transactional records. Using READ_UNCOMMITTED (isolation_level = 0) makes all records visible. With READ_COMMITTED (isolation_level = 1), non-transactional and COMMITTED transactional records are visible. To be more concrete, READ_COMMITTED returns all data from offsets smaller than the current LSO (last stable offset), and enables the inclusion of the list of aborted transactions in the result, which allows consumers to discard ABORTED transactional records.
topics	Each topic in the request.
name	The topic name.
partitions	Each partition in the request.
partition_index	The partition index.
current_leader_epoch	The current leader epoch.
timestamp	The current timestamp.
timeout_ms	The timeout to await a response in milliseconds for requests that require reading from remote storage for topics enabled with tiered storage.

Responses:

```
ListOffsets Response (Version: 1) => { [topics] }
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code timestamp offset }
      partition_index => INT32
      error_code => INT16
```

```
      timestamp => INT64
      offset => INT64
```

Response header version: 0

FIELD	DESCRIPTION
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.

```
ListOffsets Response (Version: 2) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code timestamp offset }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.

partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.

```
ListOffsets Response (Version: 3) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code timestamp offset }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.

```
ListOffsets Response (Version: 4) => { throttle_time_ms [topics] }
```

```
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code timestamp offset leader_epoch }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
      leader_epoch => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.
leader_epoch	The leader epoch associated with the returned offset.

```
ListOffsets Response (Version: 5) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code timestamp offset leader_epoch }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
      leader_epoch => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.
leader_epoch	The leader epoch associated with the returned offset.

```
ListOffsets Response (Version: 6) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code timestamp offset leader_epoch }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
      leader_epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.

name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.
leader_epoch	The leader epoch associated with the returned offset.

```
ListOffsets Response (Version: 7) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code timestamp offset leader_epoch }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
      leader_epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.

offset	The returned offset.
leader_epoch	The leader epoch associated with the returned offset.

```
ListOffsets Response (Version: 8) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code timestamp offset leader_epoch }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
      leader_epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.
leader_epoch	The leader epoch associated with the returned offset.

```
ListOffsets Response (Version: 9) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
```

```
    partitions => { partition_index error_code timestamp offset leader_epoch }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
      leader_epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.
leader_epoch	The leader epoch associated with the returned offset.

```
ListOffsets Response (Version: 10) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code timestamp offset leader_epoch }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
      leader_epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.
leader_epoch	The leader epoch associated with the returned offset.

```
ListOffsets Response (Version: 11) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code timestamp offset leader_epoch }
      partition_index => INT32
      error_code => INT16
      timestamp => INT64
      offset => INT64
      leader_epoch => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.

partitions	Each partition in the response.
partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
timestamp	The timestamp associated with the returned offset.
offset	The returned offset.
leader_epoch	The leader epoch associated with the returned offset.

Metadata API (Key: 3):

Requests:

```
Metadata Request (Version: 0) => { [topics] }
  topics => { name }
    name => STRING
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.

```
Metadata Request (Version: 1) => { ?[topics] }
  topics => { name }
    name => STRING
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.

```
Metadata Request (Version: 2) => { ?[topics] }
```

```
  topics => { name }
    name => STRING
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.

```
Metadata Request (Version: 3) => { ?[topics] }
  topics => { name }
    name => STRING
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.

```
Metadata Request (Version: 4) => { ?[topics] allow_auto_topic_creation }
  topics => { name }
    name => STRING
  allow_auto_topic_creation => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.

```
Metadata Request (Version: 5) => { ?[topics] allow_auto_topic_creation }
  topics => { name }
    name => STRING
  allow_auto_topic_creation => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.

```
Metadata Request (Version: 6) => { ?[topics] allow_auto_topic_creation }
  topics => { name }
    name => STRING
  allow_auto_topic_creation => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.

```
Metadata Request (Version: 7) => { ?[topics] allow_auto_topic_creation }
  topics => { name }
    name => STRING
  allow_auto_topic_creation => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.

```
Metadata Request (Version: 8) => { ?[topics] allow_auto_topic_creation include_cl
  topics => { name }
    name => STRING
  allow_auto_topic_creation => BOOLEAN
  include_cluster_authorized_operations => BOOLEAN
  include_topic_authorized_operations => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.
include_cluster_authorized_operations	Whether to include cluster authorized operations.
include_topic_authorized_operations	Whether to include topic authorized operations.

```
Metadata Request (Version: 9) => { ?(topics) allow_auto_topic_creation include_cl
  topics => { name }
    name => COMPACT_STRING
  allow_auto_topic_creation => BOOLEAN
  include_cluster_authorized_operations => BOOLEAN
  include_topic_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.
include_cluster_authorized_operations	Whether to include cluster authorized operations.
include_topic_authorized_operations	Whether to include topic authorized operations.

```
Metadata Request (Version: 10) => { ?(topics) allow_auto_topic_creation include_c
  topics => { topic_id name }
    topic_id => UUID
    name => COMPACT_NULLABLE_STRING
  allow_auto_topic_creation => BOOLEAN
  include_cluster_authorized_operations => BOOLEAN
  include_topic_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
topic_id	The topic id.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.
include_cluster_authorized_operations	Whether to include cluster authorized operations.
include_topic_authorized_operations	Whether to include topic authorized operations.

```
Metadata Request (Version: 11) => { ?(topics) allow_auto_topic_creation include_t
  topics => { topic_id name }
    topic_id => UUID
    name => COMPACT_NULLABLE_STRING
  allow_auto_topic_creation => BOOLEAN
  include_topic_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
topic_id	The topic id.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.
include_topic_authorized_operations	Whether to include topic authorized operations.

```
Metadata Request (Version: 12) => { ?(topics) allow_auto_topic_creation include_t
  topics => { topic_id name }
    topic_id => UUID
    name => COMPACT_NULLABLE_STRING
  allow_auto_topic_creation => BOOLEAN
  include_topic_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
topic_id	The topic id.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is

	configured to do so.
include_topic_authorized_operations	Whether to include topic authorized operations.

```
Metadata Request (Version: 13) => { ?(topics) allow_auto_topic_creation include_t
  topics => { topic_id name }
    topic_id => UUID
    name => COMPACT_NULLABLE_STRING
  allow_auto_topic_creation => BOOLEAN
  include_topic_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to fetch metadata for.
topic_id	The topic id.
name	The topic name.
allow_auto_topic_creation	If this is true, the broker may auto-create topics that we requested which do not already exist, if it is configured to do so.
include_topic_authorized_operations	Whether to include topic authorized operations.

Responses:

```
Metadata Response (Version: 0) => { [brokers] [topics] }
  brokers => { node_id host port }
    node_id => INT32
    host => STRING
    port => INT32
  topics => { error_code name [partitions] }
    error_code => INT16
    name => STRING
    partitions => { error_code partition_index leader_id [replica_nodes] [isr_nod
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      replica_nodes => INT32
      isr_nodes => INT32
```

Response header version: 0

FIELD	DESCRIPTION
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.

replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.

```
Metadata Response (Version: 1) => { [brokers] controller_id [topics] }
  brokers => { node_id host port rack }
    node_id => INT32
    host => STRING
    port => INT32
    rack => NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal [partitions] }
    error_code => INT16
    name => STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id [replica_nodes] [isr_nod
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      replica_nodes => INT32
      isr_nodes => INT32
```

Response header version: 0

FIELD	DESCRIPTION
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.

name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.

```
Metadata Response (Version: 2) => { [brokers] cluster_id controller_id [topics] }
  brokers => { node_id host port rack }
    node_id => INT32
    host => STRING
    port => INT32
    rack => NULLABLE_STRING
  cluster_id => NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal [partitions] }
    error_code => INT16
    name => STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id [replica_nodes] [isr_nod
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      replica_nodes => INT32
      isr_nodes => INT32
```

Response header version: 0

FIELD	DESCRIPTION
brokers	A list of brokers present in the cluster.

node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.

```
Metadata Response (Version: 3) => { throttle_time_ms [brokers] cluster_id control
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => STRING
    port => INT32
    rack => NULLABLE_STRING
  cluster_id => NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal [partitions] }
```

```
    error_code => INT16
    name => STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id [replica_nodes] [isr_nod
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      replica_nodes => INT32
      isr_nodes => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.

partition_index	The partition index.
leader_id	The ID of the leader broker.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.

```
Metadata Response (Version: 4) => { throttle_time_ms [brokers] cluster_id control
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => STRING
    port => INT32
    rack => NULLABLE_STRING
  cluster_id => NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal [partitions] }
    error_code => INT16
    name => STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id [replica_nodes] [isr_nod
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      replica_nodes => INT32
      isr_nodes => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.

rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.

```
Metadata Response (Version: 5) => { throttle_time_ms [brokers] cluster_id control
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => STRING
    port => INT32
    rack => NULLABLE_STRING
  cluster_id => NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal [partitions] }
    error_code => INT16
    name => STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id [replica_nodes] [isr_nod
      error_code => INT16
      partition_index => INT32
```

```
      leader_id => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.

replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.

```
Metadata Response (Version: 6) => { throttle_time_ms [brokers] cluster_id control
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => STRING
    port => INT32
    rack => NULLABLE_STRING
  cluster_id => NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal [partitions] }
    error_code => INT16
    name => STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id [replica_nodes] [isr_nod
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.

cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.

```
Metadata Response (Version: 7) => { throttle_time_ms [brokers] cluster_id control
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => STRING
    port => INT32
    rack => NULLABLE_STRING
  cluster_id => NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal [partitions] }
    error_code => INT16
    name => STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id leader_epoch [replica_no
      error_code => INT16
      partition_index => INT32
```

```
      leader_id => INT32
      leader_epoch => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.

leader_epoch	The leader epoch of this partition.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.

```
Metadata Response (Version: 8) => { throttle_time_ms [brokers] cluster_id control
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => STRING
    port => INT32
    rack => NULLABLE_STRING
  cluster_id => NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal [partitions] topic_authorized_operation
    error_code => INT16
    name => STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id leader_epoch [replica_no
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      leader_epoch => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
    topic_authorized_operations => INT32
  cluster_authorized_operations => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.

node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
leader_epoch	The leader epoch of this partition.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.
topic_authorized_operations	32-bit bitfield to represent authorized operations for this topic.
cluster_authorized_operations	32-bit bitfield to represent authorized operations for this cluster.

```
Metadata Response (Version: 9) => { throttle_time_ms (brokers) cluster_id control
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
  cluster_id => COMPACT_NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name is_internal (partitions) topic_authorized_operation
    error_code => INT16
    name => COMPACT_STRING
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id leader_epoch (replica_no
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      leader_epoch => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
    topic_authorized_operations => INT32
  cluster_authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.

cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
leader_epoch	The leader epoch of this partition.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.
topic_authorized_operations	32-bit bitfield to represent authorized operations for this topic.
cluster_authorized_operations	32-bit bitfield to represent authorized operations for this cluster.

```
Metadata Response (Version: 10) => { throttle_time_ms (brokers) cluster_id contro
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

```
  cluster_id => COMPACT_NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name topic_id is_internal (partitions) topic_authorized_
    error_code => INT16
    name => COMPACT_STRING
    topic_id => UUID
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id leader_epoch (replica_no
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      leader_epoch => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
    topic_authorized_operations => INT32
  cluster_authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.

name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
topic_id	The topic id. Zero for non-existing topics queried by name. This is never zero when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
leader_epoch	The leader epoch of this partition.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.
topic_authorized_operations	32-bit bitfield to represent authorized operations for this topic.
cluster_authorized_operations	32-bit bitfield to represent authorized operations for this cluster.

```
Metadata Response (Version: 11) => { throttle_time_ms (brokers) cluster_id contro
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
  cluster_id => COMPACT_NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name topic_id is_internal (partitions) topic_authorized_
    error_code => INT16
```

```
    name => COMPACT_STRING
    topic_id => UUID
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id leader_epoch (replica_no
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      leader_epoch => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
    topic_authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.

topic_id	The topic id. Zero for non-existing topics queried by name. This is never zero when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
leader_epoch	The leader epoch of this partition.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.
topic_authorized_operations	32-bit bitfield to represent authorized operations for this topic.

```
Metadata Response (Version: 12) => { throttle_time_ms (brokers) cluster_id contro
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
  cluster_id => COMPACT_NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name topic_id is_internal (partitions) topic_authorized_
    error_code => INT16
    name => COMPACT_NULLABLE_STRING
    topic_id => UUID
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id leader_epoch (replica_no
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
```

```
      leader_epoch => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
    topic_authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
topic_id	The topic id. Zero for non-existing topics queried by name. This is never zero when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.

error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
leader_epoch	The leader epoch of this partition.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.
topic_authorized_operations	32-bit bitfield to represent authorized operations for this topic.

```
Metadata Response (Version: 13) => { throttle_time_ms (brokers) cluster_id contro
  throttle_time_ms => INT32
  brokers => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
  cluster_id => COMPACT_NULLABLE_STRING
  controller_id => INT32
  topics => { error_code name topic_id is_internal (partitions) topic_authorized_
    error_code => INT16
    name => COMPACT_NULLABLE_STRING
    topic_id => UUID
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id leader_epoch (replica_no
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      leader_epoch => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      offline_replicas => INT32
    topic_authorized_operations => INT32
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
brokers	A list of brokers present in the cluster.
node_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller broker.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name. Null for non-existing topics queried by ID. This is never null when ErrorCode is zero. One of Name and TopicId is always populated.
topic_id	The topic id. Zero for non-existing topics queried by name. This is never zero when ErrorCode is zero. One of Name and TopicId is always populated.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.

leader_epoch	The leader epoch of this partition.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
offline_replicas	The set of offline replicas of this partition.
topic_authorized_operations	32-bit bitfield to represent authorized operations for this topic.
error_code	The top-level error code, or 0 if there was no error.

OffsetCommit API (Key: 8):

Requests:

```
OffsetCommit Request (Version: 2) => { group_id generation_id_or_member_epoch mem
  group_id => STRING
  generation_id_or_member_epoch => INT32
  member_id => STRING
  retention_time_ms => INT64
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_metadata }
      partition_index => INT32
      committed_offset => INT64
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
retention_time_ms	The time period in ms to retain the offset.

topics	The topics to commit offsets for.
name	The topic name.
partitions	Each partition to commit offsets for.
partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_metadata	Any associated metadata the client wants to keep.

```
OffsetCommit Request (Version: 3) => { group_id generation_id_or_member_epoch mem
  group_id => STRING
  generation_id_or_member_epoch => INT32
  member_id => STRING
  retention_time_ms => INT64
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_metadata }
      partition_index => INT32
      committed_offset => INT64
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
retention_time_ms	The time period in ms to retain the offset.
topics	The topics to commit offsets for.
name	The topic name.
partitions	Each partition to commit offsets for.

partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_metadata	Any associated metadata the client wants to keep.

```
OffsetCommit Request (Version: 4) => { group_id generation_id_or_member_epoch mem
  group_id => STRING
  generation_id_or_member_epoch => INT32
  member_id => STRING
  retention_time_ms => INT64
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_metadata }
      partition_index => INT32
      committed_offset => INT64
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
retention_time_ms	The time period in ms to retain the offset.
topics	The topics to commit offsets for.
name	The topic name.
partitions	Each partition to commit offsets for.
partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_metadata	Any associated metadata the client wants to keep.

```
OffsetCommit Request (Version: 5) => { group_id generation_id_or_member_epoch mem
  group_id => STRING
  generation_id_or_member_epoch => INT32
  member_id => STRING
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_metadata }
      partition_index => INT32
      committed_offset => INT64
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
topics	The topics to commit offsets for.
name	The topic name.
partitions	Each partition to commit offsets for.
partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_metadata	Any associated metadata the client wants to keep.

```
OffsetCommit Request (Version: 6) => { group_id generation_id_or_member_epoch mem
  group_id => STRING
  generation_id_or_member_epoch => INT32
  member_id => STRING
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
```

```
      committed_offset => INT64
      committed_leader_epoch => INT32
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
topics	The topics to commit offsets for.
name	The topic name.
partitions	Each partition to commit offsets for.
partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of this partition.
committed_metadata	Any associated metadata the client wants to keep.

```
OffsetCommit Request (Version: 7) => { group_id generation_id_or_member_epoch mem
  group_id => STRING
  generation_id_or_member_epoch => INT32
  member_id => STRING
  group_instance_id => NULLABLE_STRING
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
topics	The topics to commit offsets for.
name	The topic name.
partitions	Each partition to commit offsets for.
partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of this partition.
committed_metadata	Any associated metadata the client wants to keep.

```
OffsetCommit Request (Version: 8) => { group_id generation_id_or_member_epoch mem
  group_id => COMPACT_STRING
  generation_id_or_member_epoch => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      committed_metadata => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
topics	The topics to commit offsets for.
name	The topic name.
partitions	Each partition to commit offsets for.
partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of this partition.
committed_metadata	Any associated metadata the client wants to keep.

```
OffsetCommit Request (Version: 9) => { group_id generation_id_or_member_epoch mem
  group_id => COMPACT_STRING
  generation_id_or_member_epoch => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      committed_metadata => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION

group_id	The unique group identifier.
generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
topics	The topics to commit offsets for.
name	The topic name.
partitions	Each partition to commit offsets for.
partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of this partition.
committed_metadata	Any associated metadata the client wants to keep.

```
OffsetCommit Request (Version: 10) => { group_id generation_id_or_member_epoch me
  group_id => COMPACT_STRING
  generation_id_or_member_epoch => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      committed_metadata => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The unique group identifier.

generation_id_or_member_epoch	The generation of the group if using the classic group protocol or the member epoch if using the consumer protocol.
member_id	The member ID assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
topics	The topics to commit offsets for.
topic_id	The topic ID.
partitions	Each partition to commit offsets for.
partition_index	The partition index.
committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of this partition.
committed_metadata	Any associated metadata the client wants to keep.

Responses:

```
OffsetCommit Response (Version: 2) => { [topics] }
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.

error_code	The error code, or 0 if there was no error.

```
OffsetCommit Response (Version: 3) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
OffsetCommit Response (Version: 4) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
OffsetCommit Response (Version: 5) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
OffsetCommit Response (Version: 6) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
```

```
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
OffsetCommit Response (Version: 7) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.

partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
OffsetCommit Response (Version: 8) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
OffsetCommit Response (Version: 9) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
OffsetCommit Response (Version: 10) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
topic_id	The topic ID.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

OffsetFetch API (Key: 9):

Requests:

```
OffsetFetch Request (Version: 1) => { group_id [topics] }
  group_id => STRING
  topics => { name [partition_indexes] }
    name => STRING
    partition_indexes => INT32
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group to fetch offsets for.
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.

```
OffsetFetch Request (Version: 2) => { group_id ?[topics] }
  group_id => STRING
  topics => { name [partition_indexes] }
    name => STRING
    partition_indexes => INT32
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group to fetch offsets for.
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.

```
OffsetFetch Request (Version: 3) => { group_id ?[topics] }
  group_id => STRING
  topics => { name [partition_indexes] }
```

```
    name => STRING
    partition_indexes => INT32
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group to fetch offsets for.
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.

```
OffsetFetch Request (Version: 4) => { group_id ?[topics] }
  group_id => STRING
  topics => { name [partition_indexes] }
    name => STRING
    partition_indexes => INT32
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group to fetch offsets for.
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.

```
OffsetFetch Request (Version: 5) => { group_id ?[topics] }
  group_id => STRING
  topics => { name [partition_indexes] }
    name => STRING
    partition_indexes => INT32
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group to fetch offsets for.
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.

```
OffsetFetch Request (Version: 6) => { group_id ?(topics) }
  group_id => COMPACT_STRING
  topics => { name (partition_indexes) }
    name => COMPACT_STRING
    partition_indexes => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group to fetch offsets for.
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.

```
OffsetFetch Request (Version: 7) => { group_id ?(topics) require_stable }
  group_id => COMPACT_STRING
  topics => { name (partition_indexes) }
    name => COMPACT_STRING
    partition_indexes => INT32
  require_stable => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group to fetch offsets for.
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.
require_stable	Whether broker should hold on returning unstable offsets but set a retriable error code for the partitions.

```
OffsetFetch Request (Version: 8) => { (groups) require_stable }
  groups => { group_id ?(topics) }
    group_id => COMPACT_STRING
    topics => { name (partition_indexes) }
      name => COMPACT_STRING
      partition_indexes => INT32
  require_stable => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
groups	Each group we would like to fetch offsets for.
group_id	The group ID.
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.
require_stable	Whether broker should hold on returning unstable offsets but set a retriable error code for the partitions.

```
OffsetFetch Request (Version: 9) => { (groups) require_stable }
  groups => { group_id member_id member_epoch ?(topics) }
```

```
    group_id => COMPACT_STRING
    member_id => COMPACT_NULLABLE_STRING
    member_epoch => INT32
    topics => { name (partition_indexes) }
      name => COMPACT_STRING
      partition_indexes => INT32
  require_stable => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
groups	Each group we would like to fetch offsets for.
group_id	The group ID.
member_id	The member id.
member_epoch	The member epoch if using the new consumer protocol (KIP-848).
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
name	The topic name.
partition_indexes	The partition indexes we would like to fetch offsets for.
require_stable	Whether broker should hold on returning unstable offsets but set a retriable error code for the partitions.

```
OffsetFetch Request (Version: 10) => { (groups) require_stable }
  groups => { group_id member_id member_epoch ?(topics) }
    group_id => COMPACT_STRING
    member_id => COMPACT_NULLABLE_STRING
    member_epoch => INT32
    topics => { topic_id (partition_indexes) }
      topic_id => UUID
      partition_indexes => INT32
  require_stable => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
groups	Each group we would like to fetch offsets for.
group_id	The group ID.
member_id	The member id.
member_epoch	The member epoch if using the new consumer protocol (KIP-848).
topics	Each topic we would like to fetch offsets for, or null to fetch offsets for all topics.
topic_id	The topic ID.
partition_indexes	The partition indexes we would like to fetch offsets for.
require_stable	Whether broker should hold on returning unstable offsets but set a retriable error code for the partitions.

Responses:

```
OffsetFetch Response (Version: 1) => { [topics] }
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset metadata error_code }
      partition_index => INT32
      committed_offset => INT64
      metadata => NULLABLE_STRING
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.

metadata	The partition metadata.
error_code	The error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 2) => { [topics] error_code }
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset metadata error_code }
      partition_index => INT32
      committed_offset => INT64
      metadata => NULLABLE_STRING
      error_code => INT16
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.
metadata	The partition metadata.
error_code	The error code, or 0 if there was no error.
error_code	The top-level error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 3) => { throttle_time_ms [topics] error_code }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset metadata error_code }
      partition_index => INT32
      committed_offset => INT64
      metadata => NULLABLE_STRING
```

```
      error_code => INT16
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.
metadata	The partition metadata.
error_code	The error code, or 0 if there was no error.
error_code	The top-level error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 4) => { throttle_time_ms [topics] error_code }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset metadata error_code }
      partition_index => INT32
      committed_offset => INT64
      metadata => NULLABLE_STRING
      error_code => INT16
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to

	a quota violation, or zero if the request did not violate any quota.
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.
metadata	The partition metadata.
error_code	The error code, or 0 if there was no error.
error_code	The top-level error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 5) => { throttle_time_ms [topics] error_code }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_leader_epoch metad
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      metadata => NULLABLE_STRING
      error_code => INT16
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.

partition_index	The partition index.
committed_offset	The committed message offset.
committed_leader_epoch	The leader epoch.
metadata	The partition metadata.
error_code	The error code, or 0 if there was no error.
error_code	The top-level error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 6) => { throttle_time_ms (topics) error_code }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index committed_offset committed_leader_epoch metad
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      metadata => COMPACT_NULLABLE_STRING
      error_code => INT16
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.
committed_leader_epoch	The leader epoch.

metadata	The partition metadata.
error_code	The error code, or 0 if there was no error.
error_code	The top-level error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 7) => { throttle_time_ms (topics) error_code }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index committed_offset committed_leader_epoch metad
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      metadata => COMPACT_NULLABLE_STRING
      error_code => INT16
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.
committed_leader_epoch	The leader epoch.
metadata	The partition metadata.
error_code	The error code, or 0 if there was no error.
error_code	The top-level error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 8) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { group_id (topics) error_code }
    group_id => COMPACT_STRING
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => { partition_index committed_offset committed_leader_epoch met
        partition_index => INT32
        committed_offset => INT64
        committed_leader_epoch => INT32
        metadata => COMPACT_NULLABLE_STRING
        error_code => INT16
    error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	The responses per group id.
group_id	The group ID.
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.
committed_leader_epoch	The leader epoch.
metadata	The partition metadata.
error_code	The partition-level error code, or 0 if there was no error.
error_code	The group-level error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 9) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { group_id (topics) error_code }
    group_id => COMPACT_STRING
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => { partition_index committed_offset committed_leader_epoch met
        partition_index => INT32
        committed_offset => INT64
        committed_leader_epoch => INT32
        metadata => COMPACT_NULLABLE_STRING
        error_code => INT16
    error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	The responses per group id.
group_id	The group ID.
topics	The responses per topic.
name	The topic name.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.
committed_leader_epoch	The leader epoch.
metadata	The partition metadata.
error_code	The partition-level error code, or 0 if there was no error.
error_code	The group-level error code, or 0 if there was no error.

```
OffsetFetch Response (Version: 10) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { group_id (topics) error_code }
    group_id => COMPACT_STRING
    topics => { topic_id (partitions) }
      topic_id => UUID
      partitions => { partition_index committed_offset committed_leader_epoch met
        partition_index => INT32
        committed_offset => INT64
        committed_leader_epoch => INT32
        metadata => COMPACT_NULLABLE_STRING
        error_code => INT16
    error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	The responses per group id.
group_id	The group ID.
topics	The responses per topic.
topic_id	The topic ID.
partitions	The responses per partition.
partition_index	The partition index.
committed_offset	The committed message offset.
committed_leader_epoch	The leader epoch.
metadata	The partition metadata.
error_code	The partition-level error code, or 0 if there was no error.
error_code	The group-level error code, or 0 if there was no error.

FindCoordinator API (Key: 10):

Requests:

```
FindCoordinator Request (Version: 0) => { key }
  key => STRING
```

Request header version: 1

FIELD	DESCRIPTION
key	The coordinator key.

```
FindCoordinator Request (Version: 1) => { key key_type }
  key => STRING
  key_type => INT8
```

Request header version: 1

FIELD	DESCRIPTION
key	The coordinator key.
key_type	The coordinator key type. (group, transaction, share).

```
FindCoordinator Request (Version: 2) => { key key_type }
  key => STRING
  key_type => INT8
```

Request header version: 1

FIELD	DESCRIPTION
key	The coordinator key.
key_type	The coordinator key type. (group, transaction, share).

```
FindCoordinator Request (Version: 3) => { key key_type }
```

```
  key => COMPACT_STRING
  key_type => INT8
```

Request header version: 2

FIELD	DESCRIPTION
key	The coordinator key.
key_type	The coordinator key type. (group, transaction, share).

```
FindCoordinator Request (Version: 4) => { key_type (coordinator_keys) }
  key_type => INT8
  coordinator_keys => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
key_type	The coordinator key type. (group, transaction, share).
coordinator_keys	The coordinator keys.

```
FindCoordinator Request (Version: 5) => { key_type (coordinator_keys) }
  key_type => INT8
  coordinator_keys => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
key_type	The coordinator key type. (group, transaction, share).
coordinator_keys	The coordinator keys.

```
FindCoordinator Request (Version: 6) => { key_type (coordinator_keys) }
  key_type => INT8
  coordinator_keys => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
key_type	The coordinator key type. (group, transaction, share).
coordinator_keys	The coordinator keys.

Responses:

```
FindCoordinator Response (Version: 0) => { error_code node_id host port }
  error_code => INT16
  node_id => INT32
  host => STRING
  port => INT32
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
node_id	The node id.
host	The host name.
port	The port.

```
FindCoordinator Response (Version: 1) => { throttle_time_ms error_code error_mess
  throttle_time_ms => INT32
  error_code => INT16
  error_message => NULLABLE_STRING
  node_id => INT32
  host => STRING
  port => INT32
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
node_id	The node id.
host	The host name.
port	The port.

```
FindCoordinator Response (Version: 2) => { throttle_time_ms error_code error_mess
  throttle_time_ms => INT32
  error_code => INT16
  error_message => NULLABLE_STRING
  node_id => INT32
  host => STRING
  port => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
node_id	The node id.
host	The host name.
port	The port.

```
FindCoordinator Response (Version: 3) => { throttle_time_ms error_code error_mess
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
```

```
  node_id => INT32
  host => COMPACT_STRING
  port => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
node_id	The node id.
host	The host name.
port	The port.

```
FindCoordinator Response (Version: 4) => { throttle_time_ms (coordinators) }
  throttle_time_ms => INT32
  coordinators => { key node_id host port error_code error_message }
    key => COMPACT_STRING
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
coordinators	Each coordinator result in the response.
key	The coordinator key.

node_id	The node id.
host	The host name.
port	The port.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

```
FindCoordinator Response (Version: 5) => { throttle_time_ms (coordinators) }
  throttle_time_ms => INT32
  coordinators => { key node_id host port error_code error_message }
    key => COMPACT_STRING
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
coordinators	Each coordinator result in the response.
key	The coordinator key.
node_id	The node id.
host	The host name.
port	The port.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

```
FindCoordinator Response (Version: 6) => { throttle_time_ms (coordinators) }
```

```
  throttle_time_ms => INT32
  coordinators => { key node_id host port error_code error_message }
    key => COMPACT_STRING
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
coordinators	Each coordinator result in the response.
key	The coordinator key.
node_id	The node id.
host	The host name.
port	The port.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

JoinGroup API (Key: 11):

Requests:

```
JoinGroup Request (Version: 0) => { group_id session_timeout_ms member_id protoco
  group_id => STRING
  session_timeout_ms => INT32
  member_id => STRING
  protocol_type => STRING
  protocols => { name metadata }
    name => STRING
    metadata => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
member_id	The member id assigned by the group coordinator.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.
metadata	The protocol metadata.

```
JoinGroup Request (Version: 1) => { group_id session_timeout_ms rebalance_timeout
  group_id => STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => STRING
  protocol_type => STRING
  protocols => { name metadata }
    name => STRING
    metadata => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.

member_id	The member id assigned by the group coordinator.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.
metadata	The protocol metadata.

```
JoinGroup Request (Version: 2) => { group_id session_timeout_ms rebalance_timeout
  group_id => STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => STRING
  protocol_type => STRING
  protocols => { name metadata }
    name => STRING
    metadata => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.
member_id	The member id assigned by the group coordinator.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.

metadata	The protocol metadata.

```
JoinGroup Request (Version: 3) => { group_id session_timeout_ms rebalance_timeout
  group_id => STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => STRING
  protocol_type => STRING
  protocols => { name metadata }
    name => STRING
    metadata => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.
member_id	The member id assigned by the group coordinator.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.
metadata	The protocol metadata.

```
JoinGroup Request (Version: 4) => { group_id session_timeout_ms rebalance_timeout
  group_id => STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => STRING
  protocol_type => STRING
  protocols => { name metadata }
```

```
    name => STRING
    metadata => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.
member_id	The member id assigned by the group coordinator.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.
metadata	The protocol metadata.

```
JoinGroup Request (Version: 5) => { group_id session_timeout_ms rebalance_timeout
  group_id => STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => STRING
  group_instance_id => NULLABLE_STRING
  protocol_type => STRING
  protocols => { name metadata }
    name => STRING
    metadata => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group identifier.

session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.
member_id	The member id assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.
metadata	The protocol metadata.

```
JoinGroup Request (Version: 6) => { group_id session_timeout_ms rebalance_timeout
  group_id => COMPACT_STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  protocol_type => COMPACT_STRING
  protocols => { name metadata }
    name => COMPACT_STRING
    metadata => COMPACT_BYTES
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.

member_id	The member id assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.
metadata	The protocol metadata.

```
JoinGroup Request (Version: 7) => { group_id session_timeout_ms rebalance_timeout
  group_id => COMPACT_STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  protocol_type => COMPACT_STRING
  protocols => { name metadata }
    name => COMPACT_STRING
    metadata => COMPACT_BYTES
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.
member_id	The member id assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.

protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.
metadata	The protocol metadata.

```
JoinGroup Request (Version: 8) => { group_id session_timeout_ms rebalance_timeout
  group_id => COMPACT_STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  protocol_type => COMPACT_STRING
  protocols => { name metadata }
    name => COMPACT_STRING
    metadata => COMPACT_BYTES
  reason => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.
member_id	The member id assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.

name	The protocol name.
metadata	The protocol metadata.
reason	The reason why the member (re-)joins the group.

```
JoinGroup Request (Version: 9) => { group_id session_timeout_ms rebalance_timeout
  group_id => COMPACT_STRING
  session_timeout_ms => INT32
  rebalance_timeout_ms => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  protocol_type => COMPACT_STRING
  protocols => { name metadata }
    name => COMPACT_STRING
    metadata => COMPACT_BYTES
  reason => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
session_timeout_ms	The coordinator considers the consumer dead if it receives no heartbeat after this timeout in milliseconds.
rebalance_timeout_ms	The maximum time in milliseconds that the coordinator will wait for each member to rejoin when rebalancing the group.
member_id	The member id assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
protocol_type	The unique name the for class of protocols implemented by the group we want to join.
protocols	The list of protocols that the member supports.
name	The protocol name.

metadata	The protocol metadata.
reason	The reason why the member (re-)joins the group.

Responses:

```
JoinGroup Response (Version: 0) => { error_code generation_id protocol_name leade
  error_code => INT16
  generation_id => INT32
  protocol_name => STRING
  leader => STRING
  member_id => STRING
  members => { member_id metadata }
    member_id => STRING
    metadata => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
metadata	The group member metadata.

```
JoinGroup Response (Version: 1) => { error_code generation_id protocol_name leade
  error_code => INT16
  generation_id => INT32
  protocol_name => STRING
  leader => STRING
  member_id => STRING
  members => { member_id metadata }
```

```
    member_id => STRING
    metadata => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
metadata	The group member metadata.

```
JoinGroup Response (Version: 2) => { throttle_time_ms error_code generation_id pr
  throttle_time_ms => INT32
  error_code => INT16
  generation_id => INT32
  protocol_name => STRING
  leader => STRING
  member_id => STRING
  members => { member_id metadata }
    member_id => STRING
    metadata => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

generation_id	The generation ID of the group.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
metadata	The group member metadata.

```
JoinGroup Response (Version: 3) => { throttle_time_ms error_code generation_id pr
  throttle_time_ms => INT32
  error_code => INT16
  generation_id => INT32
  protocol_name => STRING
  leader => STRING
  member_id => STRING
  members => { member_id metadata }
    member_id => STRING
    metadata => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.

member_id	The group member ID.
metadata	The group member metadata.

```
JoinGroup Response (Version: 4) => { throttle_time_ms error_code generation_id pr
  throttle_time_ms => INT32
  error_code => INT16
  generation_id => INT32
  protocol_name => STRING
  leader => STRING
  member_id => STRING
  members => { member_id metadata }
    member_id => STRING
    metadata => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
metadata	The group member metadata.

```
JoinGroup Response (Version: 5) => { throttle_time_ms error_code generation_id pr
  throttle_time_ms => INT32
  error_code => INT16
  generation_id => INT32
```

```
  protocol_name => STRING
  leader => STRING
  member_id => STRING
  members => { member_id group_instance_id metadata }
    member_id => STRING
    group_instance_id => NULLABLE_STRING
    metadata => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
group_instance_id	The unique identifier of the consumer instance provided by end user.
metadata	The group member metadata.

```
JoinGroup Response (Version: 6) => { throttle_time_ms error_code generation_id pr
  throttle_time_ms => INT32
  error_code => INT16
  generation_id => INT32
  protocol_name => COMPACT_STRING
  leader => COMPACT_STRING
  member_id => COMPACT_STRING
  members => { member_id group_instance_id metadata }
    member_id => COMPACT_STRING
    group_instance_id => COMPACT_NULLABLE_STRING
    metadata => COMPACT_BYTES
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
group_instance_id	The unique identifier of the consumer instance provided by end user.
metadata	The group member metadata.

```
JoinGroup Response (Version: 7) => { throttle_time_ms error_code generation_id pr
  throttle_time_ms => INT32
  error_code => INT16
  generation_id => INT32
  protocol_type => COMPACT_NULLABLE_STRING
  protocol_name => COMPACT_NULLABLE_STRING
```

```
  leader => COMPACT_STRING
  member_id => COMPACT_STRING
  members => { member_id group_instance_id metadata }
    member_id => COMPACT_STRING
    group_instance_id => COMPACT_NULLABLE_STRING
    metadata => COMPACT_BYTES
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_type	The group protocol name.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
group_instance_id	The unique identifier of the consumer instance provided by end user.
metadata	The group member metadata.

```
JoinGroup Response (Version: 8) => { throttle_time_ms error_code generation_id pr
  throttle_time_ms => INT32
  error_code => INT16
  generation_id => INT32
  protocol_type => COMPACT_NULLABLE_STRING
  protocol_name => COMPACT_NULLABLE_STRING
  leader => COMPACT_STRING
  member_id => COMPACT_STRING
  members => { member_id group_instance_id metadata }
    member_id => COMPACT_STRING
```

```
    group_instance_id => COMPACT_NULLABLE_STRING
    metadata => COMPACT_BYTES
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_type	The group protocol name.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
group_instance_id	The unique identifier of the consumer instance provided by end user.
metadata	The group member metadata.

```
JoinGroup Response (Version: 9) => { throttle_time_ms error_code generation_id pr
  throttle_time_ms => INT32
  error_code => INT16
  generation_id => INT32
  protocol_type => COMPACT_NULLABLE_STRING
  protocol_name => COMPACT_NULLABLE_STRING
  leader => COMPACT_STRING
  skip_assignment => BOOLEAN
  member_id => COMPACT_STRING
  members => { member_id group_instance_id metadata }
    member_id => COMPACT_STRING
    group_instance_id => COMPACT_NULLABLE_STRING
    metadata => COMPACT_BYTES
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
generation_id	The generation ID of the group.
protocol_type	The group protocol name.
protocol_name	The group protocol selected by the coordinator.
leader	The leader of the group.
skip_assignment	True if the leader must skip running the assignment.
member_id	The member ID assigned by the group coordinator.
members	The group members.
member_id	The group member ID.
group_instance_id	The unique identifier of the consumer instance provided by end user.
metadata	The group member metadata.

Heartbeat API (Key: 12):

Requests:

```
Heartbeat Request (Version: 0) => { group_id generation_id member_id }
  group_id => STRING
  generation_id => INT32
  member_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group id.

generation_id	The generation of the group.
member_id	The member ID.

```
Heartbeat Request (Version: 1) => { group_id generation_id member_id }
  group_id => STRING
  generation_id => INT32
  member_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group id.
generation_id	The generation of the group.
member_id	The member ID.

```
Heartbeat Request (Version: 2) => { group_id generation_id member_id }
  group_id => STRING
  generation_id => INT32
  member_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group id.
generation_id	The generation of the group.
member_id	The member ID.

```
Heartbeat Request (Version: 3) => { group_id generation_id member_id group_instan
  group_id => STRING
  generation_id => INT32
  member_id => STRING
  group_instance_id => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The group id.
generation_id	The generation of the group.
member_id	The member ID.
group_instance_id	The unique identifier of the consumer instance provided by end user.

```
Heartbeat Request (Version: 4) => { group_id generation_id member_id group_instan
  group_id => COMPACT_STRING
  generation_id => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group id.
generation_id	The generation of the group.
member_id	The member ID.
group_instance_id	The unique identifier of the consumer instance provided by end user.

Responses:

```
Heartbeat Response (Version: 0) => { error_code }
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.

```
Heartbeat Response (Version: 1) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
Heartbeat Response (Version: 2) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
Heartbeat Response (Version: 3) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
Heartbeat Response (Version: 4) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

LeaveGroup API (Key: 13):

Requests:

```
LeaveGroup Request (Version: 0) => { group_id member_id }
  group_id => STRING
  member_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The ID of the group to leave.
member_id	The member ID to remove from the group.

```
LeaveGroup Request (Version: 1) => { group_id member_id }
  group_id => STRING
  member_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The ID of the group to leave.

member_id	The member ID to remove from the group.

```
LeaveGroup Request (Version: 2) => { group_id member_id }
  group_id => STRING
  member_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The ID of the group to leave.
member_id	The member ID to remove from the group.

```
LeaveGroup Request (Version: 3) => { group_id [members] }
  group_id => STRING
  members => { member_id group_instance_id }
    member_id => STRING
    group_instance_id => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The ID of the group to leave.
members	List of leaving member identities.
member_id	The member ID to remove from the group.
group_instance_id	The group instance ID to remove from the group.

```
LeaveGroup Request (Version: 4) => { group_id (members) }
  group_id => COMPACT_STRING
  members => { member_id group_instance_id }
    member_id => COMPACT_STRING
    group_instance_id => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The ID of the group to leave.
members	List of leaving member identities.
member_id	The member ID to remove from the group.
group_instance_id	The group instance ID to remove from the group.

```
LeaveGroup Request (Version: 5) => { group_id (members) }
  group_id => COMPACT_STRING
  members => { member_id group_instance_id reason }
    member_id => COMPACT_STRING
    group_instance_id => COMPACT_NULLABLE_STRING
    reason => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The ID of the group to leave.
members	List of leaving member identities.
member_id	The member ID to remove from the group.
group_instance_id	The group instance ID to remove from the group.
reason	The reason why the member left the group.

Responses:

```
LeaveGroup Response (Version: 0) => { error_code }
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.

```
LeaveGroup Response (Version: 1) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
LeaveGroup Response (Version: 2) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
LeaveGroup Response (Version: 3) => { throttle_time_ms error_code [members] }
  throttle_time_ms => INT32
  error_code => INT16
  members => { member_id group_instance_id error_code }
    member_id => STRING
    group_instance_id => NULLABLE_STRING
    error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
members	List of leaving member responses.
member_id	The member ID to remove from the group.
group_instance_id	The group instance ID to remove from the group.
error_code	The error code, or 0 if there was no error.

```
LeaveGroup Response (Version: 4) => { throttle_time_ms error_code (members) }
  throttle_time_ms => INT32
  error_code => INT16
  members => { member_id group_instance_id error_code }
    member_id => COMPACT_STRING
    group_instance_id => COMPACT_NULLABLE_STRING
    error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
members	List of leaving member responses.
member_id	The member ID to remove from the group.
group_instance_id	The group instance ID to remove from the group.
error_code	The error code, or 0 if there was no error.

```
LeaveGroup Response (Version: 5) => { throttle_time_ms error_code (members) }
  throttle_time_ms => INT32
  error_code => INT16
  members => { member_id group_instance_id error_code }
```

```
    member_id => COMPACT_STRING
    group_instance_id => COMPACT_NULLABLE_STRING
    error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
members	List of leaving member responses.
member_id	The member ID to remove from the group.
group_instance_id	The group instance ID to remove from the group.
error_code	The error code, or 0 if there was no error.

SyncGroup API (Key: 14):

Requests:

```
SyncGroup Request (Version: 0) => { group_id generation_id member_id [assignments
  group_id => STRING
  generation_id => INT32
  member_id => STRING
  assignments => { member_id assignment }
    member_id => STRING
    assignment => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id	The generation of the group.
member_id	The member ID assigned by the group.

assignments	Each assignment.
member_id	The ID of the member to assign.
assignment	The member assignment.

```
SyncGroup Request (Version: 1) => { group_id generation_id member_id [assignments
  group_id => STRING
  generation_id => INT32
  member_id => STRING
  assignments => { member_id assignment }
    member_id => STRING
    assignment => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id	The generation of the group.
member_id	The member ID assigned by the group.
assignments	Each assignment.
member_id	The ID of the member to assign.
assignment	The member assignment.

```
SyncGroup Request (Version: 2) => { group_id generation_id member_id [assignments
  group_id => STRING
  generation_id => INT32
  member_id => STRING
  assignments => { member_id assignment }
    member_id => STRING
    assignment => BYTES
```

Request header version: 1

FIELD	DESCRIPTION

group_id	The unique group identifier.
generation_id	The generation of the group.
member_id	The member ID assigned by the group.
assignments	Each assignment.
member_id	The ID of the member to assign.
assignment	The member assignment.

```
SyncGroup Request (Version: 3) => { group_id generation_id member_id group_instan
  group_id => STRING
  generation_id => INT32
  member_id => STRING
  group_instance_id => NULLABLE_STRING
  assignments => { member_id assignment }
    member_id => STRING
    assignment => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id	The generation of the group.
member_id	The member ID assigned by the group.
group_instance_id	The unique identifier of the consumer instance provided by end user.
assignments	Each assignment.
member_id	The ID of the member to assign.
assignment	The member assignment.

```
SyncGroup Request (Version: 4) => { group_id generation_id member_id group_instan
  group_id => COMPACT_STRING
  generation_id => INT32
```

```
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  assignments => { member_id assignment }
    member_id => COMPACT_STRING
    assignment => COMPACT_BYTES
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id	The generation of the group.
member_id	The member ID assigned by the group.
group_instance_id	The unique identifier of the consumer instance provided by end user.
assignments	Each assignment.
member_id	The ID of the member to assign.
assignment	The member assignment.

```
SyncGroup Request (Version: 5) => { group_id generation_id member_id group_instan
  group_id => COMPACT_STRING
  generation_id => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  protocol_type => COMPACT_NULLABLE_STRING
  protocol_name => COMPACT_NULLABLE_STRING
  assignments => { member_id assignment }
    member_id => COMPACT_STRING
    assignment => COMPACT_BYTES
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The unique group identifier.
generation_id	The generation of the group.

member_id	The member ID assigned by the group.
group_instance_id	The unique identifier of the consumer instance provided by end user.
protocol_type	The group protocol type.
protocol_name	The group protocol name.
assignments	Each assignment.
member_id	The ID of the member to assign.
assignment	The member assignment.

Responses:

```
SyncGroup Response (Version: 0) => { error_code assignment }
  error_code => INT16
  assignment => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
assignment	The member assignment.

```
SyncGroup Response (Version: 1) => { throttle_time_ms error_code assignment }
  throttle_time_ms => INT32
  error_code => INT16
  assignment => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

assignment	The member assignment.

```
SyncGroup Response (Version: 2) => { throttle_time_ms error_code assignment }
  throttle_time_ms => INT32
  error_code => INT16
  assignment => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
assignment	The member assignment.

```
SyncGroup Response (Version: 3) => { throttle_time_ms error_code assignment }
  throttle_time_ms => INT32
  error_code => INT16
  assignment => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
assignment	The member assignment.

```
SyncGroup Response (Version: 4) => { throttle_time_ms error_code assignment }
  throttle_time_ms => INT32
  error_code => INT16
  assignment => COMPACT_BYTES
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
assignment	The member assignment.

```
SyncGroup Response (Version: 5) => { throttle_time_ms error_code protocol_type pr
  throttle_time_ms => INT32
  error_code => INT16
  protocol_type => COMPACT_NULLABLE_STRING
  protocol_name => COMPACT_NULLABLE_STRING
  assignment => COMPACT_BYTES
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
protocol_type	The group protocol type.
protocol_name	The group protocol name.
assignment	The member assignment.

DescribeGroups API (Key: 15):

Requests:

```
DescribeGroups Request (Version: 0) => { [groups] }
  groups => STRING
```

Request header version: 1

FIELD	DESCRIPTION
groups	The names of the groups to describe.

```
DescribeGroups Request (Version: 1) => { [groups] }
  groups => STRING
```

Request header version: 1

FIELD	DESCRIPTION
groups	The names of the groups to describe.

```
DescribeGroups Request (Version: 2) => { [groups] }
  groups => STRING
```

Request header version: 1

FIELD	DESCRIPTION
groups	The names of the groups to describe.

```
DescribeGroups Request (Version: 3) => { [groups] include_authorized_operations }
  groups => STRING
  include_authorized_operations => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
groups	The names of the groups to describe.
include_authorized_operations	Whether to include authorized operations.

```
DescribeGroups Request (Version: 4) => { [groups] include_authorized_operations }
  groups => STRING
  include_authorized_operations => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
groups	The names of the groups to describe.
include_authorized_operations	Whether to include authorized operations.

```
DescribeGroups Request (Version: 5) => { (groups) include_authorized_operations }
  groups => COMPACT_STRING
  include_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
groups	The names of the groups to describe.
include_authorized_operations	Whether to include authorized operations.

```
DescribeGroups Request (Version: 6) => { (groups) include_authorized_operations }
  groups => COMPACT_STRING
  include_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
groups	The names of the groups to describe.
include_authorized_operations	Whether to include authorized operations.

Responses:

```
DescribeGroups Response (Version: 0) => { [groups] }
  groups => { error_code group_id group_state protocol_type protocol_data [member
    error_code => INT16
    group_id => STRING
    group_state => STRING
    protocol_type => STRING
    protocol_data => STRING
```

```
    members => { member_id client_id client_host member_metadata member_assignmen
      member_id => STRING
      client_id => STRING
      client_host => STRING
      member_metadata => BYTES
      member_assignment => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
protocol_type	The group protocol type, or the empty string.
protocol_data	The group protocol data, or the empty string.
members	The group members.
member_id	The member id.
client_id	The client ID used in the member's latest join group request.
client_host	The client host.
member_metadata	The metadata corresponding to the current group protocol in use.
member_assignment	The current assignment provided by the group leader.

```
DescribeGroups Response (Version: 1) => { throttle_time_ms [groups] }
  throttle_time_ms => INT32
  groups => { error_code group_id group_state protocol_type protocol_data [member
    error_code => INT16
    group_id => STRING
    group_state => STRING
    protocol_type => STRING
    protocol_data => STRING
    members => { member_id client_id client_host member_metadata member_assignmen
```

```
      member_id => STRING
      client_id => STRING
      client_host => STRING
      member_metadata => BYTES
      member_assignment => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
protocol_type	The group protocol type, or the empty string.
protocol_data	The group protocol data, or the empty string.
members	The group members.
member_id	The member id.
client_id	The client ID used in the member's latest join group request.
client_host	The client host.
member_metadata	The metadata corresponding to the current group protocol in use.
member_assignment	The current assignment provided by the group leader.

```
DescribeGroups Response (Version: 2) => { throttle_time_ms [groups] }
  throttle_time_ms => INT32
  groups => { error_code group_id group_state protocol_type protocol_data [member
    error_code => INT16
    group_id => STRING
    group_state => STRING
```

```
    protocol_type => STRING
    protocol_data => STRING
    members => { member_id client_id client_host member_metadata member_assignmen
      member_id => STRING
      client_id => STRING
      client_host => STRING
      member_metadata => BYTES
      member_assignment => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
protocol_type	The group protocol type, or the empty string.
protocol_data	The group protocol data, or the empty string.
members	The group members.
member_id	The member id.
client_id	The client ID used in the member's latest join group request.
client_host	The client host.
member_metadata	The metadata corresponding to the current group protocol in use.
member_assignment	The current assignment provided by the group leader.

```
DescribeGroups Response (Version: 3) => { throttle_time_ms [groups] }
  throttle_time_ms => INT32
  groups => { error_code group_id group_state protocol_type protocol_data [member
```

```
    error_code => INT16
    group_id => STRING
    group_state => STRING
    protocol_type => STRING
    protocol_data => STRING
    members => { member_id client_id client_host member_metadata member_assignmen
      member_id => STRING
      client_id => STRING
      client_host => STRING
      member_metadata => BYTES
      member_assignment => BYTES
    authorized_operations => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
protocol_type	The group protocol type, or the empty string.
protocol_data	The group protocol data, or the empty string.
members	The group members.
member_id	The member id.
client_id	The client ID used in the member's latest join group request.
client_host	The client host.
member_metadata	The metadata corresponding to the current group protocol in use.
member_assignment	The current assignment provided by the group leader.

authorized_operations	32-bit bitfield to represent authorized operations for this group.

```
DescribeGroups Response (Version: 4) => { throttle_time_ms [groups] }
  throttle_time_ms => INT32
  groups => { error_code group_id group_state protocol_type protocol_data [member
    error_code => INT16
    group_id => STRING
    group_state => STRING
    protocol_type => STRING
    protocol_data => STRING
    members => { member_id group_instance_id client_id client_host member_metadat
      member_id => STRING
      group_instance_id => NULLABLE_STRING
      client_id => STRING
      client_host => STRING
      member_metadata => BYTES
      member_assignment => BYTES
    authorized_operations => INT32
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
protocol_type	The group protocol type, or the empty string.
protocol_data	The group protocol data, or the empty string.
members	The group members.
member_id	The member id.

group_instance_id	The unique identifier of the consumer instance provided by end user.
client_id	The client ID used in the member's latest join group request.
client_host	The client host.
member_metadata	The metadata corresponding to the current group protocol in use.
member_assignment	The current assignment provided by the group leader.
authorized_operations	32-bit bitfield to represent authorized operations for this group.

```
DescribeGroups Response (Version: 5) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { error_code group_id group_state protocol_type protocol_data (member
    error_code => INT16
    group_id => COMPACT_STRING
    group_state => COMPACT_STRING
    protocol_type => COMPACT_STRING
    protocol_data => COMPACT_STRING
    members => { member_id group_instance_id client_id client_host member_metadat
      member_id => COMPACT_STRING
      group_instance_id => COMPACT_NULLABLE_STRING
      client_id => COMPACT_STRING
      client_host => COMPACT_STRING
      member_metadata => COMPACT_BYTES
      member_assignment => COMPACT_BYTES
    authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.

group_id	The group ID string.
group_state	The group state string, or the empty string.
protocol_type	The group protocol type, or the empty string.
protocol_data	The group protocol data, or the empty string.
members	The group members.
member_id	The member id.
group_instance_id	The unique identifier of the consumer instance provided by end user.
client_id	The client ID used in the member's latest join group request.
client_host	The client host.
member_metadata	The metadata corresponding to the current group protocol in use.
member_assignment	The current assignment provided by the group leader.
authorized_operations	32-bit bitfield to represent authorized operations for this group.

```
DescribeGroups Response (Version: 6) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { error_code error_message group_id group_state protocol_type protoco
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    group_id => COMPACT_STRING
    group_state => COMPACT_STRING
    protocol_type => COMPACT_STRING
    protocol_data => COMPACT_STRING
    members => { member_id group_instance_id client_id client_host member_metadat
      member_id => COMPACT_STRING
      group_instance_id => COMPACT_NULLABLE_STRING
      client_id => COMPACT_STRING
      client_host => COMPACT_STRING
      member_metadata => COMPACT_BYTES
      member_assignment => COMPACT_BYTES
    authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
error_message	The describe error message, or null if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
protocol_type	The group protocol type, or the empty string.
protocol_data	The group protocol data, or the empty string.
members	The group members.
member_id	The member id.
group_instance_id	The unique identifier of the consumer instance provided by end user.
client_id	The client ID used in the member's latest join group request.
client_host	The client host.
member_metadata	The metadata corresponding to the current group protocol in use.
member_assignment	The current assignment provided by the group leader.
authorized_operations	32-bit bitfield to represent authorized operations for this group.

ListGroups API (Key: 16):

Requests:

```
ListGroups Request (Version: 0) => { }
```

Request header version: 1

FIELD	DESCRIPTION

```
ListGroups Request (Version: 1) => { }
```

Request header version: 1

FIELD	DESCRIPTION

```
ListGroups Request (Version: 2) => { }
```

Request header version: 1

FIELD	DESCRIPTION

```
ListGroups Request (Version: 3) => { }
```

Request header version: 2

FIELD	DESCRIPTION

```
ListGroups Request (Version: 4) => { (states_filter) }
  states_filter => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
states_filter	The states of the groups we want to list. If empty, all groups are returned with their state.

```
ListGroups Request (Version: 5) => { (states_filter) (types_filter) }
  states_filter => COMPACT_STRING
  types_filter => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
states_filter	The states of the groups we want to list. If empty, all groups are returned with their state.
types_filter	The types of the groups we want to list. If empty, all groups are returned with their type.

Responses:

```
ListGroups Response (Version: 0) => { error_code [groups] }
  error_code => INT16
  groups => { group_id protocol_type }
    group_id => STRING
    protocol_type => STRING
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
groups	Each group in the response.
group_id	The group ID.
protocol_type	The group protocol type.

```
ListGroups Response (Version: 1) => { throttle_time_ms error_code [groups] }
  throttle_time_ms => INT32
  error_code => INT16
  groups => { group_id protocol_type }
    group_id => STRING
    protocol_type => STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

error_code	The error code, or 0 if there was no error.
groups	Each group in the response.
group_id	The group ID.
protocol_type	The group protocol type.

```
ListGroups Response (Version: 2) => { throttle_time_ms error_code [groups] }
  throttle_time_ms => INT32
  error_code => INT16
  groups => { group_id protocol_type }
    group_id => STRING
    protocol_type => STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
groups	Each group in the response.
group_id	The group ID.
protocol_type	The group protocol type.

```
ListGroups Response (Version: 3) => { throttle_time_ms error_code (groups) }
  throttle_time_ms => INT32
  error_code => INT16
  groups => { group_id protocol_type }
    group_id => COMPACT_STRING
    protocol_type => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
groups	Each group in the response.
group_id	The group ID.
protocol_type	The group protocol type.

```
ListGroups Response (Version: 4) => { throttle_time_ms error_code (groups) }
  throttle_time_ms => INT32
  error_code => INT16
  groups => { group_id protocol_type group_state }
    group_id => COMPACT_STRING
    protocol_type => COMPACT_STRING
    group_state => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
groups	Each group in the response.
group_id	The group ID.
protocol_type	The group protocol type.
group_state	The group state name.

```
ListGroups Response (Version: 5) => { throttle_time_ms error_code (groups) }
  throttle_time_ms => INT32
  error_code => INT16
  groups => { group_id protocol_type group_state group_type }
    group_id => COMPACT_STRING
    protocol_type => COMPACT_STRING
```

```
    group_state => COMPACT_STRING
    group_type => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
groups	Each group in the response.
group_id	The group ID.
protocol_type	The group protocol type.
group_state	The group state name.
group_type	The group type name.

SaslHandshake API (Key: 17):

Requests:

```
SaslHandshake Request (Version: 0) => { mechanism }
  mechanism => STRING
```

Request header version: 1

FIELD	DESCRIPTION
mechanism	The SASL mechanism chosen by the client.

```
SaslHandshake Request (Version: 1) => { mechanism }
  mechanism => STRING
```

Request header version: 1

FIELD	DESCRIPTION
mechanism	The SASL mechanism chosen by the client.

Responses:

```
SaslHandshake Response (Version: 0) => { error_code [mechanisms] }
  error_code => INT16
  mechanisms => STRING
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
mechanisms	The mechanisms enabled in the server.

```
SaslHandshake Response (Version: 1) => { error_code [mechanisms] }
  error_code => INT16
  mechanisms => STRING
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
mechanisms	The mechanisms enabled in the server.

ApiVersions API (Key: 18):

Requests:

```
ApiVersions Request (Version: 0) => { }
```

Request header version: 1

FIELD	DESCRIPTION

```
ApiVersions Request (Version: 1) => { }
```

Request header version: 1

FIELD	DESCRIPTION

```
ApiVersions Request (Version: 2) => { }
```

Request header version: 1

FIELD	DESCRIPTION

```
ApiVersions Request (Version: 3) => { client_software_name client_software_versio
  client_software_name => COMPACT_STRING
  client_software_version => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
client_software_name	The name of the client.
client_software_version	The version of the client.

```
ApiVersions Request (Version: 4) => { client_software_name client_software_versio
  client_software_name => COMPACT_STRING
  client_software_version => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
client_software_name	The name of the client.
client_software_version	The version of the client.

Responses:

```
ApiVersions Response (Version: 0) => { error_code [api_keys] }
  error_code => INT16
  api_keys => { api_key min_version max_version }
    api_key => INT16
    min_version => INT16
    max_version => INT16
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The top-level error code.
api_keys	The APIs supported by the broker.
api_key	The API index.
min_version	The minimum supported version, inclusive.
max_version	The maximum supported version, inclusive.

```
ApiVersions Response (Version: 1) => { error_code [api_keys] throttle_time_ms }
  error_code => INT16
  api_keys => { api_key min_version max_version }
    api_key => INT16
    min_version => INT16
    max_version => INT16
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The top-level error code.
api_keys	The APIs supported by the broker.
api_key	The API index.
min_version	The minimum supported version, inclusive.
max_version	The maximum supported version, inclusive.

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
ApiVersions Response (Version: 2) => { error_code [api_keys] throttle_time_ms }
  error_code => INT16
  api_keys => { api_key min_version max_version }
    api_key => INT16
    min_version => INT16
    max_version => INT16
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The top-level error code.
api_keys	The APIs supported by the broker.
api_key	The API index.
min_version	The minimum supported version, inclusive.
max_version	The maximum supported version, inclusive.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
ApiVersions Response (Version: 3) => { error_code (api_keys) throttle_time_ms sup
  error_code => INT16
  api_keys => { api_key min_version max_version }
    api_key => INT16
    min_version => INT16
    max_version => INT16
  throttle_time_ms => INT32
  supported_features<tag: 0> => { name min_version max_version }
    name => COMPACT_STRING
    min_version => INT16
    max_version => INT16
  finalized_features_epoch<tag: 1> => INT64
  finalized_features<tag: 2> => { name max_version_level min_version_level }
    name => COMPACT_STRING
```

```
    max_version_level => INT16
    min_version_level => INT16
  zk_migration_ready<tag: 3> => BOOLEAN
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The top-level error code.
api_keys	The APIs supported by the broker.
api_key	The API index.
min_version	The minimum supported version, inclusive.
max_version	The maximum supported version, inclusive.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
supported_features<tag: 0>	Features supported by the broker. Note: in v0-v3, features with MinSupportedVersion = 0 are omitted.

FIELD	DESCRIPTION
name	The name of the feature.
min_version	The minimum supported version for the feature.
max_version	The maximum supported version for the feature.

finalized_features_epoch<tag: 1>	The monotonically increasing epoch for the finalized features information. Valid values are >= 0. A value of -1 is special and represents unknown epoch.
finalized_features<tag: 2>	List of cluster-wide finalized features. The information is valid only if FinalizedFeaturesEpoch >= 0.

FIELD	DESCRIPTION

	name	The name of the feature.
	max_version_level	The cluster-wide finalized max version level for the feature.
	min_version_level	The cluster-wide finalized min version level for the feature.
zk_migration_ready<tag: 3>	Set by a KRaft controller if the required configurations for ZK migration are present.	

```
ApiVersions Response (Version: 4) => { error_code (api_keys) throttle_time_ms sup
  error_code => INT16
  api_keys => { api_key min_version max_version }
    api_key => INT16
    min_version => INT16
    max_version => INT16
  throttle_time_ms => INT32
  supported_features<tag: 0> => { name min_version max_version }
    name => COMPACT_STRING
    min_version => INT16
    max_version => INT16
  finalized_features_epoch<tag: 1> => INT64
  finalized_features<tag: 2> => { name max_version_level min_version_level }
    name => COMPACT_STRING
    max_version_level => INT16
    min_version_level => INT16
  zk_migration_ready<tag: 3> => BOOLEAN
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The top-level error code.
api_keys	The APIs supported by the broker.
api_key	The API index.
min_version	The minimum supported version, inclusive.

max_version	The maximum supported version, inclusive.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
supported_features<tag: 0>	Features supported by the broker. Note: in v0-v3, features with MinSupportedVersion = 0 are omitted.

FIELD	DESCRIPTION
name	The name of the feature.
min_version	The minimum supported version for the feature.
max_version	The maximum supported version for the feature.

finalized_features_epoch<tag: 1>	The monotonically increasing epoch for the finalized features information. Valid values are >= 0. A value of -1 is special and represents unknown epoch.
finalized_features<tag: 2>	List of cluster-wide finalized features. The information is valid only if FinalizedFeaturesEpoch >= 0.

FIELD	DESCRIPTION
name	The name of the feature.
max_version_level	The cluster-wide finalized max version level for the feature.
min_version_level	The cluster-wide finalized min version level for the feature.

zk_migration_ready<tag: 3>	Set by a KRaft controller if the required configurations for ZK migration are present.

CreateTopics API (Key: 19):

Requests:

```
CreateTopics Request (Version: 2) => { [topics] timeout_ms validate_only }
  topics => { name num_partitions replication_factor [assignments] [configs] }
    name => STRING
    num_partitions => INT32
    replication_factor => INT16
    assignments => { partition_index [broker_ids] }
      partition_index => INT32
      broker_ids => INT32
    configs => { name value }
      name => STRING
      value => NULLABLE_STRING
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to create.
name	The topic name.
num_partitions	The number of partitions to create in the topic, or -1 if we are either specifying a manual partition assignment or using the default partitions.
replication_factor	The number of replicas to create for each partition in the topic, or -1 if we are either specifying a manual partition assignment or using the default replication factor.
assignments	The manual partition assignment, or the empty array if we are using automatic assignment.
partition_index	The partition index.
broker_ids	The brokers to place the partition on.
configs	The custom topic configurations to set.
name	The configuration name.
value	The configuration value.

timeout_ms	How long to wait in milliseconds before timing out the request.
validate_only	If true, check that the topics can be created as specified, but don't create anything.

```
CreateTopics Request (Version: 3) => { [topics] timeout_ms validate_only }
  topics => { name num_partitions replication_factor [assignments] [configs] }
    name => STRING
    num_partitions => INT32
    replication_factor => INT16
    assignments => { partition_index [broker_ids] }
      partition_index => INT32
      broker_ids => INT32
    configs => { name value }
      name => STRING
      value => NULLABLE_STRING
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to create.
name	The topic name.
num_partitions	The number of partitions to create in the topic, or -1 if we are either specifying a manual partition assignment or using the default partitions.
replication_factor	The number of replicas to create for each partition in the topic, or -1 if we are either specifying a manual partition assignment or using the default replication factor.
assignments	The manual partition assignment, or the empty array if we are using automatic assignment.
partition_index	The partition index.
broker_ids	The brokers to place the partition on.

configs	The custom topic configurations to set.
name	The configuration name.
value	The configuration value.
timeout_ms	How long to wait in milliseconds before timing out the request.
validate_only	If true, check that the topics can be created as specified, but don't create anything.

```
CreateTopics Request (Version: 4) => { [topics] timeout_ms validate_only }
  topics => { name num_partitions replication_factor [assignments] [configs] }
    name => STRING
    num_partitions => INT32
    replication_factor => INT16
    assignments => { partition_index [broker_ids] }
      partition_index => INT32
      broker_ids => INT32
    configs => { name value }
      name => STRING
      value => NULLABLE_STRING
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	The topics to create.
name	The topic name.
num_partitions	The number of partitions to create in the topic, or -1 if we are either specifying a manual partition assignment or using the default partitions.
replication_factor	The number of replicas to create for each partition in the topic, or -1 if we are either specifying a manual partition assignment or using the default replication factor.
assignments	The manual partition assignment, or the empty array if we are using

	automatic assignment.
partition_index	The partition index.
broker_ids	The brokers to place the partition on.
configs	The custom topic configurations to set.
name	The configuration name.
value	The configuration value.
timeout_ms	How long to wait in milliseconds before timing out the request.
validate_only	If true, check that the topics can be created as specified, but don't create anything.

```
CreateTopics Request (Version: 5) => { (topics) timeout_ms validate_only }
  topics => { name num_partitions replication_factor (assignments) (configs) }
    name => COMPACT_STRING
    num_partitions => INT32
    replication_factor => INT16
    assignments => { partition_index (broker_ids) }
      partition_index => INT32
      broker_ids => INT32
    configs => { name value }
      name => COMPACT_STRING
      value => COMPACT_NULLABLE_STRING
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to create.
name	The topic name.
num_partitions	The number of partitions to create in the topic, or -1 if we are either specifying a manual partition assignment or using the default partitions.

replication_factor	The number of replicas to create for each partition in the topic, or -1 if we are either specifying a manual partition assignment or using the default replication factor.
assignments	The manual partition assignment, or the empty array if we are using automatic assignment.
partition_index	The partition index.
broker_ids	The brokers to place the partition on.
configs	The custom topic configurations to set.
name	The configuration name.
value	The configuration value.
timeout_ms	How long to wait in milliseconds before timing out the request.
validate_only	If true, check that the topics can be created as specified, but don't create anything.

```
CreateTopics Request (Version: 6) => { (topics) timeout_ms validate_only }
  topics => { name num_partitions replication_factor (assignments) (configs) }
    name => COMPACT_STRING
    num_partitions => INT32
    replication_factor => INT16
    assignments => { partition_index (broker_ids) }
      partition_index => INT32
      broker_ids => INT32
    configs => { name value }
      name => COMPACT_STRING
      value => COMPACT_NULLABLE_STRING
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to create.

name	The topic name.
num_partitions	The number of partitions to create in the topic, or -1 if we are either specifying a manual partition assignment or using the default partitions.
replication_factor	The number of replicas to create for each partition in the topic, or -1 if we are either specifying a manual partition assignment or using the default replication factor.
assignments	The manual partition assignment, or the empty array if we are using automatic assignment.
partition_index	The partition index.
broker_ids	The brokers to place the partition on.
configs	The custom topic configurations to set.
name	The configuration name.
value	The configuration value.
timeout_ms	How long to wait in milliseconds before timing out the request.
validate_only	If true, check that the topics can be created as specified, but don't create anything.

```
CreateTopics Request (Version: 7) => { (topics) timeout_ms validate_only }
  topics => { name num_partitions replication_factor (assignments) (configs) }
    name => COMPACT_STRING
    num_partitions => INT32
    replication_factor => INT16
    assignments => { partition_index (broker_ids) }
      partition_index => INT32
      broker_ids => INT32
    configs => { name value }
      name => COMPACT_STRING
      value => COMPACT_NULLABLE_STRING
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to create.
name	The topic name.
num_partitions	The number of partitions to create in the topic, or -1 if we are either specifying a manual partition assignment or using the default partitions.
replication_factor	The number of replicas to create for each partition in the topic, or -1 if we are either specifying a manual partition assignment or using the default replication factor.
assignments	The manual partition assignment, or the empty array if we are using automatic assignment.
partition_index	The partition index.
broker_ids	The brokers to place the partition on.
configs	The custom topic configurations to set.
name	The configuration name.
value	The configuration value.
timeout_ms	How long to wait in milliseconds before timing out the request.
validate_only	If true, check that the topics can be created as specified, but don't create anything.

Responses:

```
CreateTopics Response (Version: 2) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name error_code error_message }
    name => STRING
    error_code => INT16
    error_message => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Results for each topic we tried to create.
name	The topic name.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

```
CreateTopics Response (Version: 3) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name error_code error_message }
    name => STRING
    error_code => INT16
    error_message => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Results for each topic we tried to create.
name	The topic name.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

```
CreateTopics Response (Version: 4) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name error_code error_message }
    name => STRING
    error_code => INT16
    error_message => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Results for each topic we tried to create.
name	The topic name.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

```
CreateTopics Response (Version: 5) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name error_code error_message num_partitions replication_factor ?(c
    name => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    num_partitions => INT32
    replication_factor => INT16
    configs => { name value read_only config_source is_sensitive }
      name => COMPACT_STRING
      value => COMPACT_NULLABLE_STRING
      read_only => BOOLEAN
      config_source => INT8
      is_sensitive => BOOLEAN
    topic_config_error_code<tag: 0> => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Results for each topic we tried to create.
name	The topic name.

error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
num_partitions	Number of partitions of the topic.
replication_factor	Replication factor of the topic.
configs	Configuration of the topic.
name	The configuration name.
value	The configuration value.
read_only	True if the configuration is read-only.
config_source	The configuration source.
is_sensitive	True if this configuration is sensitive.
topic_config_error_code<tag: 0>	Optional topic config error returned if configs are not returned in the response.

```
CreateTopics Response (Version: 6) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name error_code error_message num_partitions replication_factor ?(c
    name => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    num_partitions => INT32
    replication_factor => INT16
    configs => { name value read_only config_source is_sensitive }
      name => COMPACT_STRING
      value => COMPACT_NULLABLE_STRING
      read_only => BOOLEAN
      config_source => INT8
      is_sensitive => BOOLEAN
    topic_config_error_code<tag: 0> => INT16
```

Response header version: 1

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Results for each topic we tried to create.
name	The topic name.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
num_partitions	Number of partitions of the topic.
replication_factor	Replication factor of the topic.
configs	Configuration of the topic.
name	The configuration name.
value	The configuration value.
read_only	True if the configuration is read-only.
config_source	The configuration source.
is_sensitive	True if this configuration is sensitive.
topic_config_error_code<tag: 0>	Optional topic config error returned if configs are not returned in the response.

```
CreateTopics Response (Version: 7) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name topic_id error_code error_message num_partitions replication_f
    name => COMPACT_STRING
    topic_id => UUID
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    num_partitions => INT32
    replication_factor => INT16
    configs => { name value read_only config_source is_sensitive }
      name => COMPACT_STRING
      value => COMPACT_NULLABLE_STRING
      read_only => BOOLEAN
```

```
      config_source => INT8
      is_sensitive => BOOLEAN
    topic_config_error_code<tag: 0> => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Results for each topic we tried to create.
name	The topic name.
topic_id	The unique topic ID.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
num_partitions	Number of partitions of the topic.
replication_factor	Replication factor of the topic.
configs	Configuration of the topic.
name	The configuration name.
value	The configuration value.
read_only	True if the configuration is read-only.
config_source	The configuration source.
is_sensitive	True if this configuration is sensitive.
topic_config_error_code<tag: 0>	Optional topic config error returned if configs are not returned in the response.

DeleteTopics API (Key: 20):

Requests:

```
DeleteTopics Request (Version: 1) => { [topic_names] timeout_ms }
  topic_names => STRING
  timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
topic_names	The names of the topics to delete.
timeout_ms	The length of time in milliseconds to wait for the deletions to complete.

```
DeleteTopics Request (Version: 2) => { [topic_names] timeout_ms }
  topic_names => STRING
  timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
topic_names	The names of the topics to delete.
timeout_ms	The length of time in milliseconds to wait for the deletions to complete.

```
DeleteTopics Request (Version: 3) => { [topic_names] timeout_ms }
  topic_names => STRING
  timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
topic_names	The names of the topics to delete.
timeout_ms	The length of time in milliseconds to wait for the deletions to complete.

```
DeleteTopics Request (Version: 4) => { (topic_names) timeout_ms }
  topic_names => COMPACT_STRING
  timeout_ms => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topic_names	The names of the topics to delete.
timeout_ms	The length of time in milliseconds to wait for the deletions to complete.

```
DeleteTopics Request (Version: 5) => { (topic_names) timeout_ms }
  topic_names => COMPACT_STRING
  timeout_ms => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topic_names	The names of the topics to delete.
timeout_ms	The length of time in milliseconds to wait for the deletions to complete.

```
DeleteTopics Request (Version: 6) => { (topics) timeout_ms }
  topics => { name topic_id }
    name => COMPACT_NULLABLE_STRING
    topic_id => UUID
  timeout_ms => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	The name or topic ID of the topic.
name	The topic name.
topic_id	The unique topic ID.

timeout_ms	The length of time in milliseconds to wait for the deletions to complete.

Responses:

```
DeleteTopics Response (Version: 1) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { name error_code }
    name => STRING
    error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The results for each topic we tried to delete.
name	The topic name.
error_code	The deletion error, or 0 if the deletion succeeded.

```
DeleteTopics Response (Version: 2) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { name error_code }
    name => STRING
    error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The results for each topic we tried to delete.
name	The topic name.
error_code	The deletion error, or 0 if the deletion succeeded.

```
DeleteTopics Response (Version: 3) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { name error_code }
    name => STRING
    error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The results for each topic we tried to delete.
name	The topic name.
error_code	The deletion error, or 0 if the deletion succeeded.

```
DeleteTopics Response (Version: 4) => { throttle_time_ms (responses) }
  throttle_time_ms => INT32
  responses => { name error_code }
    name => COMPACT_STRING
    error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The results for each topic we tried to delete.
name	The topic name.
error_code	The deletion error, or 0 if the deletion succeeded.

```
DeleteTopics Response (Version: 5) => { throttle_time_ms (responses) }
  throttle_time_ms => INT32
  responses => { name error_code error_message }
```

```
    name => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The results for each topic we tried to delete.
name	The topic name.
error_code	The deletion error, or 0 if the deletion succeeded.
error_message	The error message, or null if there was no error.

```
DeleteTopics Response (Version: 6) => { throttle_time_ms (responses) }
  throttle_time_ms => INT32
  responses => { name topic_id error_code error_message }
    name => COMPACT_NULLABLE_STRING
    topic_id => UUID
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The results for each topic we tried to delete.
name	The topic name.
topic_id	The unique topic ID.
error_code	The deletion error, or 0 if the deletion succeeded.

error_message	The error message, or null if there was no error.

DeleteRecords API (Key: 21):

Requests:

```
DeleteRecords Request (Version: 0) => { [topics] timeout_ms }
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index offset }
      partition_index => INT32
      offset => INT64
  timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
topics	Each topic that we want to delete records from.
name	The topic name.
partitions	Each partition that we want to delete records from.
partition_index	The partition index.
offset	The deletion offset. -1 means that records should be truncated to the high watermark.
timeout_ms	How long to wait for the deletion to complete, in milliseconds.

```
DeleteRecords Request (Version: 1) => { [topics] timeout_ms }
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index offset }
      partition_index => INT32
      offset => INT64
  timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
topics	Each topic that we want to delete records from.
name	The topic name.
partitions	Each partition that we want to delete records from.
partition_index	The partition index.
offset	The deletion offset. -1 means that records should be truncated to the high watermark.
timeout_ms	How long to wait for the deletion to complete, in milliseconds.

```
DeleteRecords Request (Version: 2) => { (topics) timeout_ms }
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index offset }
      partition_index => INT32
      offset => INT64
  timeout_ms => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	Each topic that we want to delete records from.
name	The topic name.
partitions	Each partition that we want to delete records from.
partition_index	The partition index.
offset	The deletion offset. -1 means that records should be truncated to the high watermark.
timeout_ms	How long to wait for the deletion to complete, in milliseconds.

Responses:

```
DeleteRecords Response (Version: 0) => { throttle_time_ms [topics] }
```

```
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index low_watermark error_code }
      partition_index => INT32
      low_watermark => INT64
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic that we wanted to delete records from.
name	The topic name.
partitions	Each partition that we wanted to delete records from.
partition_index	The partition index.
low_watermark	The partition low water mark.
error_code	The deletion error code, or 0 if the deletion succeeded.

```
DeleteRecords Response (Version: 1) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index low_watermark error_code }
      partition_index => INT32
      low_watermark => INT64
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

topics	Each topic that we wanted to delete records from.
name	The topic name.
partitions	Each partition that we wanted to delete records from.
partition_index	The partition index.
low_watermark	The partition low water mark.
error_code	The deletion error code, or 0 if the deletion succeeded.

```
DeleteRecords Response (Version: 2) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index low_watermark error_code }
      partition_index => INT32
      low_watermark => INT64
      error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic that we wanted to delete records from.
name	The topic name.
partitions	Each partition that we wanted to delete records from.
partition_index	The partition index.
low_watermark	The partition low water mark.
error_code	The deletion error code, or 0 if the deletion succeeded.

InitProducerId API (Key: 22):

Requests:

```
InitProducerId Request (Version: 0) => { transactional_id transaction_timeout_ms
  transactional_id => NULLABLE_STRING
  transaction_timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional id, or null if the producer is not transactional.
transaction_timeout_ms	The time in ms to wait before aborting idle transactions sent by this producer. This is only relevant if a TransactionalId has been defined.

```
InitProducerId Request (Version: 1) => { transactional_id transaction_timeout_ms
  transactional_id => NULLABLE_STRING
  transaction_timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional id, or null if the producer is not transactional.
transaction_timeout_ms	The time in ms to wait before aborting idle transactions sent by this producer. This is only relevant if a TransactionalId has been defined.

```
InitProducerId Request (Version: 2) => { transactional_id transaction_timeout_ms
  transactional_id => COMPACT_NULLABLE_STRING
  transaction_timeout_ms => INT32
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional id, or null if the producer is not transactional.

transaction_timeout_ms	The time in ms to wait before aborting idle transactions sent by this producer. This is only relevant if a TransactionalId has been defined.

```
InitProducerId Request (Version: 3) => { transactional_id transaction_timeout_ms
  transactional_id => COMPACT_NULLABLE_STRING
  transaction_timeout_ms => INT32
  producer_id => INT64
  producer_epoch => INT16
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional id, or null if the producer is not transactional.
transaction_timeout_ms	The time in ms to wait before aborting idle transactions sent by this producer. This is only relevant if a TransactionalId has been defined.
producer_id	The producer id. This is used to disambiguate requests if a transactional id is reused following its expiration.
producer_epoch	The producer's current epoch. This will be checked against the producer epoch on the broker, and the request will return an error if they do not match.

```
InitProducerId Request (Version: 4) => { transactional_id transaction_timeout_ms
  transactional_id => COMPACT_NULLABLE_STRING
  transaction_timeout_ms => INT32
  producer_id => INT64
  producer_epoch => INT16
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional id, or null if the producer is not transactional.

transaction_timeout_ms	The time in ms to wait before aborting idle transactions sent by this producer. This is only relevant if a TransactionalId has been defined.
producer_id	The producer id. This is used to disambiguate requests if a transactional id is reused following its expiration.
producer_epoch	The producer's current epoch. This will be checked against the producer epoch on the broker, and the request will return an error if they do not match.

```
InitProducerId Request (Version: 5) => { transactional_id transaction_timeout_ms
  transactional_id => COMPACT_NULLABLE_STRING
  transaction_timeout_ms => INT32
  producer_id => INT64
  producer_epoch => INT16
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional id, or null if the producer is not transactional.
transaction_timeout_ms	The time in ms to wait before aborting idle transactions sent by this producer. This is only relevant if a TransactionalId has been defined.
producer_id	The producer id. This is used to disambiguate requests if a transactional id is reused following its expiration.
producer_epoch	The producer's current epoch. This will be checked against the producer epoch on the broker, and the request will return an error if they do not match.

```
InitProducerId Request (Version: 6) => { transactional_id transaction_timeout_ms
  transactional_id => COMPACT_NULLABLE_STRING
  transaction_timeout_ms => INT32
  producer_id => INT64
  producer_epoch => INT16
  enable2_pc => BOOLEAN
  keep_prepared_txn => BOOLEAN
```

This version of the request is unstable.

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional id, or null if the producer is not transactional.
transaction_timeout_ms	The time in ms to wait before aborting idle transactions sent by this producer. This is only relevant if a TransactionalId has been defined.
producer_id	The producer id. This is used to disambiguate requests if a transactional id is reused following its expiration.
producer_epoch	The producer's current epoch. This will be checked against the producer epoch on the broker, and the request will return an error if they do not match.
enable2_pc	True if the client wants to enable two-phase commit (2PC) protocol for transactions.
keep_prepared_txn	True if the client wants to keep the currently ongoing transaction instead of aborting it.

Responses:

```
InitProducerId Response (Version: 0) => { throttle_time_ms error_code producer_id
  throttle_time_ms => INT32
  error_code => INT16
  producer_id => INT64
  producer_epoch => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
producer_id	The current producer id.
producer_epoch	The current epoch associated with the producer id.

```
InitProducerId Response (Version: 1) => { throttle_time_ms error_code producer_id
  throttle_time_ms => INT32
  error_code => INT16
  producer_id => INT64
  producer_epoch => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
producer_id	The current producer id.
producer_epoch	The current epoch associated with the producer id.

```
InitProducerId Response (Version: 2) => { throttle_time_ms error_code producer_id
  throttle_time_ms => INT32
  error_code => INT16
  producer_id => INT64
  producer_epoch => INT16
```

Response header version: 1

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
producer_id	The current producer id.
producer_epoch	The current epoch associated with the producer id.

```
InitProducerId Response (Version: 3) => { throttle_time_ms error_code producer_id
  throttle_time_ms => INT32
  error_code => INT16
  producer_id => INT64
  producer_epoch => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
producer_id	The current producer id.
producer_epoch	The current epoch associated with the producer id.

```
InitProducerId Response (Version: 4) => { throttle_time_ms error_code producer_id
  throttle_time_ms => INT32
  error_code => INT16
  producer_id => INT64
  producer_epoch => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

error_code	The error code, or 0 if there was no error.
producer_id	The current producer id.
producer_epoch	The current epoch associated with the producer id.

```
InitProducerId Response (Version: 5) => { throttle_time_ms error_code producer_id
  throttle_time_ms => INT32
  error_code => INT16
  producer_id => INT64
  producer_epoch => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
producer_id	The current producer id.
producer_epoch	The current epoch associated with the producer id.

```
InitProducerId Response (Version: 6) => { throttle_time_ms error_code producer_id
  throttle_time_ms => INT32
  error_code => INT16
  producer_id => INT64
  producer_epoch => INT16
  ongoing_txn_producer_id => INT64
  ongoing_txn_producer_epoch => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

error_code	The error code, or 0 if there was no error.
producer_id	The current producer id.
producer_epoch	The current epoch associated with the producer id.
ongoing_txn_producer_id	The producer id for ongoing transaction when KeepPreparedTxn is used, -1 if there is no transaction ongoing.
ongoing_txn_producer_epoch	The epoch associated with the producer id for ongoing transaction when KeepPreparedTxn is used, -1 if there is no transaction ongoing.

OffsetForLeaderEpoch API (Key: 23):

Requests:

```
OffsetForLeaderEpoch Request (Version: 2) => { [topics] }
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition current_leader_epoch leader_epoch }
      partition => INT32
      current_leader_epoch => INT32
      leader_epoch => INT32
```

Request header version: 1

FIELD	DESCRIPTION
topics	Each topic to get offsets for.
topic	The topic name.
partitions	Each partition to get offsets for.
partition	The partition index.
current_leader_epoch	An epoch used to fence consumers/replicas with old metadata. If the epoch provided by the client is larger than the current epoch known to the broker, then the UNKNOWN_LEADER_EPOCH error code will be returned. If the provided epoch is smaller, then the

	FENCED_LEADER_EPOCH error code will be returned.
leader_epoch	The epoch to look up an offset for.

```
OffsetForLeaderEpoch Request (Version: 3) => { replica_id [topics] }
  replica_id => INT32
  topics => { topic [partitions] }
    topic => STRING
    partitions => { partition current_leader_epoch leader_epoch }
      partition => INT32
      current_leader_epoch => INT32
      leader_epoch => INT32
```

Request header version: 1

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
topics	Each topic to get offsets for.
topic	The topic name.
partitions	Each partition to get offsets for.
partition	The partition index.
current_leader_epoch	An epoch used to fence consumers/replicas with old metadata. If the epoch provided by the client is larger than the current epoch known to the broker, then the UNKNOWN_LEADER_EPOCH error code will be returned. If the provided epoch is smaller, then the FENCED_LEADER_EPOCH error code will be returned.
leader_epoch	The epoch to look up an offset for.

```
OffsetForLeaderEpoch Request (Version: 4) => { replica_id (topics) }
  replica_id => INT32
  topics => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => { partition current_leader_epoch leader_epoch }
      partition => INT32
```

```
      current_leader_epoch => INT32
      leader_epoch => INT32
```

Request header version: 2

FIELD	DESCRIPTION
replica_id	The broker ID of the follower, of -1 if this request is from a consumer.
topics	Each topic to get offsets for.
topic	The topic name.
partitions	Each partition to get offsets for.
partition	The partition index.
current_leader_epoch	An epoch used to fence consumers/replicas with old metadata. If the epoch provided by the client is larger than the current epoch known to the broker, then the UNKNOWN_LEADER_EPOCH error code will be returned. If the provided epoch is smaller, then the FENCED_LEADER_EPOCH error code will be returned.
leader_epoch	The epoch to look up an offset for.

Responses:

```
OffsetForLeaderEpoch Response (Version: 2) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { topic [partitions] }
    topic => STRING
    partitions => { error_code partition leader_epoch end_offset }
      error_code => INT16
      partition => INT32
      leader_epoch => INT32
      end_offset => INT64
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic we fetched offsets for.
topic	The topic name.
partitions	Each partition in the topic we fetched offsets for.
error_code	The error code 0, or if there was no error.
partition	The partition index.
leader_epoch	The leader epoch of the partition.
end_offset	The end offset of the epoch.

```
OffsetForLeaderEpoch Response (Version: 3) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { topic [partitions] }
    topic => STRING
    partitions => { error_code partition leader_epoch end_offset }
      error_code => INT16
      partition => INT32
      leader_epoch => INT32
      end_offset => INT64
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic we fetched offsets for.
topic	The topic name.
partitions	Each partition in the topic we fetched offsets for.
error_code	The error code 0, or if there was no error.
partition	The partition index.

leader_epoch	The leader epoch of the partition.
end_offset	The end offset of the epoch.

```
OffsetForLeaderEpoch Response (Version: 4) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => { error_code partition leader_epoch end_offset }
      error_code => INT16
      partition => INT32
      leader_epoch => INT32
      end_offset => INT64
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic we fetched offsets for.
topic	The topic name.
partitions	Each partition in the topic we fetched offsets for.
error_code	The error code 0, or if there was no error.
partition	The partition index.
leader_epoch	The leader epoch of the partition.
end_offset	The end offset of the epoch.

AddPartitionsToTxn API (Key: 24):

Requests:

```
AddPartitionsToTxn Request (Version: 0) => { v3_and_below_transactional_id v3_and
  v3_and_below_transactional_id => STRING
  v3_and_below_producer_id => INT64
```

```
  v3_and_below_producer_epoch => INT16
  v3_and_below_topics => { name [partitions] }
    name => STRING
    partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
v3_and_below_transactional_id	The transactional id corresponding to the transaction.
v3_and_below_producer_id	Current producer id in use by the transactional id.
v3_and_below_producer_epoch	Current epoch associated with the producer id.
v3_and_below_topics	The partitions to add to the transaction.
name	The name of the topic.
partitions	The partition indexes to add to the transaction.

```
AddPartitionsToTxn Request (Version: 1) => { v3_and_below_transactional_id v3_and
  v3_and_below_transactional_id => STRING
  v3_and_below_producer_id => INT64
  v3_and_below_producer_epoch => INT16
  v3_and_below_topics => { name [partitions] }
    name => STRING
    partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
v3_and_below_transactional_id	The transactional id corresponding to the transaction.
v3_and_below_producer_id	Current producer id in use by the transactional id.
v3_and_below_producer_epoch	Current epoch associated with the producer id.
v3_and_below_topics	The partitions to add to the transaction.
name	The name of the topic.

partitions	The partition indexes to add to the transaction.

```
AddPartitionsToTxn Request (Version: 2) => { v3_and_below_transactional_id v3_and
  v3_and_below_transactional_id => STRING
  v3_and_below_producer_id => INT64
  v3_and_below_producer_epoch => INT16
  v3_and_below_topics => { name [partitions] }
    name => STRING
    partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
v3_and_below_transactional_id	The transactional id corresponding to the transaction.
v3_and_below_producer_id	Current producer id in use by the transactional id.
v3_and_below_producer_epoch	Current epoch associated with the producer id.
v3_and_below_topics	The partitions to add to the transaction.
name	The name of the topic.
partitions	The partition indexes to add to the transaction.

```
AddPartitionsToTxn Request (Version: 3) => { v3_and_below_transactional_id v3_and
  v3_and_below_transactional_id => COMPACT_STRING
  v3_and_below_producer_id => INT64
  v3_and_below_producer_epoch => INT16
  v3_and_below_topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
v3_and_below_transactional_id	The transactional id corresponding to the transaction.
v3_and_below_producer_id	Current producer id in use by the transactional id.

v3_and_below_producer_epoch	Current epoch associated with the producer id.
v3_and_below_topics	The partitions to add to the transaction.
name	The name of the topic.
partitions	The partition indexes to add to the transaction.

```
AddPartitionsToTxn Request (Version: 4) => { (transactions) }
  transactions => { transactional_id producer_id producer_epoch verify_only (topi
    transactional_id => COMPACT_STRING
    producer_id => INT64
    producer_epoch => INT16
    verify_only => BOOLEAN
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
transactions	List of transactions to add partitions to.
transactional_id	The transactional id corresponding to the transaction.
producer_id	Current producer id in use by the transactional id.
producer_epoch	Current epoch associated with the producer id.
verify_only	Boolean to signify if we want to check if the partition is in the transaction rather than add it.
topics	The partitions to add to the transaction.
name	The name of the topic.
partitions	The partition indexes to add to the transaction.

```
AddPartitionsToTxn Request (Version: 5) => { (transactions) }
  transactions => { transactional_id producer_id producer_epoch verify_only (topi
    transactional_id => COMPACT_STRING
```

```
    producer_id => INT64
    producer_epoch => INT16
    verify_only => BOOLEAN
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
transactions	List of transactions to add partitions to.
transactional_id	The transactional id corresponding to the transaction.
producer_id	Current producer id in use by the transactional id.
producer_epoch	Current epoch associated with the producer id.
verify_only	Boolean to signify if we want to check if the partition is in the transaction rather than add it.
topics	The partitions to add to the transaction.
name	The name of the topic.
partitions	The partition indexes to add to the transaction.

Responses:

```
AddPartitionsToTxn Response (Version: 0) => { throttle_time_ms [results_by_topic_
  throttle_time_ms => INT32
  results_by_topic_v3_and_below => { name [results_by_partition] }
    name => STRING
    results_by_partition => { partition_index partition_error_code }
      partition_index => INT32
      partition_error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results_by_topic_v3_and_below	The results for each topic.
name	The topic name.
results_by_partition	The results for each partition.
partition_index	The partition indexes.
partition_error_code	The response error code.

```
AddPartitionsToTxn Response (Version: 1) => { throttle_time_ms [results_by_topic_
  throttle_time_ms => INT32
  results_by_topic_v3_and_below => { name [results_by_partition] }
    name => STRING
    results_by_partition => { partition_index partition_error_code }
      partition_index => INT32
      partition_error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results_by_topic_v3_and_below	The results for each topic.
name	The topic name.
results_by_partition	The results for each partition.
partition_index	The partition indexes.
partition_error_code	The response error code.

```
AddPartitionsToTxn Response (Version: 2) => { throttle_time_ms [results_by_topic_
  throttle_time_ms => INT32
```

```
  results_by_topic_v3_and_below => { name [results_by_partition] }
    name => STRING
    results_by_partition => { partition_index partition_error_code }
      partition_index => INT32
      partition_error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results_by_topic_v3_and_below	The results for each topic.
name	The topic name.
results_by_partition	The results for each partition.
partition_index	The partition indexes.
partition_error_code	The response error code.

```
AddPartitionsToTxn Response (Version: 3) => { throttle_time_ms (results_by_topic_
  throttle_time_ms => INT32
  results_by_topic_v3_and_below => { name (results_by_partition) }
    name => COMPACT_STRING
    results_by_partition => { partition_index partition_error_code }
      partition_index => INT32
      partition_error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results_by_topic_v3_and_below	The results for each topic.

name	The topic name.
results_by_partition	The results for each partition.
partition_index	The partition indexes.
partition_error_code	The response error code.

```
AddPartitionsToTxn Response (Version: 4) => { throttle_time_ms error_code (result
  throttle_time_ms => INT32
  error_code => INT16
  results_by_transaction => { transactional_id (topic_results) }
    transactional_id => COMPACT_STRING
    topic_results => { name (results_by_partition) }
      name => COMPACT_STRING
      results_by_partition => { partition_index partition_error_code }
        partition_index => INT32
        partition_error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The response top level error code.
results_by_transaction	Results categorized by transactional ID.
transactional_id	The transactional id corresponding to the transaction.
topic_results	The results for each topic.
name	The topic name.
results_by_partition	The results for each partition.
partition_index	The partition indexes.
partition_error_code	The response error code.

```
AddPartitionsToTxn Response (Version: 5) => { throttle_time_ms error_code (result
  throttle_time_ms => INT32
  error_code => INT16
  results_by_transaction => { transactional_id (topic_results) }
    transactional_id => COMPACT_STRING
    topic_results => { name (results_by_partition) }
      name => COMPACT_STRING
      results_by_partition => { partition_index partition_error_code }
        partition_index => INT32
        partition_error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The response top level error code.
results_by_transaction	Results categorized by transactional ID.
transactional_id	The transactional id corresponding to the transaction.
topic_results	The results for each topic.
name	The topic name.
results_by_partition	The results for each partition.
partition_index	The partition indexes.
partition_error_code	The response error code.

AddOffsetsToTxn API (Key: 25):

Requests:

```
AddOffsetsToTxn Request (Version: 0) => { transactional_id producer_id producer_e
  transactional_id => STRING
  producer_id => INT64
  producer_epoch => INT16
```

```
  group_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional id corresponding to the transaction.
producer_id	Current producer id in use by the transactional id.
producer_epoch	Current epoch associated with the producer id.
group_id	The unique group identifier.

```
AddOffsetsToTxn Request (Version: 1) => { transactional_id producer_id producer_e
  transactional_id => STRING
  producer_id => INT64
  producer_epoch => INT16
  group_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional id corresponding to the transaction.
producer_id	Current producer id in use by the transactional id.
producer_epoch	Current epoch associated with the producer id.
group_id	The unique group identifier.

```
AddOffsetsToTxn Request (Version: 2) => { transactional_id producer_id producer_e
  transactional_id => STRING
  producer_id => INT64
  producer_epoch => INT16
  group_id => STRING
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The transactional id corresponding to the transaction.
producer_id	Current producer id in use by the transactional id.
producer_epoch	Current epoch associated with the producer id.
group_id	The unique group identifier.

```
AddOffsetsToTxn Request (Version: 3) => { transactional_id producer_id producer_e
  transactional_id => COMPACT_STRING
  producer_id => INT64
  producer_epoch => INT16
  group_id => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional id corresponding to the transaction.
producer_id	Current producer id in use by the transactional id.
producer_epoch	Current epoch associated with the producer id.
group_id	The unique group identifier.

```
AddOffsetsToTxn Request (Version: 4) => { transactional_id producer_id producer_e
  transactional_id => COMPACT_STRING
  producer_id => INT64
  producer_epoch => INT16
  group_id => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The transactional id corresponding to the transaction.
producer_id	Current producer id in use by the transactional id.

producer_epoch	Current epoch associated with the producer id.
group_id	The unique group identifier.

Responses:

```
AddOffsetsToTxn Response (Version: 0) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The response error code, or 0 if there was no error.

```
AddOffsetsToTxn Response (Version: 1) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The response error code, or 0 if there was no error.

```
AddOffsetsToTxn Response (Version: 2) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The response error code, or 0 if there was no error.

```
AddOffsetsToTxn Response (Version: 3) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The response error code, or 0 if there was no error.

```
AddOffsetsToTxn Response (Version: 4) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The response error code, or 0 if there was no error.

EndTxn API (Key: 26):

Requests:

```
EndTxn Request (Version: 0) => { transactional_id producer_id producer_epoch comm
  transactional_id => STRING
  producer_id => INT64
  producer_epoch => INT16
  committed => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The ID of the transaction to end.
producer_id	The producer ID.
producer_epoch	The current epoch associated with the producer.
committed	True if the transaction was committed, false if it was aborted.

```
EndTxn Request (Version: 1) => { transactional_id producer_id producer_epoch comm
  transactional_id => STRING
  producer_id => INT64
  producer_epoch => INT16
  committed => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The ID of the transaction to end.
producer_id	The producer ID.
producer_epoch	The current epoch associated with the producer.
committed	True if the transaction was committed, false if it was aborted.

```
EndTxn Request (Version: 2) => { transactional_id producer_id producer_epoch comm
  transactional_id => STRING
  producer_id => INT64
  producer_epoch => INT16
  committed => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The ID of the transaction to end.
producer_id	The producer ID.
producer_epoch	The current epoch associated with the producer.
committed	True if the transaction was committed, false if it was aborted.

```
EndTxn Request (Version: 3) => { transactional_id producer_id producer_epoch comm
  transactional_id => COMPACT_STRING
  producer_id => INT64
  producer_epoch => INT16
  committed => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The ID of the transaction to end.
producer_id	The producer ID.
producer_epoch	The current epoch associated with the producer.
committed	True if the transaction was committed, false if it was aborted.

```
EndTxn Request (Version: 4) => { transactional_id producer_id producer_epoch comm
  transactional_id => COMPACT_STRING
  producer_id => INT64
  producer_epoch => INT16
  committed => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The ID of the transaction to end.
producer_id	The producer ID.
producer_epoch	The current epoch associated with the producer.
committed	True if the transaction was committed, false if it was aborted.

```
EndTxn Request (Version: 5) => { transactional_id producer_id producer_epoch comm
  transactional_id => COMPACT_STRING
  producer_id => INT64
  producer_epoch => INT16
  committed => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The ID of the transaction to end.
producer_id	The producer ID.
producer_epoch	The current epoch associated with the producer.
committed	True if the transaction was committed, false if it was aborted.

Responses:

```
EndTxn Response (Version: 0) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
EndTxn Response (Version: 1) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
EndTxn Response (Version: 2) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
EndTxn Response (Version: 3) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
EndTxn Response (Version: 4) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

```
EndTxn Response (Version: 5) => { throttle_time_ms error_code producer_id produce
  throttle_time_ms => INT32
  error_code => INT16
  producer_id => INT64
  producer_epoch => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
producer_id	The producer ID.
producer_epoch	The current epoch associated with the producer.

WriteTxnMarkers API (Key: 27):

Requests:

```
WriteTxnMarkers Request (Version: 1) => { (markers) }
  markers => { producer_id producer_epoch transaction_result (topics) coordinator
    producer_id => INT64
    producer_epoch => INT16
    transaction_result => BOOLEAN
    topics => { name (partition_indexes) }
      name => COMPACT_STRING
      partition_indexes => INT32
    coordinator_epoch => INT32
```

Request header version: 2

FIELD	DESCRIPTION
markers	The transaction markers to be written.
producer_id	The current producer ID.
producer_epoch	The current epoch associated with the producer ID.
transaction_result	The result of the transaction to write to the partitions (false = ABORT, true = COMMIT).
topics	Each topic that we want to write transaction marker(s) for.
name	The topic name.
partition_indexes	The indexes of the partitions to write transaction markers for.
coordinator_epoch	Epoch associated with the transaction state partition hosted by this transaction coordinator.

```
WriteTxnMarkers Request (Version: 2) => { (markers) }
  markers => { producer_id producer_epoch transaction_result (topics) coordinator
    producer_id => INT64
    producer_epoch => INT16
    transaction_result => BOOLEAN
    topics => { name (partition_indexes) }
```

```
      name => COMPACT_STRING
      partition_indexes => INT32
    coordinator_epoch => INT32
    transaction_version => INT8
```

Request header version: 2

FIELD	DESCRIPTION
markers	The transaction markers to be written.
producer_id	The current producer ID.
producer_epoch	The current epoch associated with the producer ID.
transaction_result	The result of the transaction to write to the partitions (false = ABORT, true = COMMIT).
topics	Each topic that we want to write transaction marker(s) for.
name	The topic name.
partition_indexes	The indexes of the partitions to write transaction markers for.
coordinator_epoch	Epoch associated with the transaction state partition hosted by this transaction coordinator.
transaction_version	Transaction version of the marker. Ex: 0/1 = legacy (TV0/TV1), 2 = TV2 etc.

Responses:

```
WriteTxnMarkers Response (Version: 1) => { (markers) }
  markers => { producer_id (topics) }
    producer_id => INT64
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => { partition_index error_code }
        partition_index => INT32
        error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
markers	The results for writing makers.
producer_id	The current producer ID in use by the transactional ID.
topics	The results by topic.
name	The topic name.
partitions	The results by partition.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
WriteTxnMarkers Response (Version: 2) => { (markers) }
  markers => { producer_id (topics) }
    producer_id => INT64
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => { partition_index error_code }
        partition_index => INT32
        error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
markers	The results for writing makers.
producer_id	The current producer ID in use by the transactional ID.
topics	The results by topic.
name	The topic name.
partitions	The results by partition.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

TxnOffsetCommit API (Key: 28):

Requests:

```
TxnOffsetCommit Request (Version: 0) => { transactional_id group_id producer_id p
  transactional_id => STRING
  group_id => STRING
  producer_id => INT64
  producer_epoch => INT16
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_metadata }
      partition_index => INT32
      committed_offset => INT64
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The ID of the transaction.
group_id	The ID of the group.
producer_id	The current producer ID in use by the transactional ID.
producer_epoch	The current epoch associated with the producer ID.
topics	Each topic that we want to commit offsets for.
name	The topic name.
partitions	The partitions inside the topic that we want to commit offsets for.
partition_index	The index of the partition within the topic.
committed_offset	The message offset to be committed.
committed_metadata	Any associated metadata the client wants to keep.

```
TxnOffsetCommit Request (Version: 1) => { transactional_id group_id producer_id p
  transactional_id => STRING
  group_id => STRING
```

```
  producer_id => INT64
  producer_epoch => INT16
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_metadata }
      partition_index => INT32
      committed_offset => INT64
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The ID of the transaction.
group_id	The ID of the group.
producer_id	The current producer ID in use by the transactional ID.
producer_epoch	The current epoch associated with the producer ID.
topics	Each topic that we want to commit offsets for.
name	The topic name.
partitions	The partitions inside the topic that we want to commit offsets for.
partition_index	The index of the partition within the topic.
committed_offset	The message offset to be committed.
committed_metadata	Any associated metadata the client wants to keep.

```
TxnOffsetCommit Request (Version: 2) => { transactional_id group_id producer_id p
  transactional_id => STRING
  group_id => STRING
  producer_id => INT64
  producer_epoch => INT16
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
```

```
      committed_metadata => NULLABLE_STRING
```

Request header version: 1

FIELD	DESCRIPTION
transactional_id	The ID of the transaction.
group_id	The ID of the group.
producer_id	The current producer ID in use by the transactional ID.
producer_epoch	The current epoch associated with the producer ID.
topics	Each topic that we want to commit offsets for.
name	The topic name.
partitions	The partitions inside the topic that we want to commit offsets for.
partition_index	The index of the partition within the topic.
committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of the last consumed record.
committed_metadata	Any associated metadata the client wants to keep.

```
TxnOffsetCommit Request (Version: 3) => { transactional_id group_id producer_id p
  transactional_id => COMPACT_STRING
  group_id => COMPACT_STRING
  producer_id => INT64
  producer_epoch => INT16
  generation_id => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      committed_metadata => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The ID of the transaction.
group_id	The ID of the group.
producer_id	The current producer ID in use by the transactional ID.
producer_epoch	The current epoch associated with the producer ID.
generation_id	The generation of the consumer.
member_id	The member ID assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
topics	Each topic that we want to commit offsets for.
name	The topic name.
partitions	The partitions inside the topic that we want to commit offsets for.
partition_index	The index of the partition within the topic.

committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of the last consumed record.
committed_metadata	Any associated metadata the client wants to keep.

```
TxnOffsetCommit Request (Version: 4) => { transactional_id group_id producer_id p
  transactional_id => COMPACT_STRING
  group_id => COMPACT_STRING
  producer_id => INT64
  producer_epoch => INT16
  generation_id => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      committed_metadata => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The ID of the transaction.
group_id	The ID of the group.
producer_id	The current producer ID in use by the transactional ID.
producer_epoch	The current epoch associated with the producer ID.
generation_id	The generation of the consumer.
member_id	The member ID assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
topics	Each topic that we want to commit offsets for.

name	The topic name.
partitions	The partitions inside the topic that we want to commit offsets for.
partition_index	The index of the partition within the topic.
committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of the last consumed record.
committed_metadata	Any associated metadata the client wants to keep.

```
TxnOffsetCommit Request (Version: 5) => { transactional_id group_id producer_id p
  transactional_id => COMPACT_STRING
  group_id => COMPACT_STRING
  producer_id => INT64
  producer_epoch => INT16
  generation_id => INT32
  member_id => COMPACT_STRING
  group_instance_id => COMPACT_NULLABLE_STRING
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index committed_offset committed_leader_epoch commi
      partition_index => INT32
      committed_offset => INT64
      committed_leader_epoch => INT32
      committed_metadata => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION
transactional_id	The ID of the transaction.
group_id	The ID of the group.
producer_id	The current producer ID in use by the transactional ID.
producer_epoch	The current epoch associated with the producer ID.
generation_id	The generation of the consumer.

member_id	The member ID assigned by the group coordinator.
group_instance_id	The unique identifier of the consumer instance provided by end user.
topics	Each topic that we want to commit offsets for.
name	The topic name.
partitions	The partitions inside the topic that we want to commit offsets for.
partition_index	The index of the partition within the topic.
committed_offset	The message offset to be committed.
committed_leader_epoch	The leader epoch of the last consumed record.
committed_metadata	Any associated metadata the client wants to keep.

Responses:

```
TxnOffsetCommit Response (Version: 0) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.

error_code	The error code, or 0 if there was no error.

```
TxnOffsetCommit Response (Version: 1) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
TxnOffsetCommit Response (Version: 2) => { throttle_time_ms [topics] }
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
TxnOffsetCommit Response (Version: 3) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
TxnOffsetCommit Response (Version: 4) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
```

```
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
TxnOffsetCommit Response (Version: 5) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.

partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

DescribeAcls API (Key: 29):

Requests:

```
DescribeAcls Request (Version: 1) => { resource_type_filter resource_name_filter
  resource_type_filter => INT8
  resource_name_filter => NULLABLE_STRING
  pattern_type_filter => INT8
  principal_filter => NULLABLE_STRING
  host_filter => NULLABLE_STRING
  operation => INT8
  permission_type => INT8
```

Request header version: 1

FIELD	DESCRIPTION
resource_type_filter	The resource type.
resource_name_filter	The resource name, or null to match any resource name.
pattern_type_filter	The resource pattern to match.
principal_filter	The principal to match, or null to match any principal.
host_filter	The host to match, or null to match any host.
operation	The operation to match.
permission_type	The permission type to match.

```
DescribeAcls Request (Version: 2) => { resource_type_filter resource_name_filter
  resource_type_filter => INT8
  resource_name_filter => COMPACT_NULLABLE_STRING
  pattern_type_filter => INT8
  principal_filter => COMPACT_NULLABLE_STRING
  host_filter => COMPACT_NULLABLE_STRING
  operation => INT8
```

```
  permission_type => INT8
```

Request header version: 2

FIELD	DESCRIPTION
resource_type_filter	The resource type.
resource_name_filter	The resource name, or null to match any resource name.
pattern_type_filter	The resource pattern to match.
principal_filter	The principal to match, or null to match any principal.
host_filter	The host to match, or null to match any host.
operation	The operation to match.
permission_type	The permission type to match.

```
DescribeAcls Request (Version: 3) => { resource_type_filter resource_name_filter
  resource_type_filter => INT8
  resource_name_filter => COMPACT_NULLABLE_STRING
  pattern_type_filter => INT8
  principal_filter => COMPACT_NULLABLE_STRING
  host_filter => COMPACT_NULLABLE_STRING
  operation => INT8
  permission_type => INT8
```

Request header version: 2

FIELD	DESCRIPTION
resource_type_filter	The resource type.
resource_name_filter	The resource name, or null to match any resource name.
pattern_type_filter	The resource pattern to match.
principal_filter	The principal to match, or null to match any principal.
host_filter	The host to match, or null to match any host.

operation	The operation to match.
permission_type	The permission type to match.

Responses:

```
DescribeAcls Response (Version: 1) => { throttle_time_ms error_code error_message
  throttle_time_ms => INT32
  error_code => INT16
  error_message => NULLABLE_STRING
  resources => { resource_type resource_name pattern_type [acls] }
    resource_type => INT8
    resource_name => STRING
    pattern_type => INT8
    acls => { principal host operation permission_type }
      principal => STRING
      host => STRING
      operation => INT8
      permission_type => INT8
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
resources	Each Resource that is referenced in an ACL.
resource_type	The resource type.
resource_name	The resource name.
pattern_type	The resource pattern type.
acls	The ACLs.
principal	The ACL principal.
host	The ACL host.

operation	The ACL operation.
permission_type	The ACL permission type.

```
DescribeAcls Response (Version: 2) => { throttle_time_ms error_code error_message
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  resources => { resource_type resource_name pattern_type (acls) }
    resource_type => INT8
    resource_name => COMPACT_STRING
    pattern_type => INT8
    acls => { principal host operation permission_type }
      principal => COMPACT_STRING
      host => COMPACT_STRING
      operation => INT8
      permission_type => INT8
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
resources	Each Resource that is referenced in an ACL.
resource_type	The resource type.
resource_name	The resource name.
pattern_type	The resource pattern type.
acls	The ACLs.
principal	The ACL principal.
host	The ACL host.

operation	The ACL operation.
permission_type	The ACL permission type.

```
DescribeAcls Response (Version: 3) => { throttle_time_ms error_code error_message
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  resources => { resource_type resource_name pattern_type (acls) }
    resource_type => INT8
    resource_name => COMPACT_STRING
    pattern_type => INT8
    acls => { principal host operation permission_type }
      principal => COMPACT_STRING
      host => COMPACT_STRING
      operation => INT8
      permission_type => INT8
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
resources	Each Resource that is referenced in an ACL.
resource_type	The resource type.
resource_name	The resource name.
pattern_type	The resource pattern type.
acls	The ACLs.
principal	The ACL principal.
host	The ACL host.

operation	The ACL operation.
permission_type	The ACL permission type.

CreateAcls API (Key: 30):

Requests:

```
CreateAcls Request (Version: 1) => { [creations] }
  creations => { resource_type resource_name resource_pattern_type principal host
    resource_type => INT8
    resource_name => STRING
    resource_pattern_type => INT8
    principal => STRING
    host => STRING
    operation => INT8
    permission_type => INT8
```

Request header version: 1

FIELD	DESCRIPTION
creations	The ACLs that we want to create.
resource_type	The type of the resource.
resource_name	The resource name for the ACL.
resource_pattern_type	The pattern type for the ACL.
principal	The principal for the ACL.
host	The host for the ACL.
operation	The operation type for the ACL (read, write, etc.).
permission_type	The permission type for the ACL (allow, deny, etc.).

```
CreateAcls Request (Version: 2) => { (creations) }
  creations => { resource_type resource_name resource_pattern_type principal host
    resource_type => INT8
    resource_name => COMPACT_STRING
    resource_pattern_type => INT8
```

```
    principal => COMPACT_STRING
    host => COMPACT_STRING
    operation => INT8
    permission_type => INT8
```

Request header version: 2

FIELD	DESCRIPTION
creations	The ACLs that we want to create.
resource_type	The type of the resource.
resource_name	The resource name for the ACL.
resource_pattern_type	The pattern type for the ACL.
principal	The principal for the ACL.
host	The host for the ACL.
operation	The operation type for the ACL (read, write, etc.).
permission_type	The permission type for the ACL (allow, deny, etc.).

```
CreateAcls Request (Version: 3) => { (creations) }
  creations => { resource_type resource_name resource_pattern_type principal host
    resource_type => INT8
    resource_name => COMPACT_STRING
    resource_pattern_type => INT8
    principal => COMPACT_STRING
    host => COMPACT_STRING
    operation => INT8
    permission_type => INT8
```

Request header version: 2

FIELD	DESCRIPTION
creations	The ACLs that we want to create.
resource_type	The type of the resource.

resource_name	The resource name for the ACL.
resource_pattern_type	The pattern type for the ACL.
principal	The principal for the ACL.
host	The host for the ACL.
operation	The operation type for the ACL (read, write, etc.).
permission_type	The permission type for the ACL (allow, deny, etc.).

Responses:

```
CreateAcls Response (Version: 1) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { error_code error_message }
    error_code => INT16
    error_message => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for each ACL creation.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

```
CreateAcls Response (Version: 2) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { error_code error_message }
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for each ACL creation.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

```
CreateAcls Response (Version: 3) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { error_code error_message }
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for each ACL creation.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

DeleteAcls API (Key: 31):

Requests:

```
DeleteAcls Request (Version: 1) => { [filters] }
  filters => { resource_type_filter resource_name_filter pattern_type_filter prin
    resource_type_filter => INT8
    resource_name_filter => NULLABLE_STRING
    pattern_type_filter => INT8
    principal_filter => NULLABLE_STRING
    host_filter => NULLABLE_STRING
    operation => INT8
    permission_type => INT8
```

Request header version: 1

FIELD	DESCRIPTION
filters	The filters to use when deleting ACLs.
resource_type_filter	The resource type.
resource_name_filter	The resource name, or null to match any resource name.
pattern_type_filter	The pattern type.
principal_filter	The principal filter, or null to accept all principals.
host_filter	The host filter, or null to accept all hosts.
operation	The ACL operation.
permission_type	The permission type.

```
DeleteAcls Request (Version: 2) => { (filters) }
  filters => { resource_type_filter resource_name_filter pattern_type_filter prin
    resource_type_filter => INT8
    resource_name_filter => COMPACT_NULLABLE_STRING
    pattern_type_filter => INT8
    principal_filter => COMPACT_NULLABLE_STRING
    host_filter => COMPACT_NULLABLE_STRING
    operation => INT8
    permission_type => INT8
```

Request header version: 2

FIELD	DESCRIPTION
filters	The filters to use when deleting ACLs.
resource_type_filter	The resource type.
resource_name_filter	The resource name, or null to match any resource name.
pattern_type_filter	The pattern type.
principal_filter	The principal filter, or null to accept all principals.

host_filter	The host filter, or null to accept all hosts.
operation	The ACL operation.
permission_type	The permission type.

```
DeleteAcls Request (Version: 3) => { (filters) }
  filters => { resource_type_filter resource_name_filter pattern_type_filter prin
    resource_type_filter => INT8
    resource_name_filter => COMPACT_NULLABLE_STRING
    pattern_type_filter => INT8
    principal_filter => COMPACT_NULLABLE_STRING
    host_filter => COMPACT_NULLABLE_STRING
    operation => INT8
    permission_type => INT8
```

Request header version: 2

FIELD	DESCRIPTION
filters	The filters to use when deleting ACLs.
resource_type_filter	The resource type.
resource_name_filter	The resource name, or null to match any resource name.
pattern_type_filter	The pattern type.
principal_filter	The principal filter, or null to accept all principals.
host_filter	The host filter, or null to accept all hosts.
operation	The ACL operation.
permission_type	The permission type.

Responses:

```
DeleteAcls Response (Version: 1) => { throttle_time_ms [filter_results] }
  throttle_time_ms => INT32
  filter_results => { error_code error_message [matching_acls] }
    error_code => INT16
    error_message => NULLABLE_STRING
```

```
    matching_acls => { error_code error_message resource_type resource_name patte
      error_code => INT16
      error_message => NULLABLE_STRING
      resource_type => INT8
      resource_name => STRING
      pattern_type => INT8
      principal => STRING
      host => STRING
      operation => INT8
      permission_type => INT8
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
filter_results	The results for each filter.
error_code	The error code, or 0 if the filter succeeded.
error_message	The error message, or null if the filter succeeded.
matching_acls	The ACLs which matched this filter.
error_code	The deletion error code, or 0 if the deletion succeeded.
error_message	The deletion error message, or null if the deletion succeeded.
resource_type	The ACL resource type.
resource_name	The ACL resource name.
pattern_type	The ACL resource pattern type.
principal	The ACL principal.
host	The ACL host.
operation	The ACL operation.
permission_type	The ACL permission type.

```
DeleteAcls Response (Version: 2) => { throttle_time_ms (filter_results) }
  throttle_time_ms => INT32
  filter_results => { error_code error_message (matching_acls) }
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    matching_acls => { error_code error_message resource_type resource_name patte
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      resource_type => INT8
      resource_name => COMPACT_STRING
      pattern_type => INT8
      principal => COMPACT_STRING
      host => COMPACT_STRING
      operation => INT8
      permission_type => INT8
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
filter_results	The results for each filter.
error_code	The error code, or 0 if the filter succeeded.
error_message	The error message, or null if the filter succeeded.
matching_acls	The ACLs which matched this filter.
error_code	The deletion error code, or 0 if the deletion succeeded.
error_message	The deletion error message, or null if the deletion succeeded.
resource_type	The ACL resource type.
resource_name	The ACL resource name.
pattern_type	The ACL resource pattern type.
principal	The ACL principal.
host	The ACL host.

operation	The ACL operation.
permission_type	The ACL permission type.

```
DeleteAcls Response (Version: 3) => { throttle_time_ms (filter_results) }
  throttle_time_ms => INT32
  filter_results => { error_code error_message (matching_acls) }
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    matching_acls => { error_code error_message resource_type resource_name patte
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      resource_type => INT8
      resource_name => COMPACT_STRING
      pattern_type => INT8
      principal => COMPACT_STRING
      host => COMPACT_STRING
      operation => INT8
      permission_type => INT8
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
filter_results	The results for each filter.
error_code	The error code, or 0 if the filter succeeded.
error_message	The error message, or null if the filter succeeded.
matching_acls	The ACLs which matched this filter.
error_code	The deletion error code, or 0 if the deletion succeeded.
error_message	The deletion error message, or null if the deletion succeeded.
resource_type	The ACL resource type.
resource_name	The ACL resource name.

pattern_type	The ACL resource pattern type.
principal	The ACL principal.
host	The ACL host.
operation	The ACL operation.
permission_type	The ACL permission type.

DescribeConfigs API (Key: 32):

Requests:

```
DescribeConfigs Request (Version: 1) => { [resources] include_synonyms }
  resources => { resource_type resource_name ?[configuration_keys] }
    resource_type => INT8
    resource_name => STRING
    configuration_keys => STRING
  include_synonyms => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
resources	The resources whose configurations we want to describe.
resource_type	The resource type.
resource_name	The resource name.
configuration_keys	The configuration keys to list, or null to list all configuration keys.
include_synonyms	True if we should include all synonyms.

```
DescribeConfigs Request (Version: 2) => { [resources] include_synonyms }
  resources => { resource_type resource_name ?[configuration_keys] }
    resource_type => INT8
    resource_name => STRING
    configuration_keys => STRING
  include_synonyms => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
resources	The resources whose configurations we want to describe.
resource_type	The resource type.
resource_name	The resource name.
configuration_keys	The configuration keys to list, or null to list all configuration keys.
include_synonyms	True if we should include all synonyms.

```
DescribeConfigs Request (Version: 3) => { [resources] include_synonyms include_do
  resources => { resource_type resource_name ?[configuration_keys] }
    resource_type => INT8
    resource_name => STRING
    configuration_keys => STRING
  include_synonyms => BOOLEAN
  include_documentation => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
resources	The resources whose configurations we want to describe.
resource_type	The resource type.
resource_name	The resource name.
configuration_keys	The configuration keys to list, or null to list all configuration keys.
include_synonyms	True if we should include all synonyms.
include_documentation	True if we should include configuration documentation.

```
DescribeConfigs Request (Version: 4) => { (resources) include_synonyms include_do
  resources => { resource_type resource_name ?(configuration_keys) }
    resource_type => INT8
    resource_name => COMPACT_STRING
```

```
    configuration_keys => COMPACT_STRING
  include_synonyms => BOOLEAN
  include_documentation => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
resources	The resources whose configurations we want to describe.
resource_type	The resource type.
resource_name	The resource name.
configuration_keys	The configuration keys to list, or null to list all configuration keys.
include_synonyms	True if we should include all synonyms.
include_documentation	True if we should include configuration documentation.

Responses:

```
DescribeConfigs Response (Version: 1) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { error_code error_message resource_type resource_name [configs] }
    error_code => INT16
    error_message => NULLABLE_STRING
    resource_type => INT8
    resource_name => STRING
    configs => { name value read_only config_source is_sensitive [synonyms] }
      name => STRING
      value => NULLABLE_STRING
      read_only => BOOLEAN
      config_source => INT8
      is_sensitive => BOOLEAN
      synonyms => { name value source }
        name => STRING
        value => NULLABLE_STRING
        source => INT8
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for each resource.
error_code	The error code, or 0 if we were able to successfully describe the configurations.
error_message	The error message, or null if we were able to successfully describe the configurations.
resource_type	The resource type.
resource_name	The resource name.
configs	Each listed configuration.
name	The configuration name.
value	The configuration value.
read_only	True if the configuration is read-only.
config_source	The configuration source.
is_sensitive	True if this configuration is sensitive.
synonyms	The synonyms for this configuration key.
name	The synonym name.
value	The synonym value.
source	The synonym source.

```
DescribeConfigs Response (Version: 2) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { error_code error_message resource_type resource_name [configs] }
    error_code => INT16
    error_message => NULLABLE_STRING
    resource_type => INT8
    resource_name => STRING
    configs => { name value read_only config_source is_sensitive [synonyms] }
      name => STRING
```

```
      value => NULLABLE_STRING
      read_only => BOOLEAN
      config_source => INT8
      is_sensitive => BOOLEAN
      synonyms => { name value source }
        name => STRING
        value => NULLABLE_STRING
        source => INT8
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for each resource.
error_code	The error code, or 0 if we were able to successfully describe the configurations.
error_message	The error message, or null if we were able to successfully describe the configurations.
resource_type	The resource type.
resource_name	The resource name.
configs	Each listed configuration.
name	The configuration name.
value	The configuration value.
read_only	True if the configuration is read-only.
config_source	The configuration source.
is_sensitive	True if this configuration is sensitive.
synonyms	The synonyms for this configuration key.
name	The synonym name.

value	The synonym value.
source	The synonym source.

```
DescribeConfigs Response (Version: 3) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { error_code error_message resource_type resource_name [configs] }
    error_code => INT16
    error_message => NULLABLE_STRING
    resource_type => INT8
    resource_name => STRING
    configs => { name value read_only config_source is_sensitive [synonyms] confi
      name => STRING
      value => NULLABLE_STRING
      read_only => BOOLEAN
      config_source => INT8
      is_sensitive => BOOLEAN
      synonyms => { name value source }
        name => STRING
        value => NULLABLE_STRING
        source => INT8
      config_type => INT8
      documentation => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for each resource.
error_code	The error code, or 0 if we were able to successfully describe the configurations.
error_message	The error message, or null if we were able to successfully describe the configurations.
resource_type	The resource type.
resource_name	The resource name.

configs	Each listed configuration.
name	The configuration name.
value	The configuration value.
read_only	True if the configuration is read-only.
config_source	The configuration source.
is_sensitive	True if this configuration is sensitive.
synonyms	The synonyms for this configuration key.
name	The synonym name.
value	The synonym value.
source	The synonym source.
config_type	The configuration data type. Type can be one of the following values - BOOLEAN, STRING, INT, SHORT, LONG, DOUBLE, LIST, CLASS, PASSWORD.
documentation	The configuration documentation.

```
DescribeConfigs Response (Version: 4) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { error_code error_message resource_type resource_name (configs) }
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    resource_type => INT8
    resource_name => COMPACT_STRING
    configs => { name value read_only config_source is_sensitive (synonyms) confi
      name => COMPACT_STRING
      value => COMPACT_NULLABLE_STRING
      read_only => BOOLEAN
      config_source => INT8
      is_sensitive => BOOLEAN
      synonyms => { name value source }
        name => COMPACT_STRING
        value => COMPACT_NULLABLE_STRING
        source => INT8
      config_type => INT8
```

```
      documentation => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for each resource.
error_code	The error code, or 0 if we were able to successfully describe the configurations.
error_message	The error message, or null if we were able to successfully describe the configurations.
resource_type	The resource type.
resource_name	The resource name.
configs	Each listed configuration.
name	The configuration name.
value	The configuration value.
read_only	True if the configuration is read-only.
config_source	The configuration source.
is_sensitive	True if this configuration is sensitive.
synonyms	The synonyms for this configuration key.
name	The synonym name.
value	The synonym value.
source	The synonym source.
config_type	The configuration data type. Type can be one of the following values - BOOLEAN, STRING, INT, SHORT, LONG, DOUBLE, LIST, CLASS, PASSWORD.

documentation	The configuration documentation.

AlterConfigs API (Key: 33):

Requests:

```
AlterConfigs Request (Version: 0) => { [resources] validate_only }
  resources => { resource_type resource_name [configs] }
    resource_type => INT8
    resource_name => STRING
    configs => { name value }
      name => STRING
      value => NULLABLE_STRING
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
resources	The updates for each resource.
resource_type	The resource type.
resource_name	The resource name.
configs	The configurations.
name	The configuration key name.
value	The value to set for the configuration key.
validate_only	True if we should validate the request, but not change the configurations.

```
AlterConfigs Request (Version: 1) => { [resources] validate_only }
  resources => { resource_type resource_name [configs] }
    resource_type => INT8
    resource_name => STRING
    configs => { name value }
      name => STRING
      value => NULLABLE_STRING
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
resources	The updates for each resource.
resource_type	The resource type.
resource_name	The resource name.
configs	The configurations.
name	The configuration key name.
value	The value to set for the configuration key.
validate_only	True if we should validate the request, but not change the configurations.

```
AlterConfigs Request (Version: 2) => { (resources) validate_only }
  resources => { resource_type resource_name (configs) }
    resource_type => INT8
    resource_name => COMPACT_STRING
    configs => { name value }
      name => COMPACT_STRING
      value => COMPACT_NULLABLE_STRING
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
resources	The updates for each resource.
resource_type	The resource type.
resource_name	The resource name.
configs	The configurations.
name	The configuration key name.
value	The value to set for the configuration key.

validate_only	True if we should validate the request, but not change the configurations.

Responses:

```
AlterConfigs Response (Version: 0) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { error_code error_message resource_type resource_name }
    error_code => INT16
    error_message => NULLABLE_STRING
    resource_type => INT8
    resource_name => STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The responses for each resource.
error_code	The resource error code.
error_message	The resource error message, or null if there was no error.
resource_type	The resource type.
resource_name	The resource name.

```
AlterConfigs Response (Version: 1) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { error_code error_message resource_type resource_name }
    error_code => INT16
    error_message => NULLABLE_STRING
    resource_type => INT8
    resource_name => STRING
```

Response header version: 0

FIELD	DESCRIPTION

throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The responses for each resource.
error_code	The resource error code.
error_message	The resource error message, or null if there was no error.
resource_type	The resource type.
resource_name	The resource name.

```
AlterConfigs Response (Version: 2) => { throttle_time_ms (responses) }
  throttle_time_ms => INT32
  responses => { error_code error_message resource_type resource_name }
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    resource_type => INT8
    resource_name => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The responses for each resource.
error_code	The resource error code.
error_message	The resource error message, or null if there was no error.
resource_type	The resource type.
resource_name	The resource name.

AlterReplicaLogDirs API (Key: 34):

Requests:

```
AlterReplicaLogDirs Request (Version: 1) => { [dirs] }
  dirs => { path [topics] }
    path => STRING
    topics => { name [partitions] }
      name => STRING
      partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
dirs	The alterations to make for each directory.
path	The absolute directory path.
topics	The topics to add to the directory.
name	The topic name.
partitions	The partition indexes.

```
AlterReplicaLogDirs Request (Version: 2) => { (dirs) }
  dirs => { path (topics) }
    path => COMPACT_STRING
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
dirs	The alterations to make for each directory.
path	The absolute directory path.
topics	The topics to add to the directory.
name	The topic name.
partitions	The partition indexes.

Responses:

```
AlterReplicaLogDirs Response (Version: 1) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { topic_name [partitions] }
    topic_name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for each topic.
topic_name	The name of the topic.
partitions	The results for each partition.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

```
AlterReplicaLogDirs Response (Version: 2) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { topic_name (partitions) }
    topic_name => COMPACT_STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

results	The results for each topic.
topic_name	The name of the topic.
partitions	The results for each partition.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

DescribeLogDirs API (Key: 35):

Requests:

```
DescribeLogDirs Request (Version: 1) => { ?[topics] }
  topics => { topic [partitions] }
    topic => STRING
    partitions => INT32
```

Request header version: 1

FIELD	DESCRIPTION
topics	Each topic that we want to describe log directories for, or null for all topics.
topic	The topic name.
partitions	The partition indexes.

```
DescribeLogDirs Request (Version: 2) => { ?(topics) }
  topics => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	Each topic that we want to describe log directories for, or null for all topics.
topic	The topic name.

partitions	The partition indexes.

```
DescribeLogDirs Request (Version: 3) => { ?(topics) }
  topics => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	Each topic that we want to describe log directories for, or null for all topics.
topic	The topic name.
partitions	The partition indexes.

```
DescribeLogDirs Request (Version: 4) => { ?(topics) }
  topics => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	Each topic that we want to describe log directories for, or null for all topics.
topic	The topic name.
partitions	The partition indexes.

```
DescribeLogDirs Request (Version: 5) => { ?(topics) }
  topics => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	Each topic that we want to describe log directories for, or null for all topics.
topic	The topic name.
partitions	The partition indexes.

Responses:

```
DescribeLogDirs Response (Version: 1) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { error_code log_dir [topics] }
    error_code => INT16
    log_dir => STRING
    topics => { name [partitions] }
      name => STRING
      partitions => { partition_index partition_size offset_lag is_future_key }
        partition_index => INT32
        partition_size => INT64
        offset_lag => INT64
        is_future_key => BOOLEAN
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The log directories.
error_code	The error code, or 0 if there was no error.
log_dir	The absolute log directory path.
topics	The topics.
name	The topic name.
partitions	The partitions.
partition_index	The partition index.

partition_size	The size of the log segments in this partition in bytes.
offset_lag	The lag of the log's LEO w.r.t. partition's HW (if it is the current log for the partition) or current replica's LEO (if it is the future log for the partition).
is_future_key	True if this log is created by AlterReplicaLogDirsRequest and will replace the current log of the replica in the future.

```
DescribeLogDirs Response (Version: 2) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { error_code log_dir (topics) }
    error_code => INT16
    log_dir => COMPACT_STRING
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => { partition_index partition_size offset_lag is_future_key }
        partition_index => INT32
        partition_size => INT64
        offset_lag => INT64
        is_future_key => BOOLEAN
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The log directories.
error_code	The error code, or 0 if there was no error.
log_dir	The absolute log directory path.
topics	The topics.
name	The topic name.
partitions	The partitions.
partition_index	The partition index.

partition_size	The size of the log segments in this partition in bytes.
offset_lag	The lag of the log's LEO w.r.t. partition's HW (if it is the current log for the partition) or current replica's LEO (if it is the future log for the partition).
is_future_key	True if this log is created by AlterReplicaLogDirsRequest and will replace the current log of the replica in the future.

```
DescribeLogDirs Response (Version: 3) => { throttle_time_ms error_code (results)
  throttle_time_ms => INT32
  error_code => INT16
  results => { error_code log_dir (topics) }
    error_code => INT16
    log_dir => COMPACT_STRING
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => { partition_index partition_size offset_lag is_future_key }
        partition_index => INT32
        partition_size => INT64
        offset_lag => INT64
        is_future_key => BOOLEAN
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
results	The log directories.
error_code	The error code, or 0 if there was no error.
log_dir	The absolute log directory path.
topics	The topics.
name	The topic name.

partitions	The partitions.
partition_index	The partition index.
partition_size	The size of the log segments in this partition in bytes.
offset_lag	The lag of the log's LEO w.r.t. partition's HW (if it is the current log for the partition) or current replica's LEO (if it is the future log for the partition).
is_future_key	True if this log is created by AlterReplicaLogDirsRequest and will replace the current log of the replica in the future.

```
DescribeLogDirs Response (Version: 4) => { throttle_time_ms error_code (results)
  throttle_time_ms => INT32
  error_code => INT16
  results => { error_code log_dir (topics) total_bytes usable_bytes }
    error_code => INT16
    log_dir => COMPACT_STRING
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => { partition_index partition_size offset_lag is_future_key }
        partition_index => INT32
        partition_size => INT64
        offset_lag => INT64
        is_future_key => BOOLEAN
    total_bytes => INT64
    usable_bytes => INT64
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
results	The log directories.
error_code	The error code, or 0 if there was no error.

log_dir	The absolute log directory path.
topics	The topics.
name	The topic name.
partitions	The partitions.
partition_index	The partition index.
partition_size	The size of the log segments in this partition in bytes.
offset_lag	The lag of the log's LEO w.r.t. partition's HW (if it is the current log for the partition) or current replica's LEO (if it is the future log for the partition).
is_future_key	True if this log is created by AlterReplicaLogDirsRequest and will replace the current log of the replica in the future.
total_bytes	The total size in bytes of the volume the log directory is in. This value does not include the size of data stored in remote storage.
usable_bytes	The usable size in bytes of the volume the log directory is in. This value does not include the size of data stored in remote storage.

```
DescribeLogDirs Response (Version: 5) => { throttle_time_ms error_code (results)
  throttle_time_ms => INT32
  error_code => INT16
  results => { error_code log_dir (topics) total_bytes usable_bytes is_cordoned }
    error_code => INT16
    log_dir => COMPACT_STRING
    topics => { name (partitions) }
      name => COMPACT_STRING
      partitions => { partition_index partition_size offset_lag is_future_key }
        partition_index => INT32
        partition_size => INT64
        offset_lag => INT64
        is_future_key => BOOLEAN
    total_bytes => INT64
    usable_bytes => INT64
    is_cordoned => BOOLEAN
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
results	The log directories.
error_code	The error code, or 0 if there was no error.
log_dir	The absolute log directory path.
topics	The topics.
name	The topic name.
partitions	The partitions.
partition_index	The partition index.
partition_size	The size of the log segments in this partition in bytes.
offset_lag	The lag of the log's LEO w.r.t. partition's HW (if it is the current log for the partition) or current replica's LEO (if it is the future log for the partition).
is_future_key	True if this log is created by AlterReplicaLogDirsRequest and will replace the current log of the replica in the future.
total_bytes	The total size in bytes of the volume the log directory is in. This value does not include the size of data stored in remote storage.
usable_bytes	The usable size in bytes of the volume the log directory is in. This value does not include the size of data stored in remote storage.
is_cordoned	True if this log directory is cordoned.

SaslAuthenticate API (Key: 36):

Requests:

```
SaslAuthenticate Request (Version: 0) => { auth_bytes }
  auth_bytes => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
auth_bytes	The SASL authentication bytes from the client, as defined by the SASL mechanism.

```
SaslAuthenticate Request (Version: 1) => { auth_bytes }
  auth_bytes => BYTES
```

Request header version: 1

FIELD	DESCRIPTION
auth_bytes	The SASL authentication bytes from the client, as defined by the SASL mechanism.

```
SaslAuthenticate Request (Version: 2) => { auth_bytes }
  auth_bytes => COMPACT_BYTES
```

Request header version: 2

FIELD	DESCRIPTION
auth_bytes	The SASL authentication bytes from the client, as defined by the SASL mechanism.

Responses:

```
SaslAuthenticate Response (Version: 0) => { error_code error_message auth_bytes }
  error_code => INT16
  error_message => NULLABLE_STRING
  auth_bytes => BYTES
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
auth_bytes	The SASL authentication bytes from the server, as defined by the SASL mechanism.

```
SaslAuthenticate Response (Version: 1) => { error_code error_message auth_bytes s
  error_code => INT16
  error_message => NULLABLE_STRING
  auth_bytes => BYTES
  session_lifetime_ms => INT64
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
auth_bytes	The SASL authentication bytes from the server, as defined by the SASL mechanism.
session_lifetime_ms	Number of milliseconds after which only re-authentication over the existing connection to create a new session can occur.

```
SaslAuthenticate Response (Version: 2) => { error_code error_message auth_bytes s
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  auth_bytes => COMPACT_BYTES
  session_lifetime_ms => INT64
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.

error_message	The error message, or null if there was no error.
auth_bytes	The SASL authentication bytes from the server, as defined by the SASL mechanism.
session_lifetime_ms	Number of milliseconds after which only re-authentication over the existing connection to create a new session can occur.

CreatePartitions API (Key: 37):

Requests:

```
CreatePartitions Request (Version: 0) => { [topics] timeout_ms validate_only }
  topics => { name count ?[assignments] }
    name => STRING
    count => INT32
    assignments => { [broker_ids] }
      broker_ids => INT32
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	Each topic that we want to create new partitions inside.
name	The topic name.
count	The new partition count.
assignments	The new partition assignments.
broker_ids	The assigned broker IDs.
timeout_ms	The time in ms to wait for the partitions to be created.
validate_only	If true, then validate the request, but don't actually increase the number of partitions.

```
CreatePartitions Request (Version: 1) => { [topics] timeout_ms validate_only }
  topics => { name count ?[assignments] }
```

```
    name => STRING
    count => INT32
    assignments => { [broker_ids] }
      broker_ids => INT32
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
topics	Each topic that we want to create new partitions inside.
name	The topic name.
count	The new partition count.
assignments	The new partition assignments.
broker_ids	The assigned broker IDs.
timeout_ms	The time in ms to wait for the partitions to be created.
validate_only	If true, then validate the request, but don't actually increase the number of partitions.

```
CreatePartitions Request (Version: 2) => { (topics) timeout_ms validate_only }
  topics => { name count ?(assignments) }
    name => COMPACT_STRING
    count => INT32
    assignments => { (broker_ids) }
      broker_ids => INT32
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	Each topic that we want to create new partitions inside.
name	The topic name.

count	The new partition count.
assignments	The new partition assignments.
broker_ids	The assigned broker IDs.
timeout_ms	The time in ms to wait for the partitions to be created.
validate_only	If true, then validate the request, but don't actually increase the number of partitions.

```
CreatePartitions Request (Version: 3) => { (topics) timeout_ms validate_only }
  topics => { name count ?(assignments) }
    name => COMPACT_STRING
    count => INT32
    assignments => { (broker_ids) }
      broker_ids => INT32
  timeout_ms => INT32
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
topics	Each topic that we want to create new partitions inside.
name	The topic name.
count	The new partition count.
assignments	The new partition assignments.
broker_ids	The assigned broker IDs.
timeout_ms	The time in ms to wait for the partitions to be created.
validate_only	If true, then validate the request, but don't actually increase the number of partitions.

Responses:

```
CreatePartitions Response (Version: 0) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
```

```
  results => { name error_code error_message }
    name => STRING
    error_code => INT16
    error_message => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The partition creation results for each topic.
name	The topic name.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

```
CreatePartitions Response (Version: 1) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { name error_code error_message }
    name => STRING
    error_code => INT16
    error_message => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The partition creation results for each topic.
name	The topic name.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

```
CreatePartitions Response (Version: 2) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { name error_code error_message }
    name => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The partition creation results for each topic.
name	The topic name.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

```
CreatePartitions Response (Version: 3) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { name error_code error_message }
    name => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The partition creation results for each topic.
name	The topic name.
error_code	The result error, or zero if there was no error.

error_message	The result message, or null if there was no error.

CreateDelegationToken API (Key: 38):

Requests:

```
CreateDelegationToken Request (Version: 1) => { [renewers] max_lifetime_ms }
  renewers => { principal_type principal_name }
    principal_type => STRING
    principal_name => STRING
  max_lifetime_ms => INT64
```

Request header version: 1

FIELD	DESCRIPTION
renewers	A list of those who are allowed to renew this token before it expires.
principal_type	The type of the Kafka principal.
principal_name	The name of the Kafka principal.
max_lifetime_ms	The maximum lifetime of the token in milliseconds, or -1 to use the server side default.

```
CreateDelegationToken Request (Version: 2) => { (renewers) max_lifetime_ms }
  renewers => { principal_type principal_name }
    principal_type => COMPACT_STRING
    principal_name => COMPACT_STRING
  max_lifetime_ms => INT64
```

Request header version: 2

FIELD	DESCRIPTION
renewers	A list of those who are allowed to renew this token before it expires.
principal_type	The type of the Kafka principal.
principal_name	The name of the Kafka principal.

max_lifetime_ms	The maximum lifetime of the token in milliseconds, or -1 to use the server side default.

```
CreateDelegationToken Request (Version: 3) => { owner_principal_type owner_princi
  owner_principal_type => COMPACT_NULLABLE_STRING
  owner_principal_name => COMPACT_NULLABLE_STRING
  renewers => { principal_type principal_name }
    principal_type => COMPACT_STRING
    principal_name => COMPACT_STRING
  max_lifetime_ms => INT64
```

Request header version: 2

FIELD	DESCRIPTION
owner_principal_type	The principal type of the owner of the token. If it's null it defaults to the token request principal.
owner_principal_name	The principal name of the owner of the token. If it's null it defaults to the token request principal.
renewers	A list of those who are allowed to renew this token before it expires.
principal_type	The type of the Kafka principal.
principal_name	The name of the Kafka principal.
max_lifetime_ms	The maximum lifetime of the token in milliseconds, or -1 to use the server side default.

Responses:

```
CreateDelegationToken Response (Version: 1) => { error_code principal_type princi
  error_code => INT16
  principal_type => STRING
  principal_name => STRING
  issue_timestamp_ms => INT64
  expiry_timestamp_ms => INT64
  max_timestamp_ms => INT64
  token_id => STRING
  hmac => BYTES
```

```
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The top-level error, or zero if there was no error.
principal_type	The principal type of the token owner.
principal_name	The name of the token owner.
issue_timestamp_ms	When this token was generated.
expiry_timestamp_ms	When this token expires.
max_timestamp_ms	The maximum lifetime of this token.
token_id	The token UUID.
hmac	HMAC of the delegation token.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
CreateDelegationToken Response (Version: 2) => { error_code principal_type princi
  error_code => INT16
  principal_type => COMPACT_STRING
  principal_name => COMPACT_STRING
  issue_timestamp_ms => INT64
  expiry_timestamp_ms => INT64
  max_timestamp_ms => INT64
  token_id => COMPACT_STRING
  hmac => COMPACT_BYTES
  throttle_time_ms => INT32
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The top-level error, or zero if there was no error.

principal_type	The principal type of the token owner.
principal_name	The name of the token owner.
issue_timestamp_ms	When this token was generated.
expiry_timestamp_ms	When this token expires.
max_timestamp_ms	The maximum lifetime of this token.
token_id	The token UUID.
hmac	HMAC of the delegation token.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
CreateDelegationToken Response (Version: 3) => { error_code principal_type princi
  error_code => INT16
  principal_type => COMPACT_STRING
  principal_name => COMPACT_STRING
  token_requester_principal_type => COMPACT_STRING
  token_requester_principal_name => COMPACT_STRING
  issue_timestamp_ms => INT64
  expiry_timestamp_ms => INT64
  max_timestamp_ms => INT64
  token_id => COMPACT_STRING
  hmac => COMPACT_BYTES
  throttle_time_ms => INT32
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The top-level error, or zero if there was no error.
principal_type	The principal type of the token owner.
principal_name	The name of the token owner.
token_requester_principal_type	The principal type of the requester of the token.

token_requester_principal_name	The principal type of the requester of the token.
issue_timestamp_ms	When this token was generated.
expiry_timestamp_ms	When this token expires.
max_timestamp_ms	The maximum lifetime of this token.
token_id	The token UUID.
hmac	HMAC of the delegation token.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

RenewDelegationToken API (Key: 39):

Requests:

```
RenewDelegationToken Request (Version: 1) => { hmac renew_period_ms }
  hmac => BYTES
  renew_period_ms => INT64
```

Request header version: 1

FIELD	DESCRIPTION
hmac	The HMAC of the delegation token to be renewed.
renew_period_ms	The renewal time period in milliseconds.

```
RenewDelegationToken Request (Version: 2) => { hmac renew_period_ms }
  hmac => COMPACT_BYTES
  renew_period_ms => INT64
```

Request header version: 2

FIELD	DESCRIPTION
hmac	The HMAC of the delegation token to be renewed.

renew_period_ms	The renewal time period in milliseconds.

Responses:

```
RenewDelegationToken Response (Version: 1) => { error_code expiry_timestamp_ms th
  error_code => INT16
  expiry_timestamp_ms => INT64
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
expiry_timestamp_ms	The timestamp in milliseconds at which this token expires.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
RenewDelegationToken Response (Version: 2) => { error_code expiry_timestamp_ms th
  error_code => INT16
  expiry_timestamp_ms => INT64
  throttle_time_ms => INT32
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
expiry_timestamp_ms	The timestamp in milliseconds at which this token expires.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

ExpireDelegationToken API (Key: 40):

Requests:

```
ExpireDelegationToken Request (Version: 1) => { hmac expiry_time_period_ms }
  hmac => BYTES
  expiry_time_period_ms => INT64
```

Request header version: 1

FIELD	DESCRIPTION
hmac	The HMAC of the delegation token to be expired.
expiry_time_period_ms	The expiry time period in milliseconds.

```
ExpireDelegationToken Request (Version: 2) => { hmac expiry_time_period_ms }
  hmac => COMPACT_BYTES
  expiry_time_period_ms => INT64
```

Request header version: 2

FIELD	DESCRIPTION
hmac	The HMAC of the delegation token to be expired.
expiry_time_period_ms	The expiry time period in milliseconds.

Responses:

```
ExpireDelegationToken Response (Version: 1) => { error_code expiry_timestamp_ms t
  error_code => INT16
  expiry_timestamp_ms => INT64
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
expiry_timestamp_ms	The timestamp in milliseconds at which this token expires.

throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
ExpireDelegationToken Response (Version: 2) => { error_code expiry_timestamp_ms t
  error_code => INT16
  expiry_timestamp_ms => INT64
  throttle_time_ms => INT32
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
expiry_timestamp_ms	The timestamp in milliseconds at which this token expires.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

DescribeDelegationToken API (Key: 41):

Requests:

```
DescribeDelegationToken Request (Version: 1) => { ?[owners] }
  owners => { principal_type principal_name }
    principal_type => STRING
    principal_name => STRING
```

Request header version: 1

FIELD	DESCRIPTION
owners	Each owner that we want to describe delegation tokens for, or null to describe all tokens.
principal_type	The owner principal type.
principal_name	The owner principal name.

```
DescribeDelegationToken Request (Version: 2) => { ?(owners) }
  owners => { principal_type principal_name }
    principal_type => COMPACT_STRING
    principal_name => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
owners	Each owner that we want to describe delegation tokens for, or null to describe all tokens.
principal_type	The owner principal type.
principal_name	The owner principal name.

```
DescribeDelegationToken Request (Version: 3) => { ?(owners) }
  owners => { principal_type principal_name }
    principal_type => COMPACT_STRING
    principal_name => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
owners	Each owner that we want to describe delegation tokens for, or null to describe all tokens.
principal_type	The owner principal type.
principal_name	The owner principal name.

Responses:

```
DescribeDelegationToken Response (Version: 1) => { error_code [tokens] throttle_t
  error_code => INT16
  tokens => { principal_type principal_name issue_timestamp expiry_timestamp max_
    principal_type => STRING
    principal_name => STRING
    issue_timestamp => INT64
    expiry_timestamp => INT64
```

```
    max_timestamp => INT64
    token_id => STRING
    hmac => BYTES
    renewers => { principal_type principal_name }
      principal_type => STRING
      principal_name => STRING
  throttle_time_ms => INT32
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
tokens	The tokens.
principal_type	The token principal type.
principal_name	The token principal name.
issue_timestamp	The token issue timestamp in milliseconds.
expiry_timestamp	The token expiry timestamp in milliseconds.
max_timestamp	The token maximum timestamp length in milliseconds.
token_id	The token ID.
hmac	The token HMAC.
renewers	Those who are able to renew this token before it expires.
principal_type	The renewer principal type.
principal_name	The renewer principal name.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
DescribeDelegationToken Response (Version: 2) => { error_code (tokens) throttle_t
  error_code => INT16
  tokens => { principal_type principal_name issue_timestamp expiry_timestamp max_
    principal_type => COMPACT_STRING
    principal_name => COMPACT_STRING
```

```
    issue_timestamp => INT64
    expiry_timestamp => INT64
    max_timestamp => INT64
    token_id => COMPACT_STRING
    hmac => COMPACT_BYTES
    renewers => { principal_type principal_name }
      principal_type => COMPACT_STRING
      principal_name => COMPACT_STRING
  throttle_time_ms => INT32
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
tokens	The tokens.
principal_type	The token principal type.
principal_name	The token principal name.
issue_timestamp	The token issue timestamp in milliseconds.
expiry_timestamp	The token expiry timestamp in milliseconds.
max_timestamp	The token maximum timestamp length in milliseconds.
token_id	The token ID.
hmac	The token HMAC.
renewers	Those who are able to renew this token before it expires.
principal_type	The renewer principal type.
principal_name	The renewer principal name.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

```
DescribeDelegationToken Response (Version: 3) => { error_code (tokens) throttle_t
  error_code => INT16
  tokens => { principal_type principal_name token_requester_principal_type token_
```

```
    principal_type => COMPACT_STRING
    principal_name => COMPACT_STRING
    token_requester_principal_type => COMPACT_STRING
    token_requester_principal_name => COMPACT_STRING
    issue_timestamp => INT64
    expiry_timestamp => INT64
    max_timestamp => INT64
    token_id => COMPACT_STRING
    hmac => COMPACT_BYTES
    renewers => { principal_type principal_name }
      principal_type => COMPACT_STRING
      principal_name => COMPACT_STRING
  throttle_time_ms => INT32
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The error code, or 0 if there was no error.
tokens	The tokens.
principal_type	The token principal type.
principal_name	The token principal name.
token_requester_principal_type	The principal type of the requester of the token.
token_requester_principal_name	The principal type of the requester of the token.
issue_timestamp	The token issue timestamp in milliseconds.
expiry_timestamp	The token expiry timestamp in milliseconds.
max_timestamp	The token maximum timestamp length in milliseconds.
token_id	The token ID.
hmac	The token HMAC.
renewers	Those who are able to renew this token before it expires.
principal_type	The renewer principal type.

principal_name	The renewer principal name.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

DeleteGroups API (Key: 42):

Requests:

```
DeleteGroups Request (Version: 0) => { [groups_names] }
  groups_names => STRING
```

Request header version: 1

FIELD	DESCRIPTION
groups_names	The group names to delete.

```
DeleteGroups Request (Version: 1) => { [groups_names] }
  groups_names => STRING
```

Request header version: 1

FIELD	DESCRIPTION
groups_names	The group names to delete.

```
DeleteGroups Request (Version: 2) => { (groups_names) }
  groups_names => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
groups_names	The group names to delete.

Responses:

```
DeleteGroups Response (Version: 0) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { group_id error_code }
    group_id => STRING
    error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The deletion results.
group_id	The group id.
error_code	The deletion error, or 0 if the deletion succeeded.

```
DeleteGroups Response (Version: 1) => { throttle_time_ms [results] }
  throttle_time_ms => INT32
  results => { group_id error_code }
    group_id => STRING
    error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The deletion results.
group_id	The group id.
error_code	The deletion error, or 0 if the deletion succeeded.

```
DeleteGroups Response (Version: 2) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { group_id error_code }
```

```
    group_id => COMPACT_STRING
    error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The deletion results.
group_id	The group id.
error_code	The deletion error, or 0 if the deletion succeeded.

ElectLeaders API (Key: 43):

Requests:

```
ElectLeaders Request (Version: 0) => { ?[topic_partitions] timeout_ms }
  topic_partitions => { topic [partitions] }
    topic => STRING
    partitions => INT32
  timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
topic_partitions	The topic partitions to elect leaders.
topic	The name of a topic.
partitions	The partitions of this topic whose leader should be elected.
timeout_ms	The time in ms to wait for the election to complete.

```
ElectLeaders Request (Version: 1) => { election_type ?[topic_partitions] timeout_
  election_type => INT8
  topic_partitions => { topic [partitions] }
    topic => STRING
```

```
    partitions => INT32
  timeout_ms => INT32
```

Request header version: 1

FIELD	DESCRIPTION
election_type	Type of elections to conduct for the partition. A value of '0' elects the preferred replica. A value of '1' elects the first live replica if there are no in-sync replica.
topic_partitions	The topic partitions to elect leaders.
topic	The name of a topic.
partitions	The partitions of this topic whose leader should be elected.
timeout_ms	The time in ms to wait for the election to complete.

```
ElectLeaders Request (Version: 2) => { election_type ?(topic_partitions) timeout_
  election_type => INT8
  topic_partitions => { topic (partitions) }
    topic => COMPACT_STRING
    partitions => INT32
  timeout_ms => INT32
```

Request header version: 2

FIELD	DESCRIPTION
election_type	Type of elections to conduct for the partition. A value of '0' elects the preferred replica. A value of '1' elects the first live replica if there are no in-sync replica.
topic_partitions	The topic partitions to elect leaders.
topic	The name of a topic.
partitions	The partitions of this topic whose leader should be elected.
timeout_ms	The time in ms to wait for the election to complete.

Responses:

```
ElectLeaders Response (Version: 0) => { throttle_time_ms [replica_election_result
  throttle_time_ms => INT32
  replica_election_results => { topic [partition_result] }
    topic => STRING
    partition_result => { partition_id error_code error_message }
      partition_id => INT32
      error_code => INT16
      error_message => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
replica_election_results	The election results, or an empty array if the requester did not have permission and the request asks for all partitions.
topic	The topic name.
partition_result	The results for each partition.
partition_id	The partition id.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

```
ElectLeaders Response (Version: 1) => { throttle_time_ms error_code [replica_elec
  throttle_time_ms => INT32
  error_code => INT16
  replica_election_results => { topic [partition_result] }
    topic => STRING
    partition_result => { partition_id error_code error_message }
      partition_id => INT32
      error_code => INT16
      error_message => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
replica_election_results	The election results, or an empty array if the requester did not have permission and the request asks for all partitions.
topic	The topic name.
partition_result	The results for each partition.
partition_id	The partition id.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

```
ElectLeaders Response (Version: 2) => { throttle_time_ms error_code (replica_elec
  throttle_time_ms => INT32
  error_code => INT16
  replica_election_results => { topic (partition_result) }
    topic => COMPACT_STRING
    partition_result => { partition_id error_code error_message }
      partition_id => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.

replica_election_results	The election results, or an empty array if the requester did not have permission and the request asks for all partitions.
topic	The topic name.
partition_result	The results for each partition.
partition_id	The partition id.
error_code	The result error, or zero if there was no error.
error_message	The result message, or null if there was no error.

IncrementalAlterConfigs API (Key: 44):

Requests:

```
IncrementalAlterConfigs Request (Version: 0) => { [resources] validate_only }
  resources => { resource_type resource_name [configs] }
    resource_type => INT8
    resource_name => STRING
    configs => { name config_operation value }
      name => STRING
      config_operation => INT8
      value => NULLABLE_STRING
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
resources	The incremental updates for each resource.
resource_type	The resource type.
resource_name	The resource name.
configs	The configurations.
name	The configuration key name.
config_operation	The type (Set, Delete, Append, Subtract) of operation.

value	The value to set for the configuration key.
validate_only	True if we should validate the request, but not change the configurations.

```
IncrementalAlterConfigs Request (Version: 1) => { (resources) validate_only }
  resources => { resource_type resource_name (configs) }
    resource_type => INT8
    resource_name => COMPACT_STRING
    configs => { name config_operation value }
      name => COMPACT_STRING
      config_operation => INT8
      value => COMPACT_NULLABLE_STRING
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
resources	The incremental updates for each resource.
resource_type	The resource type.
resource_name	The resource name.
configs	The configurations.
name	The configuration key name.
config_operation	The type (Set, Delete, Append, Subtract) of operation.
value	The value to set for the configuration key.
validate_only	True if we should validate the request, but not change the configurations.

Responses:

```
IncrementalAlterConfigs Response (Version: 0) => { throttle_time_ms [responses] }
  throttle_time_ms => INT32
  responses => { error_code error_message resource_type resource_name }
    error_code => INT16
    error_message => NULLABLE_STRING
```

```
    resource_type => INT8
    resource_name => STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The responses for each resource.
error_code	The resource error code.
error_message	The resource error message, or null if there was no error.
resource_type	The resource type.
resource_name	The resource name.

```
IncrementalAlterConfigs Response (Version: 1) => { throttle_time_ms (responses) }
  throttle_time_ms => INT32
  responses => { error_code error_message resource_type resource_name }
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    resource_type => INT8
    resource_name => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
responses	The responses for each resource.
error_code	The resource error code.
error_message	The resource error message, or null if there was no error.
resource_type	The resource type.

resource_name	The resource name.

AlterPartitionReassignments API (Key: 45):

Requests:

```
AlterPartitionReassignments Request (Version: 0) => { timeout_ms (topics) }
  timeout_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index ?(replicas) }
      partition_index => INT32
      replicas => INT32
```

Request header version: 2

FIELD	DESCRIPTION
timeout_ms	The time in ms to wait for the request to complete.
topics	The topics to reassign.
name	The topic name.
partitions	The partitions to reassign.
partition_index	The partition index.
replicas	The replicas to place the partitions on, or null to cancel a pending reassignment for this partition.

```
AlterPartitionReassignments Request (Version: 1) => { timeout_ms allow_replicatio
  timeout_ms => INT32
  allow_replication_factor_change => BOOLEAN
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index ?(replicas) }
      partition_index => INT32
      replicas => INT32
```

Request header version: 2

FIELD	DESCRIPTION
timeout_ms	The time in ms to wait for the request to complete.
allow_replication_factor_change	The option indicating whether changing the replication factor of any given partition as part of this request is a valid move.
topics	The topics to reassign.
name	The topic name.
partitions	The partitions to reassign.
partition_index	The partition index.
replicas	The replicas to place the partitions on, or null to cancel a pending reassignment for this partition.

Responses:

```
AlterPartitionReassignments Response (Version: 0) => { throttle_time_ms error_cod
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  responses => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code error_message }
      partition_index => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.

responses	The responses to topics to reassign.
name	The topic name.
partitions	The responses to partitions to reassign.
partition_index	The partition index.
error_code	The error code for this partition, or 0 if there was no error.
error_message	The error message for this partition, or null if there was no error.

```
AlterPartitionReassignments Response (Version: 1) => { throttle_time_ms allow_rep
  throttle_time_ms => INT32
  allow_replication_factor_change => BOOLEAN
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  responses => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code error_message }
      partition_index => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
allow_replication_factor_change	The option indicating whether changing the replication factor of any given partition as part of the request was allowed.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
responses	The responses to topics to reassign.

name	The topic name.
partitions	The responses to partitions to reassign.
partition_index	The partition index.
error_code	The error code for this partition, or 0 if there was no error.
error_message	The error message for this partition, or null if there was no error.

ListPartitionReassignments API (Key: 46):

Requests:

```
ListPartitionReassignments Request (Version: 0) => { timeout_ms ?(topics) }
  timeout_ms => INT32
  topics => { name (partition_indexes) }
    name => COMPACT_STRING
    partition_indexes => INT32
```

Request header version: 2

FIELD	DESCRIPTION
timeout_ms	The time in ms to wait for the request to complete.
topics	The topics to list partition reassignments for, or null to list everything.
name	The topic name.
partition_indexes	The partitions to list partition reassignments for.

Responses:

```
ListPartitionReassignments Response (Version: 0) => { throttle_time_ms error_code
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index (replicas) (adding_replicas) (removing_replic
```

```
      partition_index => INT32
      replicas => INT32
      adding_replicas => INT32
      removing_replicas => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
topics	The ongoing reassignments for each topic.
name	The topic name.
partitions	The ongoing reassignments for each partition.
partition_index	The index of the partition.
replicas	The current replica set.
adding_replicas	The set of replicas we are currently adding.
removing_replicas	The set of replicas we are currently removing.

OffsetDelete API (Key: 47):

Requests:

```
OffsetDelete Request (Version: 0) => { group_id [topics] }
  group_id => STRING
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index }
      partition_index => INT32
```

Request header version: 1

FIELD	DESCRIPTION
group_id	The unique group identifier.
topics	The topics to delete offsets for.
name	The topic name.
partitions	Each partition to delete offsets for.
partition_index	The partition index.

Responses:

```
OffsetDelete Response (Version: 0) => { error_code throttle_time_ms [topics] }
  error_code => INT16
  throttle_time_ms => INT32
  topics => { name [partitions] }
    name => STRING
    partitions => { partition_index error_code }
      partition_index => INT32
      error_code => INT16
```

Response header version: 0

FIELD	DESCRIPTION
error_code	The top-level error code, or 0 if there was no error.
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	The responses for each topic.
name	The topic name.
partitions	The responses for each partition in the topic.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.

DescribeClientQuotas API (Key: 48):

Requests:

```
DescribeClientQuotas Request (Version: 0) => { [components] strict }
  components => { entity_type match_type match }
    entity_type => STRING
    match_type => INT8
    match => NULLABLE_STRING
  strict => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
components	Filter components to apply to quota entities.
entity_type	The entity type that the filter component applies to.
match_type	How to match the entity {0 = exact name, 1 = default name, 2 = any specified name}.
match	The string to match against, or null if unused for the match type.
strict	Whether the match is strict, i.e. should exclude entities with unspecified entity types.

```
DescribeClientQuotas Request (Version: 1) => { (components) strict }
  components => { entity_type match_type match }
    entity_type => COMPACT_STRING
    match_type => INT8
    match => COMPACT_NULLABLE_STRING
  strict => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
components	Filter components to apply to quota entities.
entity_type	The entity type that the filter component applies to.
match_type	How to match the entity {0 = exact name, 1 = default name, 2 = any specified name}.

match	The string to match against, or null if unused for the match type.
strict	Whether the match is strict, i.e. should exclude entities with unspecified entity types.

Responses:

```
DescribeClientQuotas Response (Version: 0) => { throttle_time_ms error_code error
  throttle_time_ms => INT32
  error_code => INT16
  error_message => NULLABLE_STRING
  entries => { [entity] [values] }
    entity => { entity_type entity_name }
      entity_type => STRING
      entity_name => NULLABLE_STRING
    values => { key value }
      key => STRING
      value => FLOAT64
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or `0` if the quota description succeeded.
error_message	The error message, or `null` if the quota description succeeded.
entries	A result entry.
entity	The quota entity description.
entity_type	The entity type.
entity_name	The entity name, or null if the default.
values	The quota values for the entity.
key	The quota configuration key.
value	The quota configuration value.

```
DescribeClientQuotas Response (Version: 1) => { throttle_time_ms error_code error
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  entries => { (entity) (values) }
    entity => { entity_type entity_name }
      entity_type => COMPACT_STRING
      entity_name => COMPACT_NULLABLE_STRING
    values => { key value }
      key => COMPACT_STRING
      value => FLOAT64
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or `0` if the quota description succeeded.
error_message	The error message, or `null` if the quota description succeeded.
entries	A result entry.
entity	The quota entity description.
entity_type	The entity type.
entity_name	The entity name, or null if the default.
values	The quota values for the entity.
key	The quota configuration key.
value	The quota configuration value.

AlterClientQuotas API (Key: 49):

Requests:

```
AlterClientQuotas Request (Version: 0) => { [entries] validate_only }
  entries => { [entity] [ops] }
```

```
    entity => { entity_type entity_name }
      entity_type => STRING
      entity_name => NULLABLE_STRING
    ops => { key value remove }
      key => STRING
      value => FLOAT64
      remove => BOOLEAN
  validate_only => BOOLEAN
```

Request header version: 1

FIELD	DESCRIPTION
entries	The quota configuration entries to alter.
entity	The quota entity to alter.
entity_type	The entity type.
entity_name	The name of the entity, or null if the default.
ops	An individual quota configuration entry to alter.
key	The quota configuration key.
value	The value to set, otherwise ignored if the value is to be removed.
remove	Whether the quota configuration value should be removed, otherwise set.
validate_only	Whether the alteration should be validated, but not performed.

```
AlterClientQuotas Request (Version: 1) => { (entries) validate_only }
  entries => { (entity) (ops) }
    entity => { entity_type entity_name }
      entity_type => COMPACT_STRING
      entity_name => COMPACT_NULLABLE_STRING
    ops => { key value remove }
      key => COMPACT_STRING
      value => FLOAT64
      remove => BOOLEAN
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
entries	The quota configuration entries to alter.
entity	The quota entity to alter.
entity_type	The entity type.
entity_name	The name of the entity, or null if the default.
ops	An individual quota configuration entry to alter.
key	The quota configuration key.
value	The value to set, otherwise ignored if the value is to be removed.
remove	Whether the quota configuration value should be removed, otherwise set.
validate_only	Whether the alteration should be validated, but not performed.

Responses:

```
AlterClientQuotas Response (Version: 0) => { throttle_time_ms [entries] }
  throttle_time_ms => INT32
  entries => { error_code error_message [entity] }
    error_code => INT16
    error_message => NULLABLE_STRING
    entity => { entity_type entity_name }
      entity_type => STRING
      entity_name => NULLABLE_STRING
```

Response header version: 0

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
entries	The quota configuration entries to alter.
error_code	The error code, or `0` if the quota alteration succeeded.
error_message	The error message, or `null` if the quota alteration succeeded.

entity	The quota entity to alter.
entity_type	The entity type.
entity_name	The name of the entity, or null if the default.

```
AlterClientQuotas Response (Version: 1) => { throttle_time_ms (entries) }
  throttle_time_ms => INT32
  entries => { error_code error_message (entity) }
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    entity => { entity_type entity_name }
      entity_type => COMPACT_STRING
      entity_name => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
entries	The quota configuration entries to alter.
error_code	The error code, or `0` if the quota alteration succeeded.
error_message	The error message, or `null` if the quota alteration succeeded.
entity	The quota entity to alter.
entity_type	The entity type.
entity_name	The name of the entity, or null if the default.

DescribeUserScramCredentials API (Key: 50):

Requests:

```
DescribeUserScramCredentials Request (Version: 0) => { ?(users) }
  users => { name }
    name => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
users	The users to describe, or null/empty to describe all users.
name	The user name.

Responses:

```
DescribeUserScramCredentials Response (Version: 0) => { throttle_time_ms error_co
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  results => { user error_code error_message (credential_infos) }
    user => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    credential_infos => { mechanism iterations }
      mechanism => INT8
      iterations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The message-level error code, 0 except for user authorization or infrastructure issues.
error_message	The message-level error message, if any.
results	The results for descriptions, one per user.
user	The user name.
error_code	The user-level error code.
error_message	The user-level error message, if any.

credential_infos	The mechanism and related information associated with the user's SCRAM credentials.
mechanism	The SCRAM mechanism.
iterations	The number of iterations used in the SCRAM credential.

AlterUserScramCredentials API (Key: 51):

Requests:

```
AlterUserScramCredentials Request (Version: 0) => { (deletions) (upsertions) }
  deletions => { name mechanism }
    name => COMPACT_STRING
    mechanism => INT8
  upsertions => { name mechanism iterations salt salted_password }
    name => COMPACT_STRING
    mechanism => INT8
    iterations => INT32
    salt => COMPACT_BYTES
    salted_password => COMPACT_BYTES
```

Request header version: 2

FIELD	DESCRIPTION
deletions	The SCRAM credentials to remove.
name	The user name.
mechanism	The SCRAM mechanism.
upsertions	The SCRAM credentials to update/insert.
name	The user name.
mechanism	The SCRAM mechanism.
iterations	The number of iterations.
salt	A random salt generated by the client.
salted_password	The salted password.

Responses:

```
AlterUserScramCredentials Response (Version: 0) => { throttle_time_ms (results) }
  throttle_time_ms => INT32
  results => { user error_code error_message }
    user => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
results	The results for deletions and alterations, one per affected user.
user	The user name.
error_code	The error code.
error_message	The error message, if any.

DescribeQuorum API (Key: 55):

Requests:

```
DescribeQuorum Request (Version: 0) => { (topics) }
  topics => { topic_name (partitions) }
    topic_name => COMPACT_STRING
    partitions => { partition_index }
      partition_index => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to describe.
topic_name	The topic name.

partitions	The partitions to describe.
partition_index	The partition index.

```
DescribeQuorum Request (Version: 1) => { (topics) }
  topics => { topic_name (partitions) }
    topic_name => COMPACT_STRING
    partitions => { partition_index }
      partition_index => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to describe.
topic_name	The topic name.
partitions	The partitions to describe.
partition_index	The partition index.

```
DescribeQuorum Request (Version: 2) => { (topics) }
  topics => { topic_name (partitions) }
    topic_name => COMPACT_STRING
    partitions => { partition_index }
      partition_index => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to describe.
topic_name	The topic name.
partitions	The partitions to describe.
partition_index	The partition index.

Responses:

```
DescribeQuorum Response (Version: 0) => { error_code (topics) }
  error_code => INT16
  topics => { topic_name (partitions) }
    topic_name => COMPACT_STRING
    partitions => { partition_index error_code leader_id leader_epoch high_waterm
      partition_index => INT32
      error_code => INT16
      leader_id => INT32
      leader_epoch => INT32
      high_watermark => INT64
      current_voters => { replica_id log_end_offset }
        replica_id => INT32
        log_end_offset => INT64
      observers => { replica_id log_end_offset }
        replica_id => INT32
        log_end_offset => INT64
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The top level error code.
topics	The response from the describe quorum API.
topic_name	The topic name.
partitions	The partition data.
partition_index	The partition index.
error_code	The partition error code.
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.
high_watermark	The high water mark.
current_voters	The current voters of the partition.
replica_id	The ID of the replica.
log_end_offset	The last known log end offset of the follower or -1 if it is unknown.

observers	The observers of the partition.

```
DescribeQuorum Response (Version: 1) => { error_code (topics) }
  error_code => INT16
  topics => { topic_name (partitions) }
    topic_name => COMPACT_STRING
    partitions => { partition_index error_code leader_id leader_epoch high_waterm
      partition_index => INT32
      error_code => INT16
      leader_id => INT32
      leader_epoch => INT32
      high_watermark => INT64
      current_voters => { replica_id log_end_offset last_fetch_timestamp last_cau
        replica_id => INT32
        log_end_offset => INT64
        last_fetch_timestamp => INT64
        last_caught_up_timestamp => INT64
      observers => { replica_id log_end_offset last_fetch_timestamp last_caught_u
        replica_id => INT32
        log_end_offset => INT64
        last_fetch_timestamp => INT64
        last_caught_up_timestamp => INT64
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The top level error code.
topics	The response from the describe quorum API.
topic_name	The topic name.
partitions	The partition data.
partition_index	The partition index.
error_code	The partition error code.
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.

high_watermark	The high water mark.
current_voters	The current voters of the partition.
replica_id	The ID of the replica.
log_end_offset	The last known log end offset of the follower or -1 if it is unknown.
last_fetch_timestamp	The last known leader wall clock time time when a follower fetched from the leader. This is reported as -1 both for the current leader or if it is unknown for a voter.
last_caught_up_timestamp	The leader wall clock append time of the offset for which the follower made the most recent fetch request. This is reported as the current time for the leader and -1 if unknown for a voter.
observers	The observers of the partition.

```
DescribeQuorum Response (Version: 2) => { error_code error_message (topics) (node
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  topics => { topic_name (partitions) }
    topic_name => COMPACT_STRING
    partitions => { partition_index error_code error_message leader_id leader_epo
      partition_index => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      leader_id => INT32
      leader_epoch => INT32
      high_watermark => INT64
      current_voters => { replica_id replica_directory_id log_end_offset last_fet
        replica_id => INT32
        replica_directory_id => UUID
        log_end_offset => INT64
        last_fetch_timestamp => INT64
        last_caught_up_timestamp => INT64
      observers => { replica_id replica_directory_id log_end_offset last_fetch_ti
        replica_id => INT32
        replica_directory_id => UUID
        log_end_offset => INT64
        last_fetch_timestamp => INT64
```

```
        last_caught_up_timestamp => INT64
    nodes => { node_id (listeners) }
      node_id => INT32
      listeners => { name host port }
        name => COMPACT_STRING
        host => COMPACT_STRING
        port => UINT16
```

Response header version: 1

FIELD	DESCRIPTION
error_code	The top level error code.
error_message	The error message, or null if there was no error.
topics	The response from the describe quorum API.
topic_name	The topic name.
partitions	The partition data.
partition_index	The partition index.
error_code	The partition error code.
error_message	The error message, or null if there was no error.
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.
high_watermark	The high water mark.
current_voters	The current voters of the partition.
replica_id	The ID of the replica.
replica_directory_id	The replica directory ID of the replica.
log_end_offset	The last known log end offset of the follower or -1 if it is unknown.
last_fetch_timestamp	The last known leader wall clock time time when a follower fetched from the leader. This is reported as -1 both for the

	current leader or if it is unknown for a voter.
last_caught_up_timestamp	The leader wall clock append time of the offset for which the follower made the most recent fetch request. This is reported as the current time for the leader and -1 if unknown for a voter.
observers	The observers of the partition.
nodes	The nodes in the quorum.
node_id	The ID of the associated node.
listeners	The listeners of this controller.
name	The name of the endpoint.
host	The hostname.
port	The port.

UpdateFeatures API (Key: 57):

Requests:

```
UpdateFeatures Request (Version: 0) => { timeout_ms (feature_updates) }
  timeout_ms => INT32
  feature_updates => { feature max_version_level allow_downgrade }
    feature => COMPACT_STRING
    max_version_level => INT16
    allow_downgrade => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
timeout_ms	How long to wait in milliseconds before timing out the request.
feature_updates	The list of updates to finalized features.
feature	The name of the finalized feature to be updated.
max_version_level	The new maximum version level for the finalized feature. A value >= 1 is valid. A value < 1, is special, and can be used to request the deletion

	of the finalized feature.
allow_downgrade	DEPRECATED in version 1 (see DowngradeType). When set to true, the finalized feature version level is allowed to be downgraded/deleted. The downgrade request will fail if the new maximum version level is a value that's not lower than the existing maximum finalized version level.

```
UpdateFeatures Request (Version: 1) => { timeout_ms (feature_updates) validate_on
  timeout_ms => INT32
  feature_updates => { feature max_version_level upgrade_type }
    feature => COMPACT_STRING
    max_version_level => INT16
    upgrade_type => INT8
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
timeout_ms	How long to wait in milliseconds before timing out the request.
feature_updates	The list of updates to finalized features.
feature	The name of the finalized feature to be updated.
max_version_level	The new maximum version level for the finalized feature. A value >= 1 is valid. A value < 1, is special, and can be used to request the deletion of the finalized feature.
upgrade_type	Determine which type of upgrade will be performed: 1 will perform an upgrade only (default), 2 is safe downgrades only (lossless), 3 is unsafe downgrades (lossy).
validate_only	True if we should validate the request, but not perform the upgrade or downgrade.

```
UpdateFeatures Request (Version: 2) => { timeout_ms (feature_updates) validate_on
  timeout_ms => INT32
  feature_updates => { feature max_version_level upgrade_type }
    feature => COMPACT_STRING
```

```
    max_version_level => INT16
    upgrade_type => INT8
  validate_only => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
timeout_ms	How long to wait in milliseconds before timing out the request.
feature_updates	The list of updates to finalized features.
feature	The name of the finalized feature to be updated.
max_version_level	The new maximum version level for the finalized feature. A value >= 1 is valid. A value < 1, is special, and can be used to request the deletion of the finalized feature.
upgrade_type	Determine which type of upgrade will be performed: 1 will perform an upgrade only (default), 2 is safe downgrades only (lossless), 3 is unsafe downgrades (lossy).
validate_only	True if we should validate the request, but not perform the upgrade or downgrade.

Responses:

```
UpdateFeatures Response (Version: 0) => { throttle_time_ms error_code error_messa
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  results => { feature error_code error_message }
    feature => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

error_code	The top-level error code, or `0` if there was no top-level error.
error_message	The top-level error message, or `null` if there was no top-level error.
results	Results for each feature update.
feature	The name of the finalized feature.
error_code	The feature update error code or `0` if the feature update succeeded.
error_message	The feature update error, or `null` if the feature update succeeded.

```
UpdateFeatures Response (Version: 1) => { throttle_time_ms error_code error_messa
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  results => { feature error_code error_message }
    feature => COMPACT_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or `0` if there was no top-level error.
error_message	The top-level error message, or `null` if there was no top-level error.
results	Results for each feature update.
feature	The name of the finalized feature.
error_code	The feature update error code or `0` if the feature update succeeded.
error_message	The feature update error, or `null` if the feature update succeeded.

```
UpdateFeatures Response (Version: 2) => { throttle_time_ms error_code error_messa
  throttle_time_ms => INT32
```

```
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or `0` if there was no top-level error.
error_message	The top-level error message, or `null` if there was no top-level error.

DescribeCluster API (Key: 60):

Requests:

```
DescribeCluster Request (Version: 0) => { include_cluster_authorized_operations }
  include_cluster_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
include_cluster_authorized_operations	Whether to include cluster authorized operations.

```
DescribeCluster Request (Version: 1) => { include_cluster_authorized_operations e
  include_cluster_authorized_operations => BOOLEAN
  endpoint_type => INT8
```

Request header version: 2

FIELD	DESCRIPTION
include_cluster_authorized_operations	Whether to include cluster authorized operations.
endpoint_type	The endpoint type to describe. 1=brokers, 2=controllers.

```
DescribeCluster Request (Version: 2) => { include_cluster_authorized_operations e
  include_cluster_authorized_operations => BOOLEAN
  endpoint_type => INT8
  include_fenced_brokers => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
include_cluster_authorized_operations	Whether to include cluster authorized operations.
endpoint_type	The endpoint type to describe. 1=brokers, 2=controllers.
include_fenced_brokers	Whether to include fenced brokers when listing brokers.

Responses:

```
DescribeCluster Response (Version: 0) => { throttle_time_ms error_code error_mess
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  cluster_id => COMPACT_STRING
  controller_id => INT32
  brokers => { broker_id host port rack }
    broker_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
  cluster_authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.

error_message	The top-level error message, or null if there was no error.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller. When handled by a controller, returns the current voter leader ID. When handled by a broker, returns a random alive broker ID as a fallback.
brokers	Each broker in the response.
broker_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_authorized_operations	32-bit bitfield to represent authorized operations for this cluster.

```
DescribeCluster Response (Version: 1) => { throttle_time_ms error_code error_mess
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  endpoint_type => INT8
  cluster_id => COMPACT_STRING
  controller_id => INT32
  brokers => { broker_id host port rack }
    broker_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
  cluster_authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request

	did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
endpoint_type	The endpoint type that was described. 1=brokers, 2=controllers.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller. When handled by a controller, returns the current voter leader ID. When handled by a broker, returns a random alive broker ID as a fallback.
brokers	Each broker in the response.
broker_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
cluster_authorized_operations	32-bit bitfield to represent authorized operations for this cluster.

```
DescribeCluster Response (Version: 2) => { throttle_time_ms error_code error_mess
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  endpoint_type => INT8
  cluster_id => COMPACT_STRING
  controller_id => INT32
  brokers => { broker_id host port rack is_fenced }
    broker_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
    is_fenced => BOOLEAN
  cluster_authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
endpoint_type	The endpoint type that was described. 1=brokers, 2=controllers.
cluster_id	The cluster ID that responding broker belongs to.
controller_id	The ID of the controller. When handled by a controller, returns the current voter leader ID. When handled by a broker, returns a random alive broker ID as a fallback.
brokers	Each broker in the response.
broker_id	The broker ID.
host	The broker hostname.
port	The broker port.
rack	The rack of the broker, or null if it has not been assigned to a rack.
is_fenced	Whether the broker is fenced
cluster_authorized_operations	32-bit bitfield to represent authorized operations for this cluster.

DescribeProducers API (Key: 61):

Requests:

```
DescribeProducers Request (Version: 0) => { (topics) }
  topics => { name (partition_indexes) }
    name => COMPACT_STRING
```

```
    partition_indexes => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to list producers for.
name	The topic name.
partition_indexes	The indexes of the partitions to list producers for.

Responses:

```
DescribeProducers Response (Version: 0) => { throttle_time_ms (topics) }
  throttle_time_ms => INT32
  topics => { name (partitions) }
    name => COMPACT_STRING
    partitions => { partition_index error_code error_message (active_producers) }
      partition_index => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      active_producers => { producer_id producer_epoch last_sequence last_timesta
        producer_id => INT64
        producer_epoch => INT32
        last_sequence => INT32
        last_timestamp => INT64
        coordinator_epoch => INT32
        current_txn_start_offset => INT64
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
name	The topic name.
partitions	Each partition in the response.

partition_index	The partition index.
error_code	The partition error code, or 0 if there was no error.
error_message	The partition error message, which may be null if no additional details are available.
active_producers	The active producers for the partition.
producer_id	The producer id.
producer_epoch	The producer epoch.
last_sequence	The last sequence number sent by the producer.
last_timestamp	The last timestamp sent by the producer.
coordinator_epoch	The current epoch of the producer group.
current_txn_start_offset	The current transaction start offset of the producer.

UnregisterBroker API (Key: 64):

Requests:

```
UnregisterBroker Request (Version: 0) => { broker_id }
  broker_id => INT32
```

Request header version: 2

FIELD	DESCRIPTION
broker_id	The broker ID to unregister.

Responses:

```
UnregisterBroker Response (Version: 0) => { throttle_time_ms error_code error_mes
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	Duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The top-level error message, or `null` if there was no top-level error.

DescribeTransactions API (Key: 65):

Requests:

```
DescribeTransactions Request (Version: 0) => { (transactional_ids) }
  transactional_ids => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
transactional_ids	Array of transactionalIds to include in describe results. If empty, then no results will be returned.

Responses:

```
DescribeTransactions Response (Version: 0) => { throttle_time_ms (transaction_sta
  throttle_time_ms => INT32
  transaction_states => { error_code transactional_id transaction_state transacti
    error_code => INT16
    transactional_id => COMPACT_STRING
    transaction_state => COMPACT_STRING
    transaction_timeout_ms => INT32
    transaction_start_time_ms => INT64
    producer_id => INT64
    producer_epoch => INT16
    topics => { topic (partitions) }
      topic => COMPACT_STRING
      partitions => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
transaction_states	The current state of the transaction.
error_code	The error code.
transactional_id	The transactional id.
transaction_state	The current transaction state of the producer.
transaction_timeout_ms	The timeout in milliseconds for the transaction.
transaction_start_time_ms	The start time of the transaction in milliseconds.
producer_id	The current producer id associated with the transaction.
producer_epoch	The current epoch associated with the producer id.
topics	The set of partitions included in the current transaction (if active). When a transaction is preparing to commit or abort, this will include only partitions which do not have markers.
topic	The topic name.
partitions	The partition ids included in the current transaction.

ListTransactions API (Key: 66):

Requests:

```
ListTransactions Request (Version: 0) => { (state_filters) (producer_id_filters)
  state_filters => COMPACT_STRING
  producer_id_filters => INT64
```

Request header version: 2

FIELD	DESCRIPTION

state_filters	The transaction states to filter by: if empty, all transactions are returned; if non-empty, then only transactions matching one of the filtered states will be returned.
producer_id_filters	The producerIds to filter by: if empty, all transactions will be returned; if non-empty, only transactions which match one of the filtered producerIds will be returned.

```
ListTransactions Request (Version: 1) => { (state_filters) (producer_id_filters)
  state_filters => COMPACT_STRING
  producer_id_filters => INT64
  duration_filter => INT64
```

Request header version: 2

FIELD	DESCRIPTION
state_filters	The transaction states to filter by: if empty, all transactions are returned; if non-empty, then only transactions matching one of the filtered states will be returned.
producer_id_filters	The producerIds to filter by: if empty, all transactions will be returned; if non-empty, only transactions which match one of the filtered producerIds will be returned.
duration_filter	Duration (in millis) to filter by: if < 0, all transactions will be returned; otherwise, only transactions running longer than this duration will be returned.

```
ListTransactions Request (Version: 2) => { (state_filters) (producer_id_filters)
  state_filters => COMPACT_STRING
  producer_id_filters => INT64
  duration_filter => INT64
  transactional_id_pattern => COMPACT_NULLABLE_STRING
```

Request header version: 2

FIELD	DESCRIPTION

state_filters	The transaction states to filter by: if empty, all transactions are returned; if non-empty, then only transactions matching one of the filtered states will be returned.
producer_id_filters	The producerIds to filter by: if empty, all transactions will be returned; if non-empty, only transactions which match one of the filtered producerIds will be returned.
duration_filter	Duration (in millis) to filter by: if < 0, all transactions will be returned; otherwise, only transactions running longer than this duration will be returned.
transactional_id_pattern	The transactional ID regular expression pattern to filter by: if it is empty or null, all transactions are returned; Otherwise then only the transactions matching the given regular expression will be returned.

Responses:

```
ListTransactions Response (Version: 0) => { throttle_time_ms error_code (unknown_
  throttle_time_ms => INT32
  error_code => INT16
  unknown_state_filters => COMPACT_STRING
  transaction_states => { transactional_id producer_id transaction_state }
    transactional_id => COMPACT_STRING
    producer_id => INT64
    transaction_state => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
unknown_state_filters	Set of state filters provided in the request which were unknown to the transaction coordinator.
transaction_states	The current state of the transaction for the transactional id.

transactional_id	The transactional id.
producer_id	The producer id.
transaction_state	The current transaction state of the producer.

```
ListTransactions Response (Version: 1) => { throttle_time_ms error_code (unknown_
  throttle_time_ms => INT32
  error_code => INT16
  unknown_state_filters => COMPACT_STRING
  transaction_states => { transactional_id producer_id transaction_state }
    transactional_id => COMPACT_STRING
    producer_id => INT64
    transaction_state => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
unknown_state_filters	Set of state filters provided in the request which were unknown to the transaction coordinator.
transaction_states	The current state of the transaction for the transactional id.
transactional_id	The transactional id.
producer_id	The producer id.
transaction_state	The current transaction state of the producer.

```
ListTransactions Response (Version: 2) => { throttle_time_ms error_code (unknown_
  throttle_time_ms => INT32
  error_code => INT16
  unknown_state_filters => COMPACT_STRING
  transaction_states => { transactional_id producer_id transaction_state }
    transactional_id => COMPACT_STRING
```

```
    producer_id => INT64
    transaction_state => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
unknown_state_filters	Set of state filters provided in the request which were unknown to the transaction coordinator.
transaction_states	The current state of the transaction for the transactional id.
transactional_id	The transactional id.
producer_id	The producer id.
transaction_state	The current transaction state of the producer.

ConsumerGroupHeartbeat API (Key: 68):

Requests:

```
ConsumerGroupHeartbeat Request (Version: 0) => { group_id member_id member_epoch
  group_id => COMPACT_STRING
  member_id => COMPACT_STRING
  member_epoch => INT32
  instance_id => COMPACT_NULLABLE_STRING
  rack_id => COMPACT_NULLABLE_STRING
  rebalance_timeout_ms => INT32
  subscribed_topic_names => COMPACT_STRING
  server_assignor => COMPACT_NULLABLE_STRING
  topic_partitions => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
member_id	The member id generated by the consumer. The member id must be kept during the entire lifetime of the consumer process.
member_epoch	The current member epoch; 0 to join the group; -1 to leave the group; -2 to indicate that the static member will rejoin.
instance_id	null if not provided or if it didn't change since the last heartbeat; the instance Id otherwise.
rack_id	null if not provided or if it didn't change since the last heartbeat; the rack ID of consumer otherwise.
rebalance_timeout_ms	-1 if it didn't change since the last heartbeat; the maximum time in milliseconds that the coordinator will wait on the member to revoke its partitions otherwise.
subscribed_topic_names	null if it didn't change since the last heartbeat; the subscribed topic names otherwise.
server_assignor	null if not used or if it didn't change since the last heartbeat; the server side assignor to use otherwise.
topic_partitions	null if it didn't change since the last heartbeat; the partitions owned by the member.
topic_id	The topic ID.
partitions	The partitions.

```
ConsumerGroupHeartbeat Request (Version: 1) => { group_id member_id member_epoch
  group_id => COMPACT_STRING
  member_id => COMPACT_STRING
  member_epoch => INT32
  instance_id => COMPACT_NULLABLE_STRING
  rack_id => COMPACT_NULLABLE_STRING
  rebalance_timeout_ms => INT32
  subscribed_topic_names => COMPACT_STRING
  subscribed_topic_regex => COMPACT_NULLABLE_STRING
  server_assignor => COMPACT_NULLABLE_STRING
```

```
  topic_partitions => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
member_id	The member id generated by the consumer. The member id must be kept during the entire lifetime of the consumer process.
member_epoch	The current member epoch; 0 to join the group; -1 to leave the group; -2 to indicate that the static member will rejoin.
instance_id	null if not provided or if it didn't change since the last heartbeat; the instance Id otherwise.
rack_id	null if not provided or if it didn't change since the last heartbeat; the rack ID of consumer otherwise.
rebalance_timeout_ms	-1 if it didn't change since the last heartbeat; the maximum time in milliseconds that the coordinator will wait on the member to revoke its partitions otherwise.
subscribed_topic_names	null if it didn't change since the last heartbeat; the subscribed topic names otherwise.
subscribed_topic_regex	null if it didn't change since the last heartbeat; the subscribed topic regex otherwise.
server_assignor	null if not used or if it didn't change since the last heartbeat; the server side assignor to use otherwise.
topic_partitions	null if it didn't change since the last heartbeat; the partitions owned by the member.
topic_id	The topic ID.
partitions	The partitions.

Responses:

```
ConsumerGroupHeartbeat Response (Version: 0) => { throttle_time_ms error_code err
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  member_id => COMPACT_NULLABLE_STRING
  member_epoch => INT32
  heartbeat_interval_ms => INT32
  assignment => ?{ (topic_partitions) }
    topic_partitions => { topic_id (partitions) }
      topic_id => UUID
      partitions => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
member_id	The member id is generated by the consumer starting from version 1, while in version 0, it can be provided by users or generated by the group coordinator.
member_epoch	The member epoch.
heartbeat_interval_ms	The heartbeat interval in milliseconds.
assignment	null if not provided; the assignment otherwise.
topic_partitions	The partitions assigned to the member that can be used immediately.
topic_id	The topic ID.
partitions	The partitions.

```
ConsumerGroupHeartbeat Response (Version: 1) => { throttle_time_ms error_code err
```

```
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  member_id => COMPACT_NULLABLE_STRING
  member_epoch => INT32
  heartbeat_interval_ms => INT32
  assignment => ?{ (topic_partitions) }
    topic_partitions => { topic_id (partitions) }
      topic_id => UUID
      partitions => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
member_id	The member id is generated by the consumer starting from version 1, while in version 0, it can be provided by users or generated by the group coordinator.
member_epoch	The member epoch.
heartbeat_interval_ms	The heartbeat interval in milliseconds.
assignment	null if not provided; the assignment otherwise.
topic_partitions	The partitions assigned to the member that can be used immediately.
topic_id	The topic ID.
partitions	The partitions.

ConsumerGroupDescribe API (Key: 69):

Requests:

```
ConsumerGroupDescribe Request (Version: 0) => { (group_ids) include_authorized_op
  group_ids => COMPACT_STRING
  include_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
group_ids	The ids of the groups to describe.
include_authorized_operations	Whether to include authorized operations.

```
ConsumerGroupDescribe Request (Version: 1) => { (group_ids) include_authorized_op
  group_ids => COMPACT_STRING
  include_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
group_ids	The ids of the groups to describe.
include_authorized_operations	Whether to include authorized operations.

Responses:

```
ConsumerGroupDescribe Response (Version: 0) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { error_code error_message group_id group_state group_epoch assignmen
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    group_id => COMPACT_STRING
    group_state => COMPACT_STRING
    group_epoch => INT32
    assignment_epoch => INT32
    assignor_name => COMPACT_STRING
    members => { member_id instance_id rack_id member_epoch client_id client_host
      member_id => COMPACT_STRING
      instance_id => COMPACT_NULLABLE_STRING
      rack_id => COMPACT_NULLABLE_STRING
      member_epoch => INT32
      client_id => COMPACT_STRING
```

```
      client_host => COMPACT_STRING
      subscribed_topic_names => COMPACT_STRING
      subscribed_topic_regex => COMPACT_NULLABLE_STRING
      assignment => { (topic_partitions) }
        topic_partitions => { topic_id topic_name (partitions) }
          topic_id => UUID
          topic_name => COMPACT_STRING
          partitions => INT32
      target_assignment => { (topic_partitions) }
        topic_partitions => { topic_id topic_name (partitions) }
          topic_id => UUID
          topic_name => COMPACT_STRING
          partitions => INT32
    authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
group_epoch	The group epoch.
assignment_epoch	The assignment epoch.
assignor_name	The selected assignor.
members	The members.
member_id	The member ID.
instance_id	The member instance ID.

rack_id	The member rack ID.
member_epoch	The current member epoch.
client_id	The client ID.
client_host	The client host.
subscribed_topic_names	The subscribed topic names.
subscribed_topic_regex	the subscribed topic regex otherwise or null of not provided.
assignment	The current assignment.
topic_partitions	The assigned topic-partitions to the member.
topic_id	The topic ID.
topic_name	The topic name.
partitions	The partitions.
target_assignment	The target assignment.
authorized_operations	32-bit bitfield to represent authorized operations for this group.

```
ConsumerGroupDescribe Response (Version: 1) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { error_code error_message group_id group_state group_epoch assignmen
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    group_id => COMPACT_STRING
    group_state => COMPACT_STRING
    group_epoch => INT32
    assignment_epoch => INT32
    assignor_name => COMPACT_STRING
    members => { member_id instance_id rack_id member_epoch client_id client_host
      member_id => COMPACT_STRING
      instance_id => COMPACT_NULLABLE_STRING
      rack_id => COMPACT_NULLABLE_STRING
      member_epoch => INT32
      client_id => COMPACT_STRING
      client_host => COMPACT_STRING
      subscribed_topic_names => COMPACT_STRING
      subscribed_topic_regex => COMPACT_NULLABLE_STRING
```

```
      assignment => { (topic_partitions) }
        topic_partitions => { topic_id topic_name (partitions) }
          topic_id => UUID
          topic_name => COMPACT_STRING
          partitions => INT32
      target_assignment => { (topic_partitions) }
        topic_partitions => { topic_id topic_name (partitions) }
          topic_id => UUID
          topic_name => COMPACT_STRING
          partitions => INT32
      member_type => INT8
    authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
group_epoch	The group epoch.
assignment_epoch	The assignment epoch.
assignor_name	The selected assignor.
members	The members.
member_id	The member ID.
instance_id	The member instance ID.
rack_id	The member rack ID.

member_epoch	The current member epoch.
client_id	The client ID.
client_host	The client host.
subscribed_topic_names	The subscribed topic names.
subscribed_topic_regex	the subscribed topic regex otherwise or null of not provided.
assignment	The current assignment.
topic_partitions	The assigned topic-partitions to the member.
topic_id	The topic ID.
topic_name	The topic name.
partitions	The partitions.
target_assignment	The target assignment.
member_type	-1 for unknown. 0 for classic member. +1 for consumer member.
authorized_operations	32-bit bitfield to represent authorized operations for this group.

GetTelemetrySubscriptions API (Key: 71):

Requests:

```
GetTelemetrySubscriptions Request (Version: 0) => { client_instance_id }
  client_instance_id => UUID
```

Request header version: 2

FIELD	DESCRIPTION
client_instance_id	Unique id for this client instance, must be set to 0 on the first request.

Responses:

```
GetTelemetrySubscriptions Response (Version: 0) => { throttle_time_ms error_code
  throttle_time_ms => INT32
  error_code => INT16
```

```
  client_instance_id => UUID
  subscription_id => INT32
  accepted_compression_types => INT8
  push_interval_ms => INT32
  telemetry_max_bytes => INT32
  delta_temporality => BOOLEAN
  requested_metrics => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
client_instance_id	Assigned client instance id if ClientInstanceId was 0 in the request, else 0.
subscription_id	Unique identifier for the current subscription set for this client instance.
accepted_compression_types	Compression types that broker accepts for the PushTelemetryRequest.
push_interval_ms	Configured push interval, which is the lowest configured interval in the current subscription set.
telemetry_max_bytes	The maximum bytes of binary data the broker accepts in PushTelemetryRequest.
delta_temporality	Flag to indicate monotonic/counter metrics are to be emitted as deltas or cumulative values.
requested_metrics	Requested metrics prefix string match. Empty array: No metrics subscribed, Array[0] empty string: All metrics subscribed.

PushTelemetry API (Key: 72):

Requests:

```
PushTelemetry Request (Version: 0) => { client_instance_id subscription_id termin
  client_instance_id => UUID
  subscription_id => INT32
  terminating => BOOLEAN
  compression_type => INT8
  metrics => COMPACT_BYTES
```

Request header version: 2

FIELD	DESCRIPTION
client_instance_id	Unique id for this client instance.
subscription_id	Unique identifier for the current subscription.
terminating	Client is terminating the connection.
compression_type	Compression codec used to compress the metrics.
metrics	Metrics encoded in OpenTelemetry MetricsData v1 protobuf format.

Responses:

```
PushTelemetry Response (Version: 0) => { throttle_time_ms error_code }
  throttle_time_ms => INT32
  error_code => INT16
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.

ListConfigResources API (Key: 74):

Requests:

```
ListConfigResources Request (Version: 0) => { }
```

Request header version: 2

FIELD	DESCRIPTION

```
ListConfigResources Request (Version: 1) => { (resource_types) }
  resource_types => INT8
```

Request header version: 2

FIELD	DESCRIPTION
resource_types	The list of resource type. If the list is empty, it uses default supported config resource types.

Responses:

```
ListConfigResources Response (Version: 0) => { throttle_time_ms error_code (confi
  throttle_time_ms => INT32
  error_code => INT16
  config_resources => { resource_name }
    resource_name => COMPACT_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
config_resources	Each config resource in the response.
resource_name	The resource name.

```
ListConfigResources Response (Version: 1) => { throttle_time_ms error_code (confi
  throttle_time_ms => INT32
```

```
  error_code => INT16
  config_resources => { resource_name resource_type }
    resource_name => COMPACT_STRING
    resource_type => INT8
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
config_resources	Each config resource in the response.
resource_name	The resource name.
resource_type	The resource type.

DescribeTopicPartitions API (Key: 75):

Requests:

```
DescribeTopicPartitions Request (Version: 0) => { (topics) response_partition_lim
  topics => { name }
    name => COMPACT_STRING
  response_partition_limit => INT32
  cursor => ?{ topic_name partition_index }
    topic_name => COMPACT_STRING
    partition_index => INT32
```

Request header version: 2

FIELD	DESCRIPTION
topics	The topics to fetch details for.
name	The topic name.
response_partition_limit	The maximum number of partitions included in the response.

cursor	The first topic and partition index to fetch details for.
topic_name	The name for the first topic to process.
partition_index	The partition index to start with.

Responses:

```
DescribeTopicPartitions Response (Version: 0) => { throttle_time_ms (topics) next
  throttle_time_ms => INT32
  topics => { error_code name topic_id is_internal (partitions) topic_authorized_
    error_code => INT16
    name => COMPACT_NULLABLE_STRING
    topic_id => UUID
    is_internal => BOOLEAN
    partitions => { error_code partition_index leader_id leader_epoch (replica_no
      error_code => INT16
      partition_index => INT32
      leader_id => INT32
      leader_epoch => INT32
      replica_nodes => INT32
      isr_nodes => INT32
      eligible_leader_replicas => INT32
      last_known_elr => INT32
      offline_replicas => INT32
    topic_authorized_operations => INT32
  next_cursor => ?{ topic_name partition_index }
    topic_name => COMPACT_STRING
    partition_index => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
topics	Each topic in the response.
error_code	The topic error, or 0 if there was no error.
name	The topic name.

topic_id	The topic id.
is_internal	True if the topic is internal.
partitions	Each partition in the topic.
error_code	The partition error, or 0 if there was no error.
partition_index	The partition index.
leader_id	The ID of the leader broker.
leader_epoch	The leader epoch of this partition.
replica_nodes	The set of all nodes that host this partition.
isr_nodes	The set of nodes that are in sync with the leader for this partition.
eligible_leader_replicas	The new eligible leader replicas otherwise.
last_known_elr	The last known ELR.
offline_replicas	The set of offline replicas of this partition.
topic_authorized_operations	32-bit bitfield to represent authorized operations for this topic.
next_cursor	The next topic and partition index to fetch details for.
topic_name	The name for the first topic to process.
partition_index	The partition index to start with.

ShareGroupHeartbeat API (Key: 76):

Requests:

```
ShareGroupHeartbeat Request (Version: 1) => { group_id member_id member_epoch rac
  group_id => COMPACT_STRING
  member_id => COMPACT_STRING
  member_epoch => INT32
  rack_id => COMPACT_NULLABLE_STRING
  subscribed_topic_names => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
member_id	The member id generated by the consumer. The member id must be kept during the entire lifetime of the consumer process.
member_epoch	The current member epoch; 0 to join the group; -1 to leave the group.
rack_id	null if not provided or if it didn't change since the last heartbeat; the rack ID of consumer otherwise.
subscribed_topic_names	null if it didn't change since the last heartbeat; the subscribed topic names otherwise.

Responses:

```
ShareGroupHeartbeat Response (Version: 1) => { throttle_time_ms error_code error_
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  member_id => COMPACT_NULLABLE_STRING
  member_epoch => INT32
  heartbeat_interval_ms => INT32
  assignment => ?{ (topic_partitions) }
    topic_partitions => { topic_id (partitions) }
      topic_id => UUID
      partitions => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.

member_id	The member ID is generated by the consumer and provided by the consumer for all requests.
member_epoch	The member epoch.
heartbeat_interval_ms	The heartbeat interval in milliseconds.
assignment	null if not provided; the assignment otherwise.
topic_partitions	The partitions assigned to the member.
topic_id	The topic ID.
partitions	The partitions.

ShareGroupDescribe API (Key: 77):

Requests:

```
ShareGroupDescribe Request (Version: 1) => { (group_ids) include_authorized_opera
  group_ids => COMPACT_STRING
  include_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
group_ids	The ids of the groups to describe.
include_authorized_operations	Whether to include authorized operations.

Responses:

```
ShareGroupDescribe Response (Version: 1) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { error_code error_message group_id group_state group_epoch assignmen
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    group_id => COMPACT_STRING
    group_state => COMPACT_STRING
    group_epoch => INT32
    assignment_epoch => INT32
    assignor_name => COMPACT_STRING
```

```
    members => { member_id rack_id member_epoch client_id client_host (subscribed
      member_id => COMPACT_STRING
      rack_id => COMPACT_NULLABLE_STRING
      member_epoch => INT32
      client_id => COMPACT_STRING
      client_host => COMPACT_STRING
      subscribed_topic_names => COMPACT_STRING
      assignment => { (topic_partitions) }
        topic_partitions => { topic_id topic_name (partitions) }
          topic_id => UUID
          topic_name => COMPACT_STRING
          partitions => INT32
    authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
group_epoch	The group epoch.
assignment_epoch	The assignment epoch.
assignor_name	The selected assignor.
members	The members.
member_id	The member ID.
rack_id	The member rack ID.

member_epoch	The current member epoch.
client_id	The client ID.
client_host	The client host.
subscribed_topic_names	The subscribed topic names.
assignment	The current assignment.
topic_partitions	The assigned topic-partitions to the member.
topic_id	The topic ID.
topic_name	The topic name.
partitions	The partitions.
authorized_operations	32-bit bitfield to represent authorized operations for this group.

ShareFetch API (Key: 78):

Requests:

```
ShareFetch Request (Version: 1) => { group_id member_id share_session_epoch max_w
  group_id => COMPACT_NULLABLE_STRING
  member_id => COMPACT_NULLABLE_STRING
  share_session_epoch => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  max_records => INT32
  batch_size => INT32
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index (acknowledgement_batches) }
      partition_index => INT32
      acknowledgement_batches => { first_offset last_offset (acknowledge_types) }
        first_offset => INT64
        last_offset => INT64
        acknowledge_types => INT8
  forgotten_topics_data => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
member_id	The member ID.
share_session_epoch	The current share session epoch: 0 to open a share session; -1 to close it; otherwise increments for consecutive requests.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.
max_records	The maximum number of records to fetch. This limit can be exceeded for alignment of batch boundaries.
batch_size	The optimal number of records for batches of acquired records and acknowledgements.
topics	The topics to fetch.
topic_id	The unique topic ID.
partitions	The partitions to fetch.
partition_index	The partition index.
acknowledgement_batches	Record batches to acknowledge.
first_offset	First offset of batch of records to acknowledge.
last_offset	Last offset (inclusive) of batch of records to acknowledge.
acknowledge_types	Array of acknowledge types - 0:Gap,1:Accept,2:Release,3:Reject,4:Renew.
forgotten_topics_data	The partitions to remove from this share session.
topic_id	The unique topic ID.

partitions	The partitions indexes to forget.

```
ShareFetch Request (Version: 2) => { group_id member_id share_session_epoch max_w
  group_id => COMPACT_NULLABLE_STRING
  member_id => COMPACT_NULLABLE_STRING
  share_session_epoch => INT32
  max_wait_ms => INT32
  min_bytes => INT32
  max_bytes => INT32
  max_records => INT32
  batch_size => INT32
  share_acquire_mode => INT8
  is_renew_ack => BOOLEAN
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index (acknowledgement_batches) }
      partition_index => INT32
      acknowledgement_batches => { first_offset last_offset (acknowledge_types) }
        first_offset => INT64
        last_offset => INT64
        acknowledge_types => INT8
  forgotten_topics_data => { topic_id (partitions) }
    topic_id => UUID
    partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
member_id	The member ID.
share_session_epoch	The current share session epoch: 0 to open a share session; -1 to close it; otherwise increments for consecutive requests.
max_wait_ms	The maximum time in milliseconds to wait for the response.
min_bytes	The minimum bytes to accumulate in the response.
max_bytes	The maximum bytes to fetch. See KIP-74 for cases where this limit may not be honored.

max_records	The maximum number of records to fetch. This limit can be exceeded for alignment of batch boundaries.
batch_size	The optimal number of records for batches of acquired records and acknowledgements.
share_acquire_mode	The acquire mode to control the fetch behavior - 0:batch-optimized,1:record-limit.
is_renew_ack	Whether Renew type acknowledgements present in AcknowledgementBatches.
topics	The topics to fetch.
topic_id	The unique topic ID.
partitions	The partitions to fetch.
partition_index	The partition index.
acknowledgement_batches	Record batches to acknowledge.
first_offset	First offset of batch of records to acknowledge.
last_offset	Last offset (inclusive) of batch of records to acknowledge.
acknowledge_types	Array of acknowledge types - 0:Gap,1:Accept,2:Release,3:Reject,4:Renew.
forgotten_topics_data	The partitions to remove from this share session.
topic_id	The unique topic ID.
partitions	The partitions indexes to forget.

Responses:

```
ShareFetch Response (Version: 1) => { throttle_time_ms error_code error_message a
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  acquisition_lock_timeout_ms => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code error_message acknowledge_error_co
```

```
      partition_index => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      acknowledge_error_code => INT16
      acknowledge_error_message => COMPACT_NULLABLE_STRING
      current_leader => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
      records => COMPACT_RECORDS
      acquired_records => { first_offset last_offset delivery_count }
        first_offset => INT64
        last_offset => INT64
        delivery_count => INT16
  node_endpoints => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level response error code.
error_message	The top-level error message, or null if there was no error.
acquisition_lock_timeout_ms	The time in milliseconds for which the acquired records are locked.
responses	The response topics.
topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The fetch error code, or 0 if there was no fetch error.

error_message	The fetch error message, or null if there was no fetch error.
acknowledge_error_code	The acknowledge error code, or 0 if there was no acknowledge error.
acknowledge_error_message	The acknowledge error message, or null if there was no acknowledge error.
current_leader	The current leader of the partition.
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.
records	The record data.
acquired_records	The acquired records.
first_offset	The earliest offset in this batch of acquired records.
last_offset	The last offset of this batch of acquired records.
delivery_count	The delivery count of this batch of acquired records.
node_endpoints	Endpoints for all current leaders enumerated in PartitionData with error NOT_LEADER_OR_FOLLOWER.
node_id	The ID of the associated node.
host	The node's hostname.
port	The node's port.
rack	The rack of the node, or null if it has not been assigned to a rack.

```
ShareFetch Response (Version: 2) => { throttle_time_ms error_code error_message a
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  acquisition_lock_timeout_ms => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code error_message acknowledge_error_co
```

```
      partition_index => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      acknowledge_error_code => INT16
      acknowledge_error_message => COMPACT_NULLABLE_STRING
      current_leader => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
      records => COMPACT_RECORDS
      acquired_records => { first_offset last_offset delivery_count }
        first_offset => INT64
        last_offset => INT64
        delivery_count => INT16
  node_endpoints => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level response error code.
error_message	The top-level error message, or null if there was no error.
acquisition_lock_timeout_ms	The time in milliseconds for which the acquired records are locked.
responses	The response topics.
topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The fetch error code, or 0 if there was no fetch error.

error_message	The fetch error message, or null if there was no fetch error.
acknowledge_error_code	The acknowledge error code, or 0 if there was no acknowledge error.
acknowledge_error_message	The acknowledge error message, or null if there was no acknowledge error.
current_leader	The current leader of the partition.
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.
records	The record data.
acquired_records	The acquired records.
first_offset	The earliest offset in this batch of acquired records.
last_offset	The last offset of this batch of acquired records.
delivery_count	The delivery count of this batch of acquired records.
node_endpoints	Endpoints for all current leaders enumerated in PartitionData with error NOT_LEADER_OR_FOLLOWER.
node_id	The ID of the associated node.
host	The node's hostname.
port	The node's port.
rack	The rack of the node, or null if it has not been assigned to a rack.

ShareAcknowledge API (Key: 79):

Requests:

```
ShareAcknowledge Request (Version: 1) => { group_id member_id share_session_epoch
  group_id => COMPACT_NULLABLE_STRING
  member_id => COMPACT_NULLABLE_STRING
  share_session_epoch => INT32
```

```
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index (acknowledgement_batches) }
      partition_index => INT32
      acknowledgement_batches => { first_offset last_offset (acknowledge_types) }
        first_offset => INT64
        last_offset => INT64
        acknowledge_types => INT8
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
member_id	The member ID.
share_session_epoch	The current share session epoch: 0 to open a share session; -1 to close it; otherwise increments for consecutive requests.
topics	The topics containing records to acknowledge.
topic_id	The unique topic ID.
partitions	The partitions containing records to acknowledge.
partition_index	The partition index.
acknowledgement_batches	Record batches to acknowledge.
first_offset	First offset of batch of records to acknowledge.
last_offset	Last offset (inclusive) of batch of records to acknowledge.
acknowledge_types	Array of acknowledge types - 0:Gap,1:Accept,2:Release,3:Reject,4:Renew.

```
ShareAcknowledge Request (Version: 2) => { group_id member_id share_session_epoch
  group_id => COMPACT_NULLABLE_STRING
  member_id => COMPACT_NULLABLE_STRING
  share_session_epoch => INT32
  is_renew_ack => BOOLEAN
  topics => { topic_id (partitions) }
    topic_id => UUID
```

```
    partitions => { partition_index (acknowledgement_batches) }
      partition_index => INT32
      acknowledgement_batches => { first_offset last_offset (acknowledge_types) }
        first_offset => INT64
        last_offset => INT64
        acknowledge_types => INT8
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
member_id	The member ID.
share_session_epoch	The current share session epoch: 0 to open a share session; -1 to close it; otherwise increments for consecutive requests.
is_renew_ack	Whether Renew type acknowledgements present in AcknowledgementBatches.
topics	The topics containing records to acknowledge.
topic_id	The unique topic ID.
partitions	The partitions containing records to acknowledge.
partition_index	The partition index.
acknowledgement_batches	Record batches to acknowledge.
first_offset	First offset of batch of records to acknowledge.
last_offset	Last offset (inclusive) of batch of records to acknowledge.
acknowledge_types	Array of acknowledge types - 0:Gap,1:Accept,2:Release,3:Reject,4:Renew.

Responses:

```
ShareAcknowledge Response (Version: 1) => { throttle_time_ms error_code error_mes
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
```

```
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code error_message current_leader }
      partition_index => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      current_leader => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
  node_endpoints => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
error_message	The top-level error message, or null if there was no error.
responses	The response topics.
topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
current_leader	The current leader of the partition.
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.

node_endpoints	Endpoints for all current leaders enumerated in PartitionData with error NOT_LEADER_OR_FOLLOWER.
node_id	The ID of the associated node.
host	The node's hostname.
port	The node's port.
rack	The rack of the node, or null if it has not been assigned to a rack.

```
ShareAcknowledge Response (Version: 2) => { throttle_time_ms error_code error_mes
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  acquisition_lock_timeout_ms => INT32
  responses => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition_index error_code error_message current_leader }
      partition_index => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      current_leader => { leader_id leader_epoch }
        leader_id => INT32
        leader_epoch => INT32
  node_endpoints => { node_id host port rack }
    node_id => INT32
    host => COMPACT_STRING
    port => INT32
    rack => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top level response error code.
error_message	The top-level error message, or null if there was no error.

acquisition_lock_timeout_ms	The time in milliseconds for which the acquired records are locked.
responses	The response topics.
topic_id	The unique topic ID.
partitions	The topic partitions.
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
current_leader	The current leader of the partition.
leader_id	The ID of the current leader or -1 if the leader is unknown.
leader_epoch	The latest known leader epoch.
node_endpoints	Endpoints for all current leaders enumerated in PartitionData with error NOT_LEADER_OR_FOLLOWER.
node_id	The ID of the associated node.
host	The node's hostname.
port	The node's port.
rack	The rack of the node, or null if it has not been assigned to a rack.

AddRaftVoter API (Key: 80):

Requests:

```
AddRaftVoter Request (Version: 0) => { cluster_id timeout_ms voter_id voter_direc
  cluster_id => COMPACT_NULLABLE_STRING
  timeout_ms => INT32
  voter_id => INT32
  voter_directory_id => UUID
  listeners => { name host port }
    name => COMPACT_STRING
```

```
    host => COMPACT_STRING
    port => UINT16
```

Request header version: 2

FIELD	DESCRIPTION
cluster_id	The cluster id.
timeout_ms	The maximum time to wait for the request to complete before returning.
voter_id	The replica id of the voter getting added to the topic partition.
voter_directory_id	The directory id of the voter getting added to the topic partition.
listeners	The endpoints that can be used to communicate with the voter.
name	The name of the endpoint.
host	The hostname.
port	The port.

```
AddRaftVoter Request (Version: 1) => { cluster_id timeout_ms voter_id voter_direc
  cluster_id => COMPACT_NULLABLE_STRING
  timeout_ms => INT32
  voter_id => INT32
  voter_directory_id => UUID
  listeners => { name host port }
    name => COMPACT_STRING
    host => COMPACT_STRING
    port => UINT16
  ack_when_committed => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
cluster_id	The cluster id.
timeout_ms	The maximum time to wait for the request to complete before

	returning.
voter_id	The replica id of the voter getting added to the topic partition.
voter_directory_id	The directory id of the voter getting added to the topic partition.
listeners	The endpoints that can be used to communicate with the voter.
name	The name of the endpoint.
host	The hostname.
port	The port.
ack_when_committed	When true, return a response after the new voter set is committed. Otherwise, return after the leader writes the changes locally.

Responses:

```
AddRaftVoter Response (Version: 0) => { throttle_time_ms error_code error_message
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

```
AddRaftVoter Response (Version: 1) => { throttle_time_ms error_code error_message
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

RemoveRaftVoter API (Key: 81):

Requests:

```
RemoveRaftVoter Request (Version: 0) => { cluster_id voter_id voter_directory_id
  cluster_id => COMPACT_NULLABLE_STRING
  voter_id => INT32
  voter_directory_id => UUID
```

Request header version: 2

FIELD	DESCRIPTION
cluster_id	The cluster id of the request.
voter_id	The replica id of the voter getting removed from the topic partition.
voter_directory_id	The directory id of the voter getting removed from the topic partition.

Responses:

```
RemoveRaftVoter Response (Version: 0) => { throttle_time_ms error_code error_mess
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.

error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

InitializeShareGroupState API (Key: 83):

Requests:

```
InitializeShareGroupState Request (Version: 0) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition state_epoch start_offset }
      partition => INT32
      state_epoch => INT32
      start_offset => INT64
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
topics	The data for the topics.
topic_id	The topic identifier.
partitions	The data for the partitions.
partition	The partition index.
state_epoch	The state epoch for this share-partition.
start_offset	The share-partition start offset, or -1 if the start offset is not being initialized.

Responses:

```
InitializeShareGroupState Response (Version: 0) => { (results) }
  results => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition error_code error_message }
      partition => INT32
      error_code => INT16
```

```
      error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
results	The initialization results.
topic_id	The topic identifier.
partitions	The results for the partitions.
partition	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

ReadShareGroupState API (Key: 84):

Requests:

```
ReadShareGroupState Request (Version: 0) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition leader_epoch }
      partition => INT32
      leader_epoch => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
topics	The data for the topics.
topic_id	The topic identifier.
partitions	The data for the partitions.
partition	The partition index.

leader_epoch	The leader epoch of the share-partition.

Responses:

```
ReadShareGroupState Response (Version: 0) => { (results) }
  results => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition error_code error_message state_epoch start_offset (
      partition => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      state_epoch => INT32
      start_offset => INT64
      state_batches => { first_offset last_offset delivery_state delivery_count }
        first_offset => INT64
        last_offset => INT64
        delivery_state => INT8
        delivery_count => INT16
```

Response header version: 1

FIELD	DESCRIPTION
results	The read results.
topic_id	The topic identifier.
partitions	The results for the partitions.
partition	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
state_epoch	The state epoch of the share-partition.
start_offset	The share-partition start offset, which can be -1 if it is not yet initialized.
state_batches	The state batches for this share-partition.
first_offset	The first offset of this state batch.
last_offset	The last offset of this state batch.

delivery_state	The delivery state - 0:Available,2:Acked,4:Archived.
delivery_count	The delivery count.

WriteShareGroupState API (Key: 85):

Requests:

```
WriteShareGroupState Request (Version: 0) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition state_epoch leader_epoch start_offset (state_batche
      partition => INT32
      state_epoch => INT32
      leader_epoch => INT32
      start_offset => INT64
      state_batches => { first_offset last_offset delivery_state delivery_count }
        first_offset => INT64
        last_offset => INT64
        delivery_state => INT8
        delivery_count => INT16
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
topics	The data for the topics.
topic_id	The topic identifier.
partitions	The data for the partitions.
partition	The partition index.
state_epoch	The state epoch of the share-partition.
leader_epoch	The leader epoch of the share-partition.
start_offset	The share-partition start offset, or -1 if the start offset is not being written.

state_batches	The state batches for the share-partition.
first_offset	The first offset of this state batch.
last_offset	The last offset of this state batch.
delivery_state	The delivery state - 0:Available,2:Acked,4:Archived.
delivery_count	The delivery count.

```
WriteShareGroupState Request (Version: 1) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition state_epoch leader_epoch start_offset delivery_comp
      partition => INT32
      state_epoch => INT32
      leader_epoch => INT32
      start_offset => INT64
      delivery_complete_count => INT32
      state_batches => { first_offset last_offset delivery_state delivery_count }
        first_offset => INT64
        last_offset => INT64
        delivery_state => INT8
        delivery_count => INT16
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
topics	The data for the topics.
topic_id	The topic identifier.
partitions	The data for the partitions.
partition	The partition index.
state_epoch	The state epoch of the share-partition.
leader_epoch	The leader epoch of the share-partition.

start_offset	The share-partition start offset, or -1 if the start offset is not being written.
delivery_complete_count	The number of offsets greater than or equal to share-partition start offset for which delivery has been completed.
state_batches	The state batches for the share-partition.
first_offset	The first offset of this state batch.
last_offset	The last offset of this state batch.
delivery_state	The delivery state - 0:Available,2:Acked,4:Archived.
delivery_count	The delivery count.

Responses:

```
WriteShareGroupState Response (Version: 0) => { (results) }
  results => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition error_code error_message }
      partition => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
results	The write results.
topic_id	The topic identifier.
partitions	The results for the partitions.
partition	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

```
WriteShareGroupState Response (Version: 1) => { (results) }
```

```
  results => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition error_code error_message }
      partition => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
results	The write results.
topic_id	The topic identifier.
partitions	The results for the partitions.
partition	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

DeleteShareGroupState API (Key: 86):

Requests:

```
DeleteShareGroupState Request (Version: 0) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition }
      partition => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
topics	The data for the topics.
topic_id	The topic identifier.

partitions	The data for the partitions.
partition	The partition index.

Responses:

```
DeleteShareGroupState Response (Version: 0) => { (results) }
  results => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition error_code error_message }
      partition => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
results	The delete results.
topic_id	The topic identifier.
partitions	The results for the partitions.
partition	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

ReadShareGroupStateSummary API (Key: 87):

Requests:

```
ReadShareGroupStateSummary Request (Version: 0) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition leader_epoch }
      partition => INT32
      leader_epoch => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
topics	The data for the topics.
topic_id	The topic identifier.
partitions	The data for the partitions.
partition	The partition index.
leader_epoch	The leader epoch of the share-partition.

```
ReadShareGroupStateSummary Request (Version: 1) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition leader_epoch }
      partition => INT32
      leader_epoch => INT32
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
topics	The data for the topics.
topic_id	The topic identifier.
partitions	The data for the partitions.
partition	The partition index.
leader_epoch	The leader epoch of the share-partition.

Responses:

```
ReadShareGroupStateSummary Response (Version: 0) => { (results) }
  results => { topic_id (partitions) }
    topic_id => UUID
```

```
    partitions => { partition error_code error_message state_epoch leader_epoch s
      partition => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      state_epoch => INT32
      leader_epoch => INT32
      start_offset => INT64
```

Response header version: 1

FIELD	DESCRIPTION
results	The read results.
topic_id	The topic identifier.
partitions	The results for the partitions.
partition	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
state_epoch	The state epoch of the share-partition.
leader_epoch	The leader epoch of the share-partition.
start_offset	The share-partition start offset.

```
ReadShareGroupStateSummary Response (Version: 1) => { (results) }
  results => { topic_id (partitions) }
    topic_id => UUID
    partitions => { partition error_code error_message state_epoch leader_epoch s
      partition => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
      state_epoch => INT32
      leader_epoch => INT32
      start_offset => INT64
      delivery_complete_count => INT32
```

Response header version: 1

FIELD	DESCRIPTION
results	The read results.
topic_id	The topic identifier.
partitions	The results for the partitions.
partition	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.
state_epoch	The state epoch of the share-partition.
leader_epoch	The leader epoch of the share-partition.
start_offset	The share-partition start offset.
delivery_complete_count	The number of offsets greater than or equal to share-partition start offset for which delivery has been completed.

StreamsGroupHeartbeat API (Key: 88):

Requests:

```
StreamsGroupHeartbeat Request (Version: 0) => { group_id member_id member_epoch e
  group_id => COMPACT_STRING
  member_id => COMPACT_STRING
  member_epoch => INT32
  endpoint_information_epoch => INT32
  instance_id => COMPACT_NULLABLE_STRING
  rack_id => COMPACT_NULLABLE_STRING
  rebalance_timeout_ms => INT32
  topology => ?{ epoch (subtopologies) }
    epoch => INT32
    subtopologies => { subtopology_id (source_topics) (source_topic_regex) (state
      subtopology_id => COMPACT_STRING
      source_topics => COMPACT_STRING
      source_topic_regex => COMPACT_STRING
      state_changelog_topics => { name partitions replication_factor (topic_confi
        name => COMPACT_STRING
        partitions => INT32
        replication_factor => INT16
```

```
      topic_configs => { key value }
        key => COMPACT_STRING
        value => COMPACT_STRING
    repartition_sink_topics => COMPACT_STRING
    repartition_source_topics => { name partitions replication_factor (topic_co
      name => COMPACT_STRING
      partitions => INT32
      replication_factor => INT16
      topic_configs => { key value }
        key => COMPACT_STRING
        value => COMPACT_STRING
    copartition_groups => { (source_topics) (source_topic_regex) (repartition_s
      source_topics => INT16
      source_topic_regex => INT16
      repartition_source_topics => INT16
active_tasks => { subtopology_id (partitions) }
  subtopology_id => COMPACT_STRING
  partitions => INT32
standby_tasks => { subtopology_id (partitions) }
  subtopology_id => COMPACT_STRING
  partitions => INT32
warmup_tasks => { subtopology_id (partitions) }
  subtopology_id => COMPACT_STRING
  partitions => INT32
process_id => COMPACT_NULLABLE_STRING
user_endpoint => ?{ host port }
  host => COMPACT_STRING
  port => UINT16
client_tags => { key value }
  key => COMPACT_STRING
  value => COMPACT_STRING
task_offsets => { subtopology_id partition offset }
  subtopology_id => COMPACT_STRING
  partition => INT32
  offset => INT64
task_end_offsets => { subtopology_id partition offset }
  subtopology_id => COMPACT_STRING
  partition => INT32
  offset => INT64
shutdown_application => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION

group_id	The group identifier.
member_id	The member ID generated by the streams consumer. The member ID must be kept during the entire lifetime of the streams consumer process.
member_epoch	The current member epoch; 0 to join the group; -1 to leave the group; -2 to indicate that the static member will rejoin.
endpoint_information_epoch	The current endpoint epoch of this client, represents the latest endpoint epoch this client received
instance_id	null if not provided or if it didn't change since the last heartbeat; the instance ID for static membership otherwise.
rack_id	null if not provided or if it didn't change since the last heartbeat; the rack ID of the member otherwise.
rebalance_timeout_ms	-1 if it didn't change since the last heartbeat; the maximum time in milliseconds that the coordinator will wait on the member to revoke its tasks otherwise.
topology	The topology metadata of the streams application. Used to initialize the topology of the group and to check if the topology corresponds to the topology initialized for the group. Only sent when memberEpoch = 0, must be non-empty. Null otherwise.
epoch	The epoch of the topology. Used to check if the topology corresponds to the topology initialized on the brokers.
subtopologies	The sub-topologies of the streams application.
subtopology_id	String to uniquely identify the subtopology. Deterministically generated from the topology
source_topics	The topics the topology reads from.
source_topic_regex	The regular expressions identifying topics the subtopology reads from.
state_changelog_topics	The set of state changelog topics associated with this subtopology. Created automatically.

name	The name of the topic.
partitions	The number of partitions in the topic. Can be 0 if no specific number of partitions is enforced. Always 0 for changelog topics.
replication_factor	The replication factor of the topic. Can be 0 if the default replication factor should be used.
topic_configs	Topic-level configurations as key-value pairs.
key	key of the config
value	value of the config
repartition_sink_topics	The repartition topics the subtopology writes to.
repartition_source_topics	The set of source topics that are internally created repartition topics. Created automatically.
copartition_groups	A subset of source topics that must be copartitioned.
source_topics	The topics the topology reads from. Index into the array on the subtopology level.
source_topic_regex	Regular expressions identifying topics the subtopology reads from. Index into the array on the subtopology level.
repartition_source_topics	The set of source topics that are internally created repartition topics. Index into the array on the subtopology level.
active_tasks	Currently owned active tasks for this client. Null if unchanged since last heartbeat.
subtopology_id	The subtopology identifier.
partitions	The partitions of the input topics processed by this member.
standby_tasks	Currently owned standby tasks for this client. Null if unchanged since last heartbeat.

warmup_tasks	Currently owned warm-up tasks for this client. Null if unchanged since last heartbeat.
process_id	Identity of the streams instance that may have multiple consumers. Null if unchanged since last heartbeat.
user_endpoint	User-defined endpoint for Interactive Queries. Null if unchanged since last heartbeat, or if not defined on the client.
host	host of the endpoint
port	port of the endpoint
client_tags	Used for rack-aware assignment algorithm. Null if unchanged since last heartbeat.
task_offsets	Cumulative changelog offsets for tasks. Only updated when a warm-up task has caught up, and according to the task offset interval. Null if unchanged since last heartbeat.
subtopology_id	The subtopology identifier.
partition	The partition.
offset	The offset.
task_end_offsets	Cumulative changelog end-offsets for tasks. Only updated when a warm-up task has caught up, and according to the task offset interval. Null if unchanged since last heartbeat.
shutdown_application	Whether all Streams clients in the group should shut down.

Responses:

```
StreamsGroupHeartbeat Response (Version: 0) => { throttle_time_ms error_code erro
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  member_id => COMPACT_STRING
  member_epoch => INT32
  heartbeat_interval_ms => INT32
  acceptable_recovery_lag => INT32
  task_offset_interval_ms => INT32
```

```
  status => { status_code status_detail }
    status_code => INT8
    status_detail => COMPACT_STRING
  active_tasks => { subtopology_id (partitions) }
    subtopology_id => COMPACT_STRING
    partitions => INT32
  standby_tasks => { subtopology_id (partitions) }
    subtopology_id => COMPACT_STRING
    partitions => INT32
  warmup_tasks => { subtopology_id (partitions) }
    subtopology_id => COMPACT_STRING
    partitions => INT32
  endpoint_information_epoch => INT32
  partitions_by_user_endpoint => { user_endpoint (active_partitions) (standby_par
    user_endpoint => { host port }
      host => COMPACT_STRING
      port => UINT16
    active_partitions => { topic (partitions) }
      topic => COMPACT_STRING
      partitions => INT32
    standby_partitions => { topic (partitions) }
      topic => COMPACT_STRING
      partitions => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error
error_message	The top-level error message, or null if there was no error.
member_id	The member id is always generated by the streams consumer.
member_epoch	The member epoch.
heartbeat_interval_ms	The heartbeat interval in milliseconds.

acceptable_recovery_lag	The maximal lag a warm-up task can have to be considered caught-up.
task_offset_interval_ms	The interval in which the task changelog offsets on a client are updated on the broker. The offsets are sent with the next heartbeat after this time has passed.
status	Indicate zero or more status for the group.
status_code	A code to indicate that a particular status is active for the group membership
status_detail	A string representation of the status.
active_tasks	Assigned active tasks for this client. Null if unchanged since last heartbeat.
subtopology_id	The subtopology identifier.
partitions	The partitions of the input topics processed by this member.
standby_tasks	Assigned standby tasks for this client. Null if unchanged since last heartbeat.
warmup_tasks	Assigned warm-up tasks for this client. Null if unchanged since last heartbeat.
endpoint_information_epoch	The endpoint epoch set in the response
partitions_by_user_endpoint	Global assignment information used for IQ. Null if unchanged since last heartbeat.
user_endpoint	User-defined endpoint to connect to the node
host	host of the endpoint
port	port of the endpoint
active_partitions	All topic partitions materialized by active tasks on the node
topic	topic name
partitions	partitions

standby_partitions	All topic partitions materialized by standby tasks on the node

StreamsGroupDescribe API (Key: 89):

Requests:

```
StreamsGroupDescribe Request (Version: 0) => { (group_ids) include_authorized_ope
  group_ids => COMPACT_STRING
  include_authorized_operations => BOOLEAN
```

Request header version: 2

FIELD	DESCRIPTION
group_ids	The ids of the groups to describe
include_authorized_operations	Whether to include authorized operations.

Responses:

```
StreamsGroupDescribe Response (Version: 0) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { error_code error_message group_id group_state group_epoch assignmen
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
    group_id => COMPACT_STRING
    group_state => COMPACT_STRING
    group_epoch => INT32
    assignment_epoch => INT32
    topology => ?{ epoch ?(subtopologies) }
      epoch => INT32
      subtopologies => { subtopology_id (source_topics) (repartition_sink_topics)
        subtopology_id => COMPACT_STRING
        source_topics => COMPACT_STRING
        repartition_sink_topics => COMPACT_STRING
        state_changelog_topics => { name partitions replication_factor (topic_con
          name => COMPACT_STRING
          partitions => INT32
          replication_factor => INT16
          topic_configs => { key value }
            key => COMPACT_STRING
            value => COMPACT_STRING
```

```
    repartition_source_topics => { name partitions replication_factor (topic_
      name => COMPACT_STRING
      partitions => INT32
      replication_factor => INT16
      topic_configs => { key value }
        key => COMPACT_STRING
        value => COMPACT_STRING
  members => { member_id member_epoch instance_id rack_id client_id client_host
    member_id => COMPACT_STRING
    member_epoch => INT32
    instance_id => COMPACT_NULLABLE_STRING
    rack_id => COMPACT_NULLABLE_STRING
    client_id => COMPACT_STRING
    client_host => COMPACT_STRING
    topology_epoch => INT32
    process_id => COMPACT_STRING
    user_endpoint => ?{ host port }
      host => COMPACT_STRING
      port => UINT16
    client_tags => { key value }
      key => COMPACT_STRING
      value => COMPACT_STRING
    task_offsets => { subtopology_id partition offset }
      subtopology_id => COMPACT_STRING
      partition => INT32
      offset => INT64
    task_end_offsets => { subtopology_id partition offset }
      subtopology_id => COMPACT_STRING
      partition => INT32
      offset => INT64
    assignment => { (active_tasks) (standby_tasks) (warmup_tasks) }
      active_tasks => { subtopology_id (partitions) }
        subtopology_id => COMPACT_STRING
        partitions => INT32
      standby_tasks => { subtopology_id (partitions) }
        subtopology_id => COMPACT_STRING
        partitions => INT32
      warmup_tasks => { subtopology_id (partitions) }
        subtopology_id => COMPACT_STRING
        partitions => INT32
    target_assignment => { (active_tasks) (standby_tasks) (warmup_tasks) }
      active_tasks => { subtopology_id (partitions) }
        subtopology_id => COMPACT_STRING
        partitions => INT32
      standby_tasks => { subtopology_id (partitions) }
```

```
          subtopology_id => COMPACT_STRING
          partitions => INT32
        warmup_tasks => { subtopology_id (partitions) }
          subtopology_id => COMPACT_STRING
          partitions => INT32
      is_classic => BOOLEAN
    authorized_operations => INT32
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	Each described group.
error_code	The describe error, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
group_id	The group ID string.
group_state	The group state string, or the empty string.
group_epoch	The group epoch.
assignment_epoch	The assignment epoch.
topology	The topology metadata currently initialized for the streams application. Can be null in case of a describe error.
epoch	The epoch of the currently initialized topology for this group.
subtopologies	The subtopologies of the streams application. This contains the configured subtopologies, where the number of partitions are set and any regular expressions are resolved to actual topics. Null if the group is uninitialized, source topics are missing or incorrectly partitioned.
subtopology_id	String to uniquely identify the subtopology.
source_topics	The topics the subtopology reads from.

repartition_sink_topics	The repartition topics the subtopology writes to.
state_changelog_topics	The set of state changelog topics associated with this subtopology. Created automatically.
name	The name of the topic.
partitions	The number of partitions in the topic. Can be 0 if no specific number of partitions is enforced. Always 0 for changelog topics.
replication_factor	The replication factor of the topic. Can be 0 if the default replication factor should be used.
topic_configs	Topic-level configurations as key-value pairs.
key	key of the config
value	value of the config
repartition_source_topics	The set of source topics that are internally created repartition topics. Created automatically.
members	The members.
member_id	The member ID.
member_epoch	The member epoch.
instance_id	The member instance ID for static membership.
rack_id	The rack ID.
client_id	The client ID.
client_host	The client host.
topology_epoch	The epoch of the topology on the client.
process_id	Identity of the streams instance that may have multiple clients.
user_endpoint	User-defined endpoint for Interactive Queries. Null if not defined for this client.
host	host of the endpoint

port	port of the endpoint
client_tags	Used for rack-aware assignment algorithm.
task_offsets	Cumulative changelog offsets for tasks.
subtopology_id	The subtopology identifier.
partition	The partition.
offset	The offset.
task_end_offsets	Cumulative changelog end offsets for tasks.
assignment	The current assignment.
active_tasks	Active tasks for this client.
subtopology_id	The subtopology identifier.
partitions	The partitions of the input topics processed by this member.
standby_tasks	Standby tasks for this client.
warmup_tasks	Warm-up tasks for this client.
target_assignment	The target assignment.
is_classic	True for classic members that have not been upgraded yet.
authorized_operations	32-bit bitfield to represent authorized operations for this group.

DescribeShareGroupOffsets API (Key: 90):

Requests:

```
DescribeShareGroupOffsets Request (Version: 0) => { (groups) }
  groups => { group_id ?(topics) }
    group_id => COMPACT_STRING
    topics => { topic_name (partitions) }
      topic_name => COMPACT_STRING
      partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
groups	The groups to describe offsets for.
group_id	The group identifier.
topics	The topics to describe offsets for, or null for all topic-partitions.
topic_name	The topic name.
partitions	The partitions.

```
DescribeShareGroupOffsets Request (Version: 1) => { (groups) }
  groups => { group_id ?(topics) }
    group_id => COMPACT_STRING
    topics => { topic_name (partitions) }
      topic_name => COMPACT_STRING
      partitions => INT32
```

Request header version: 2

FIELD	DESCRIPTION
groups	The groups to describe offsets for.
group_id	The group identifier.
topics	The topics to describe offsets for, or null for all topic-partitions.
topic_name	The topic name.
partitions	The partitions.

Responses:

```
DescribeShareGroupOffsets Response (Version: 0) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { group_id (topics) error_code error_message }
    group_id => COMPACT_STRING
    topics => { topic_name topic_id (partitions) }
      topic_name => COMPACT_STRING
      topic_id => UUID
      partitions => { partition_index start_offset leader_epoch error_code error_
```

```
      partition_index => INT32
      start_offset => INT64
      leader_epoch => INT32
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	The results for each group.
group_id	The group identifier.
topics	The results for each topic.
topic_name	The topic name.
topic_id	The unique topic ID.
partitions	
partition_index	The partition index.
start_offset	The share-partition start offset.
leader_epoch	The leader epoch of the partition.
error_code	The partition-level error code, or 0 if there was no error.
error_message	The partition-level error message, or null if there was no error.
error_code	The group-level error code, or 0 if there was no error.
error_message	The group-level error message, or null if there was no error.

```
DescribeShareGroupOffsets Response (Version: 1) => { throttle_time_ms (groups) }
  throttle_time_ms => INT32
  groups => { group_id (topics) error_code error_message }
```

```
    group_id => COMPACT_STRING
    topics => { topic_name topic_id (partitions) }
      topic_name => COMPACT_STRING
      topic_id => UUID
      partitions => { partition_index start_offset leader_epoch lag error_code er
        partition_index => INT32
        start_offset => INT64
        leader_epoch => INT32
        lag => INT64
        error_code => INT16
        error_message => COMPACT_NULLABLE_STRING
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
groups	The results for each group.
group_id	The group identifier.
topics	The results for each topic.
topic_name	The topic name.
topic_id	The unique topic ID.
partitions	
partition_index	The partition index.
start_offset	The share-partition start offset.
leader_epoch	The leader epoch of the partition.
lag	The share-partition lag.
error_code	The partition-level error code, or 0 if there was no error.
error_message	The partition-level error message, or null if there was no error.

error_code	The group-level error code, or 0 if there was no error.
error_message	The group-level error message, or null if there was no error.

AlterShareGroupOffsets API (Key: 91):

Requests:

```
AlterShareGroupOffsets Request (Version: 0) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_name (partitions) }
    topic_name => COMPACT_STRING
    partitions => { partition_index start_offset }
      partition_index => INT32
      start_offset => INT64
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.
topics	The topics to alter offsets for.
topic_name	The topic name.
partitions	Each partition to alter offsets for.
partition_index	The partition index.
start_offset	The share-partition start offset.

Responses:

```
AlterShareGroupOffsets Response (Version: 0) => { throttle_time_ms error_code err
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  responses => { topic_name topic_id (partitions) }
    topic_name => COMPACT_STRING
    topic_id => UUID
    partitions => { partition_index error_code error_message }
      partition_index => INT32
```

```
      error_code => INT16
      error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
responses	The results for each topic.
topic_name	The topic name.
topic_id	The unique topic ID.
partitions	
partition_index	The partition index.
error_code	The error code, or 0 if there was no error.
error_message	The error message, or null if there was no error.

DeleteShareGroupOffsets API (Key: 92):

Requests:

```
DeleteShareGroupOffsets Request (Version: 0) => { group_id (topics) }
  group_id => COMPACT_STRING
  topics => { topic_name }
    topic_name => COMPACT_STRING
```

Request header version: 2

FIELD	DESCRIPTION
group_id	The group identifier.

topics	The topics to delete offsets for.
topic_name	The topic name.

Responses:

```
DeleteShareGroupOffsets Response (Version: 0) => { throttle_time_ms error_code er
  throttle_time_ms => INT32
  error_code => INT16
  error_message => COMPACT_NULLABLE_STRING
  responses => { topic_name topic_id error_code error_message }
    topic_name => COMPACT_STRING
    topic_id => UUID
    error_code => INT16
    error_message => COMPACT_NULLABLE_STRING
```

Response header version: 1

FIELD	DESCRIPTION
throttle_time_ms	The duration in milliseconds for which the request was throttled due to a quota violation, or zero if the request did not violate any quota.
error_code	The top-level error code, or 0 if there was no error.
error_message	The top-level error message, or null if there was no error.
responses	The results for each topic.
topic_name	The topic name.
topic_id	The unique topic ID.
error_code	The topic-level error code, or 0 if there was no error.
error_message	The topic-level error message, or null if there was no error.

Some Common Philosophical Questions

Some people have asked why we don't use HTTP. There are a number of reasons, the best is that client implementors can make use of some of the more advanced TCP features–the

ability to multiplex requests, the ability to simultaneously poll many connections, etc. We have also found HTTP libraries in many languages to be surprisingly shabby.

Others have asked if maybe we shouldn't support many different protocols. Prior experience with this was that it makes it very hard to add and test new features if they have to be ported across many protocol implementations. Our feeling is that most users don't really see multiple protocols as a feature, they just want a good reliable client in the language of their choice.

Another question is why we don't adopt XMPP, STOMP, AMQP or an existing protocol. The answer to this varies by protocol, but in general the problem is that the protocol does determine large parts of the implementation and we couldn't do what we are doing if we didn't have control over the protocol. Our belief is that it is possible to do better than existing messaging systems have in providing a truly distributed messaging system, and to do this we need to build something that works differently.

A final question is why we don't use a system like Protocol Buffers or Thrift to define our request messages. These packages excel at helping you to managing lots and lots of serialized messages. However we have only a few messages. Support across languages is somewhat spotty (depending on the package). Finally the mapping between binary log format and wire protocol is something we manage somewhat carefully and this would not be possible with these systems. Finally we prefer the style of versioning APIs explicitly and checking this to inferring new values as nulls as it allows more nuanced control of compatibility.